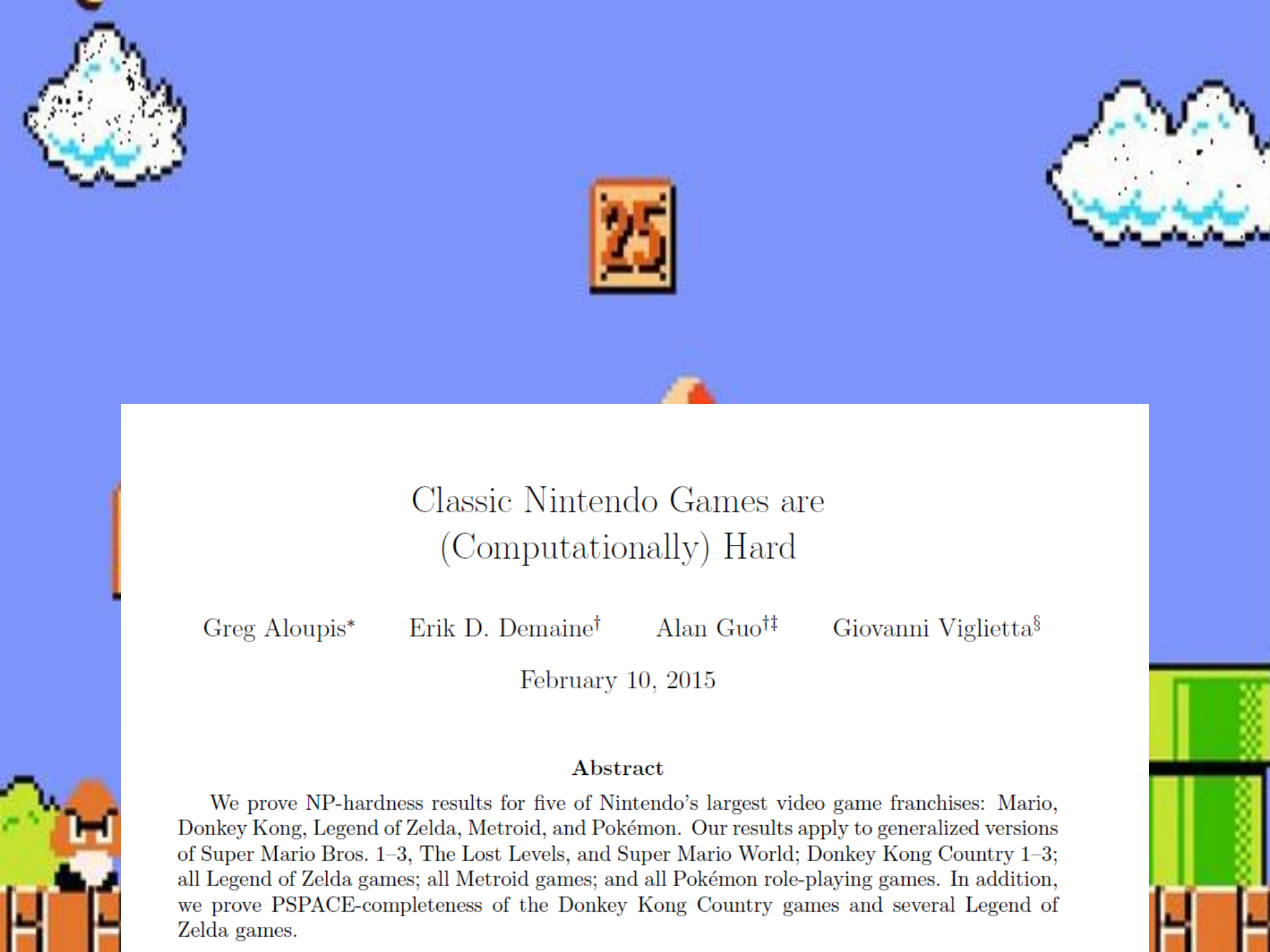


Some other polynomial-time reductions
proving NP-completeness is fun





Classic Nintendo Games are (Computationally) Hard

Greg Aloupis*

Erik D. Demaine[†]

Alan Guo^{†‡}

Giovanni Viglietta[§]

February 10, 2015

Abstract

We prove NP-hardness results for five of Nintendo's largest video game franchises: Mario, Donkey Kong, Legend of Zelda, Metroid, and Pokémon. Our results apply to generalized versions of Super Mario Bros. 1–3, The Lost Levels, and Super Mario World; Donkey Kong Country 1–3; all Legend of Zelda games; all Metroid games; and all Pokémon role-playing games. In addition, we prove PSPACE-completeness of the Donkey Kong Country games and several Legend of Zelda games.

Reduction from: 3-SAT

3-SAT

input: a Boolean formula consisting of 3-literal clauses over n variables

goal: does there exist a satisfying assignment (making all clauses true)?

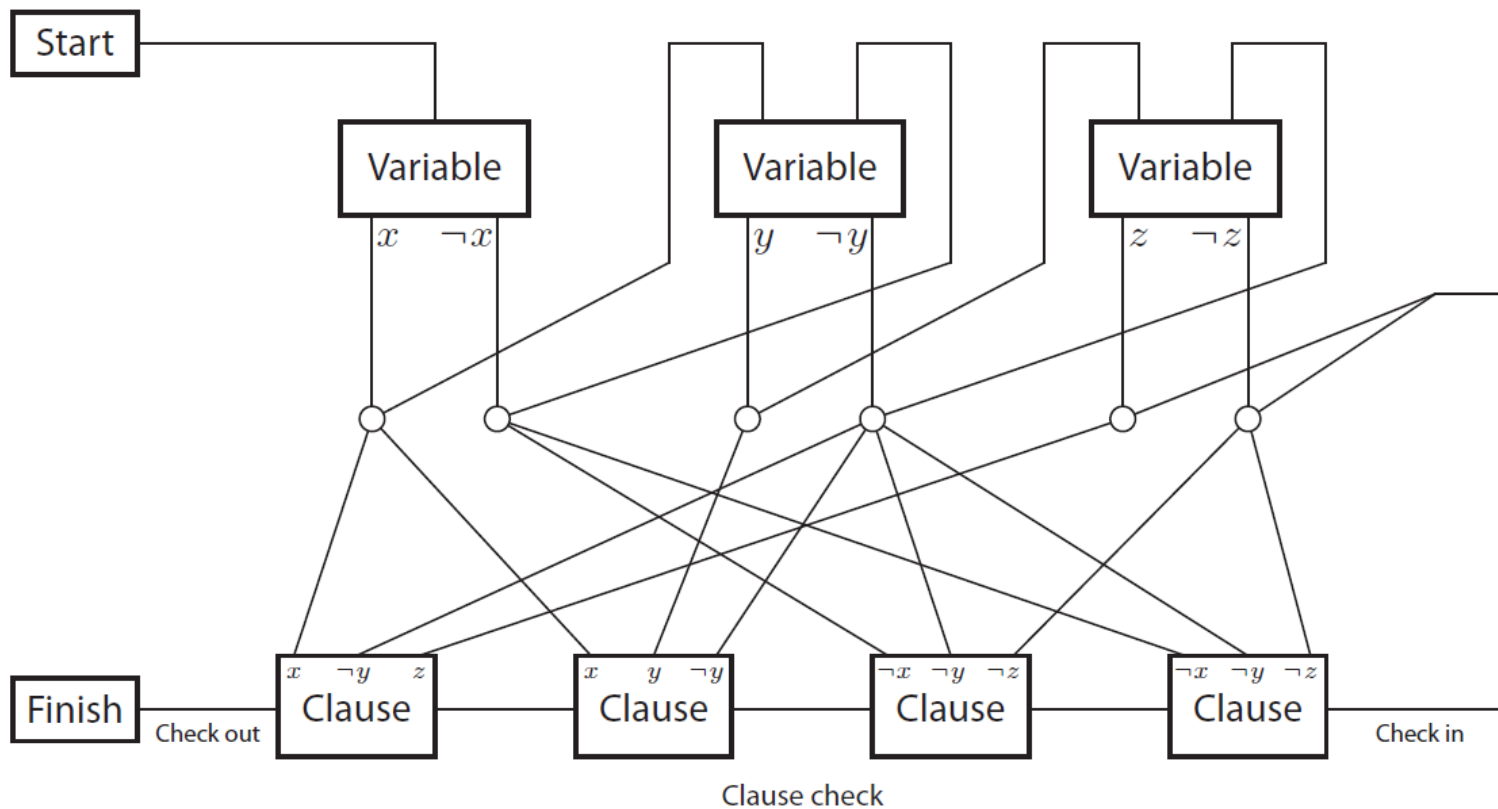
$$\Phi = (\overline{x_1} \vee x_2 \vee x_3) \wedge (x_1 \vee \overline{x_2} \vee x_3) \wedge (\overline{x_1} \vee x_2 \vee x_4)$$

yes instance: $x_1 = \text{true}$, $x_2 = \text{true}$, $x_3 = \text{false}$, $x_4 = \text{false}$

3-SAT is NP-complete

idea:

given a 3-SAT instance we build a level/instance of Super Mario that is solvable if and only if the formula is satisfiable



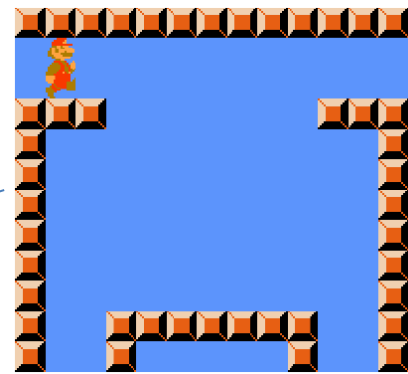


Start

Variable

Variable

Variable



x $\neg x$

y $\neg y$

z $\neg z$

Finish

Check out

x $\neg y$ z
Clause

x y $\neg y$
Clause

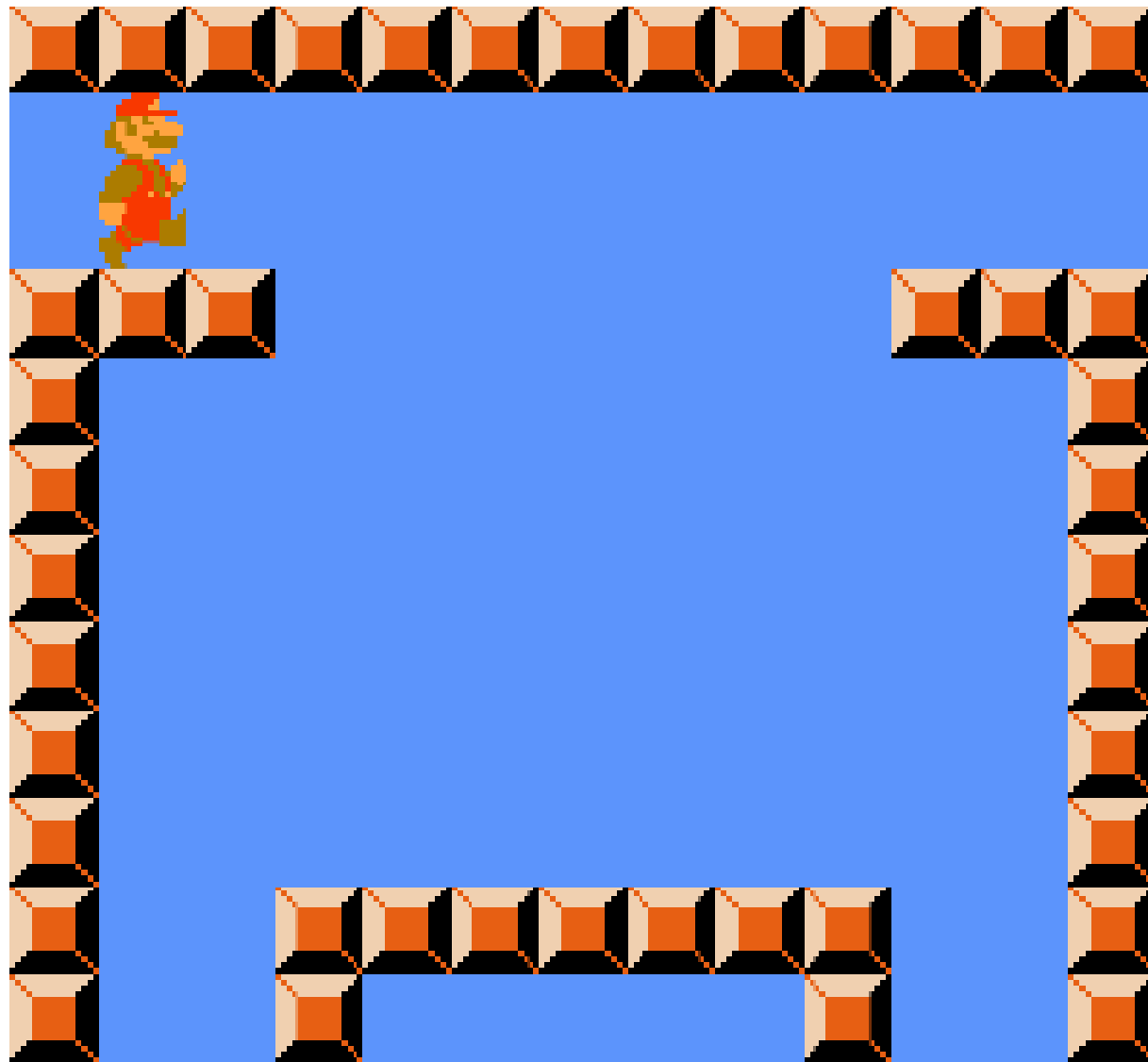
$\neg x$ $\neg y$ $\neg z$
Clause

$\neg x$ $\neg y$ $\neg z$
Clause

Check in

Clause check

variable gadget



x_i

$\neg x_i$



Start

Variable

$x \quad \neg x$

Variable

$y \quad \neg y$

Variable

$z \quad \neg z$

Finish

Check out

$x \quad \neg y \quad z$
Clause

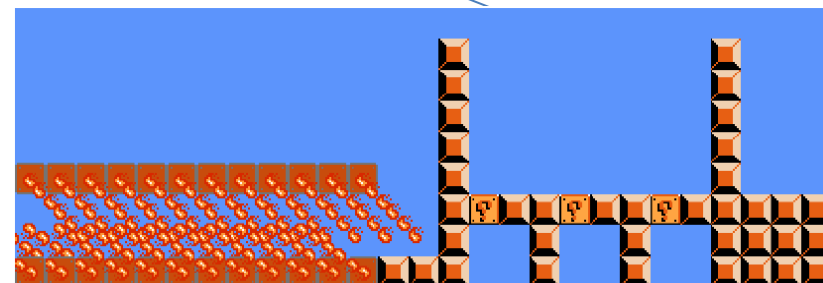
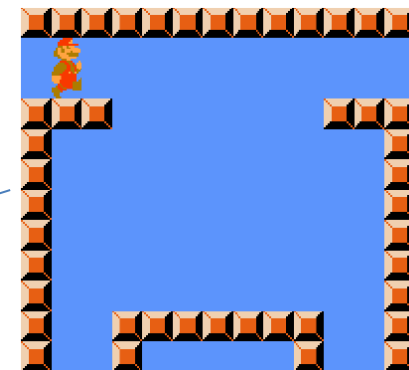
$x \quad y \quad \neg y$
Clause

$\neg x \quad \neg y \quad \neg z$
Clause

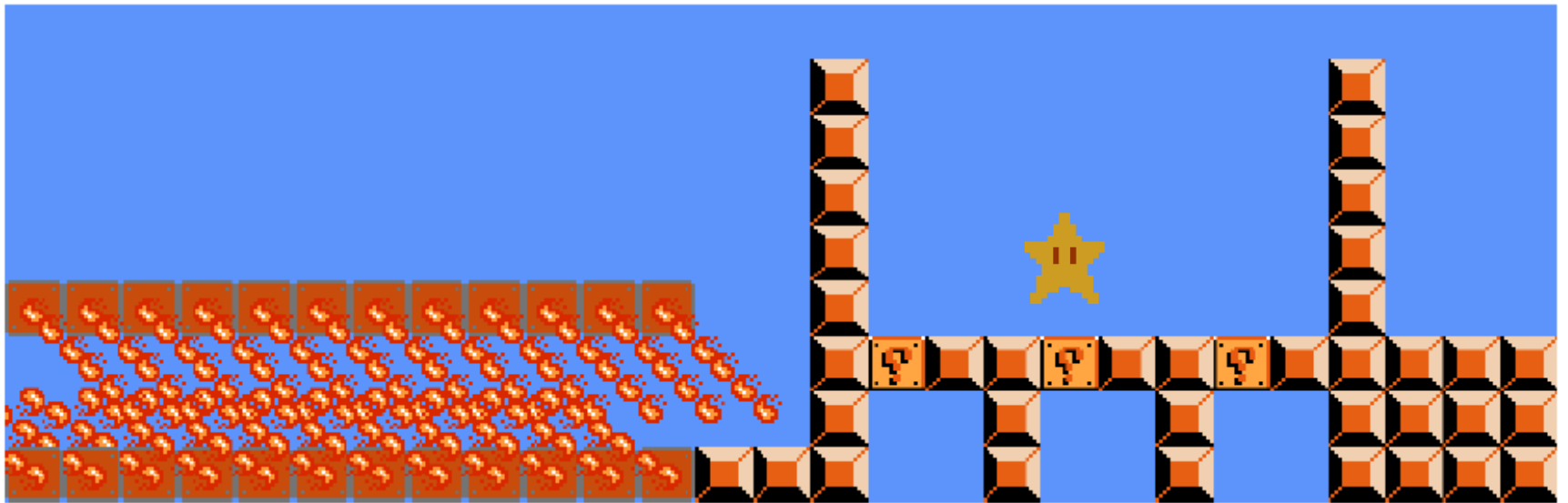
$\neg x \quad \neg y \quad \neg z$
Clause

Check in

Clause check



Clause gadget





Start

Variable

$x \quad \neg x$

Variable

$y \quad \neg y$

Variable

$z \quad \neg z$

Finish

Check out

$x \quad \neg y \quad z$
Clause

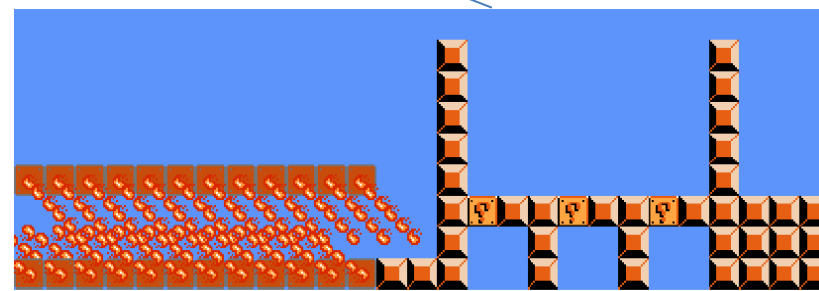
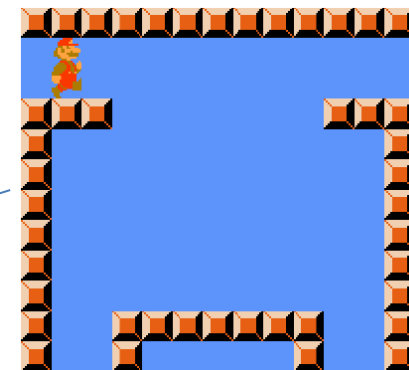
$x \quad y \quad \neg y$
Clause

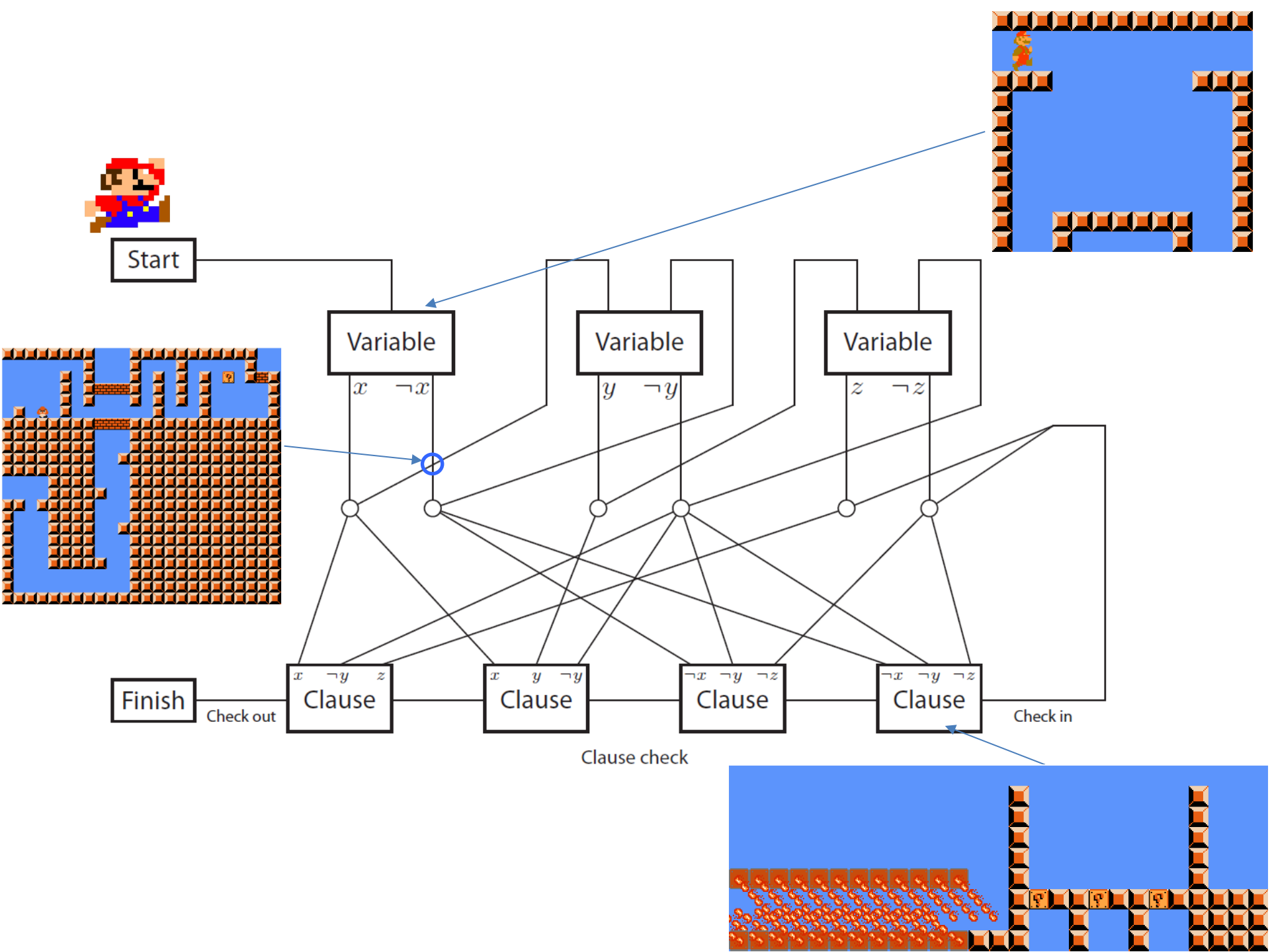
$\neg x \quad \neg y \quad \neg z$
Clause

$\neg x \quad \neg y \quad \neg z$
Clause

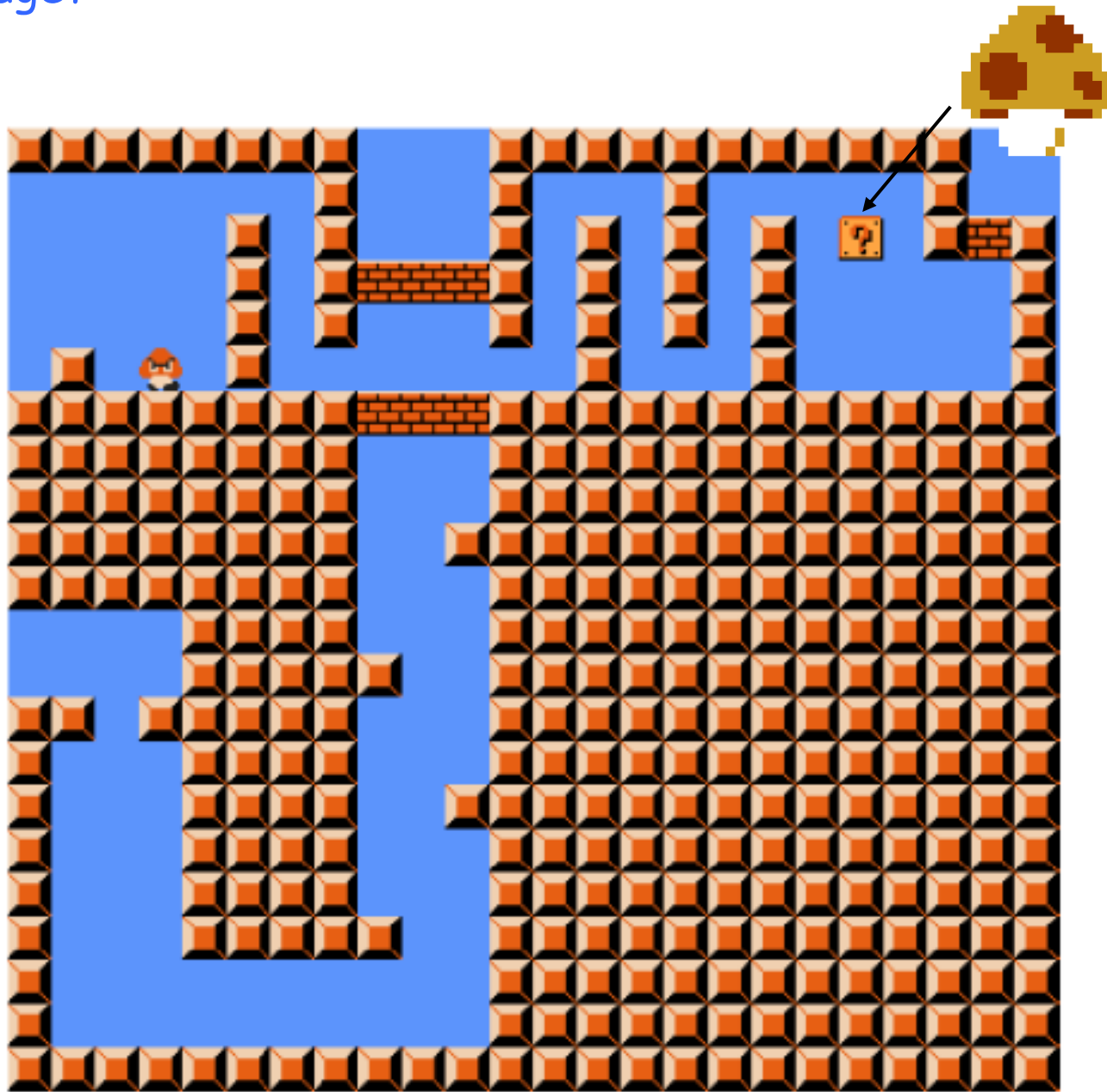
Check in

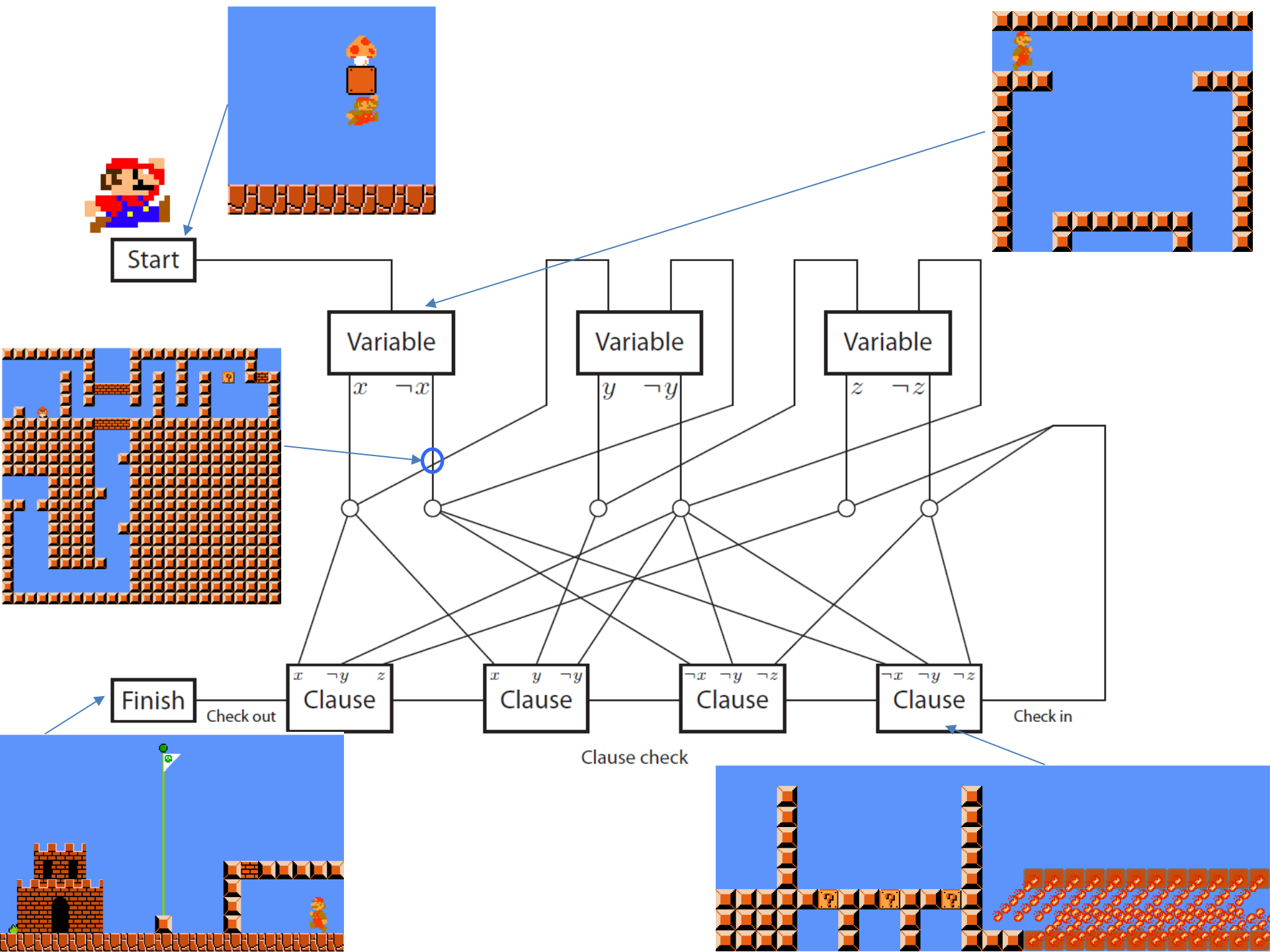
Clause check





Crossover gadget



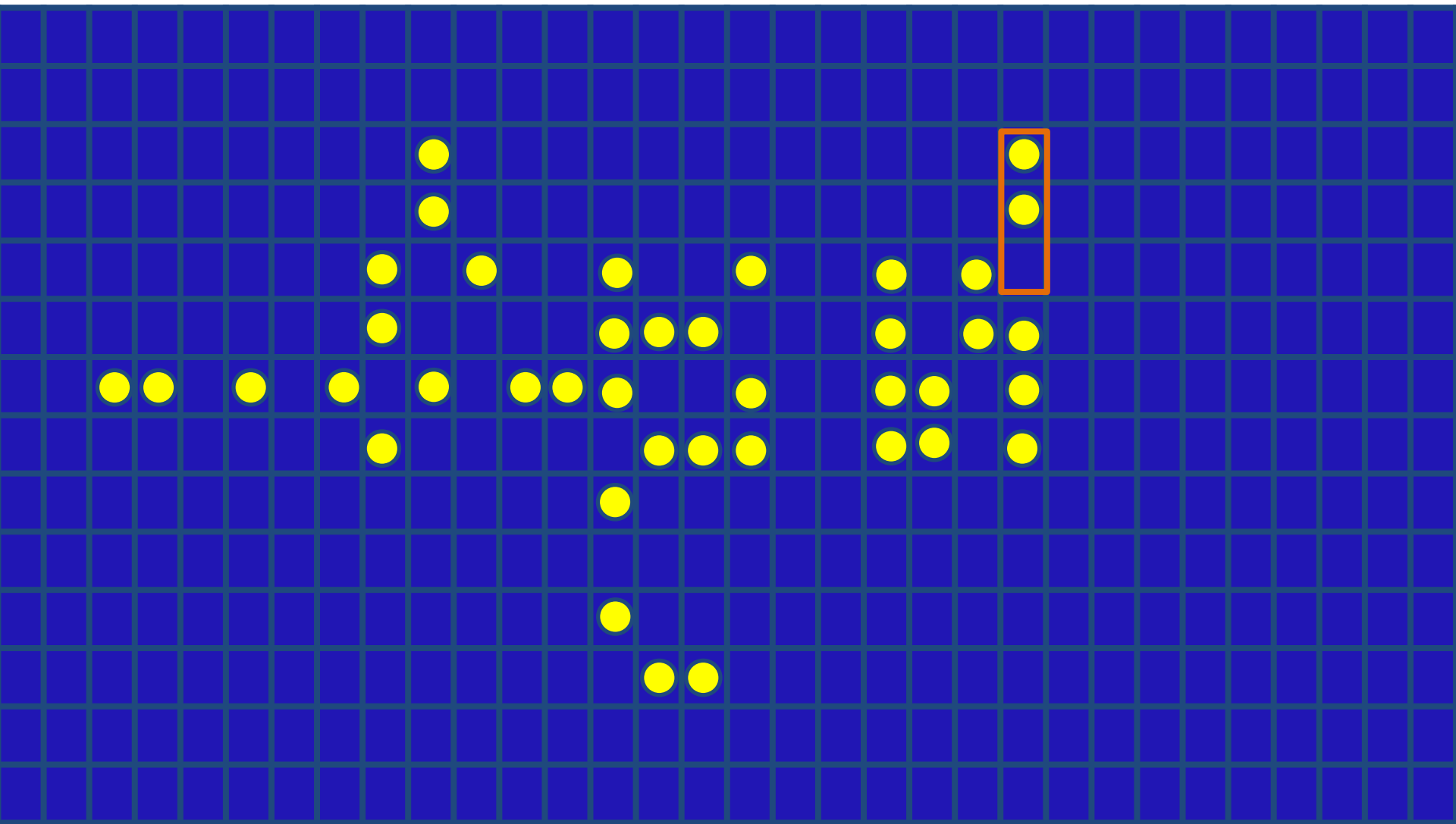


Peg Solitaire

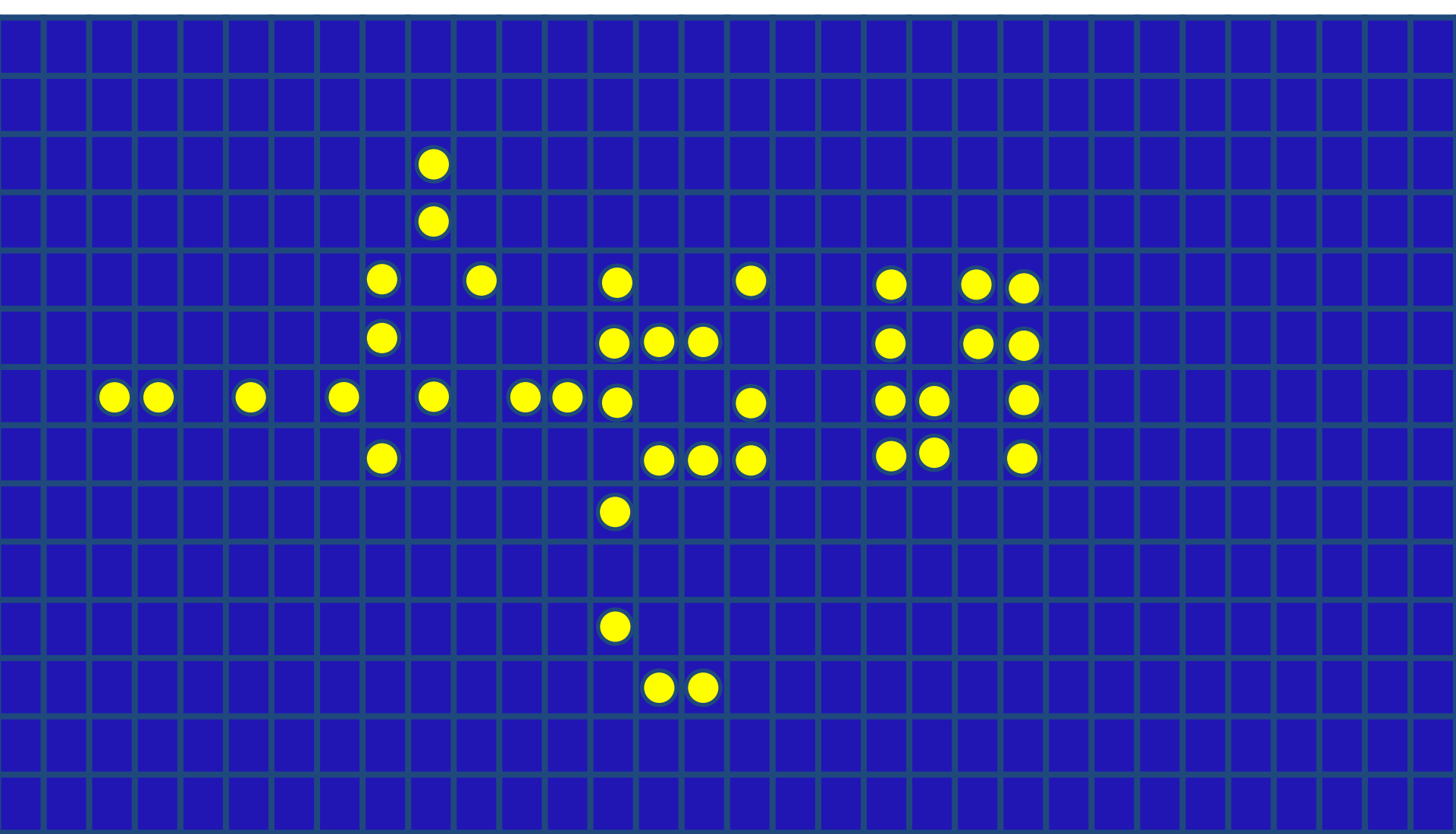


A 20x20 grid with a dark blue background and a light blue grid. Yellow dots are placed at various intersections. A red rectangle highlights two dots at the top left. The dots are located at the following coordinates (row, column) starting from the top left: (1, 1), (1, 2), (2, 3), (3, 2), (3, 4), (4, 1), (4, 2), (4, 3), (4, 4), (4, 5), (4, 6), (4, 7), (4, 8), (4, 9), (4, 10), (4, 11), (4, 12), (4, 13), (4, 14), (4, 15), (4, 16), (4, 17), (4, 18), (4, 19), (5, 2), (5, 3), (5, 4), (5, 5), (5, 6), (5, 7), (5, 8), (5, 9), (5, 10), (5, 11), (5, 12), (5, 13), (5, 14), (5, 15), (5, 16), (5, 17), (5, 18), (5, 19), (6, 1), (6, 2), (6, 3), (6, 4), (6, 5), (6, 6), (6, 7), (6, 8), (6, 9), (6, 10), (6, 11), (6, 12), (6, 13), (6, 14), (6, 15), (6, 16), (6, 17), (6, 18), (6, 19), (7, 1), (7, 2), (7, 3), (7, 4), (7, 5), (7, 6), (7, 7), (7, 8), (7, 9), (7, 10), (7, 11), (7, 12), (7, 13), (7, 14), (7, 15), (7, 16), (7, 17), (7, 18), (7, 19), (8, 1), (8, 2), (8, 3), (8, 4), (8, 5), (8, 6), (8, 7), (8, 8), (8, 9), (8, 10), (8, 11), (8, 12), (8, 13), (8, 14), (8, 15), (8, 16), (8, 17), (8, 18), (8, 19), (9, 1), (9, 2), (9, 3), (9, 4), (9, 5), (9, 6), (9, 7), (9, 8), (9, 9), (9, 10), (9, 11), (9, 12), (9, 13), (9, 14), (9, 15), (9, 16), (9, 17), (9, 18), (9, 19), (10, 1), (10, 2), (10, 3), (10, 4), (10, 5), (10, 6), (10, 7), (10, 8), (10, 9), (10, 10), (10, 11), (10, 12), (10, 13), (10, 14), (10, 15), (10, 16), (10, 17), (10, 18), (10, 19), (11, 1), (11, 2), (11, 3), (11, 4), (11, 5), (11, 6), (11, 7), (11, 8), (11, 9), (11, 10), (11, 11), (11, 12), (11, 13), (11, 14), (11, 15), (11, 16), (11, 17), (11, 18), (11, 19), (12, 1), (12, 2), (12, 3), (12, 4), (12, 5), (12, 6), (12, 7), (12, 8), (12, 9), (12, 10), (12, 11), (12, 12), (12, 13), (12, 14), (12, 15), (12, 16), (12, 17), (12, 18), (12, 19), (13, 1), (13, 2), (13, 3), (13, 4), (13, 5), (13, 6), (13, 7), (13, 8), (13, 9), (13, 10), (13, 11), (13, 12), (13, 13), (13, 14), (13, 15), (13, 16), (13, 17), (13, 18), (13, 19), (14, 1), (14, 2), (14, 3), (14, 4), (14, 5), (14, 6), (14, 7), (14, 8), (14, 9), (14, 10), (14, 11), (14, 12), (14, 13), (14, 14), (14, 15), (14, 16), (14, 17), (14, 18), (14, 19), (15, 1), (15, 2), (15, 3), (15, 4), (15, 5), (15, 6), (15, 7), (15, 8), (15, 9), (15, 10), (15, 11), (15, 12), (15, 13), (15, 14), (15, 15), (15, 16), (15, 17), (15, 18), (15, 19), (16, 1), (16, 2), (16, 3), (16, 4), (16, 5), (16, 6), (16, 7), (16, 8), (16, 9), (16, 10), (16, 11), (16, 12), (16, 13), (16, 14), (16, 15), (16, 16), (16, 17), (16, 18), (16, 19), (17, 1), (17, 2), (17, 3), (17, 4), (17, 5), (17, 6), (17, 7), (17, 8), (17, 9), (17, 10), (17, 11), (17, 12), (17, 13), (17, 14), (17, 15), (17, 16), (17, 17), (17, 18), (17, 19), (18, 1), (18, 2), (18, 3), (18, 4), (18, 5), (18, 6), (18, 7), (18, 8), (18, 9), (18, 10), (18, 11), (18, 12), (18, 13), (18, 14), (18, 15), (18, 16), (18, 17), (18, 18), (18, 19), (19, 1), (19, 2), (19, 3), (19, 4), (19, 5), (19, 6), (19, 7), (19, 8), (19, 9), (19, 10), (19, 11), (19, 12), (19, 13), (19, 14), (19, 15), (19, 16), (19, 17), (19, 18), (19, 19), (20, 1), (20, 2), (20, 3), (20, 4), (20, 5), (20, 6), (20, 7), (20, 8), (20, 9), (20, 10), (20, 11), (20, 12), (20, 13), (20, 14), (20, 15), (20, 16), (20, 17), (20, 18), (20, 19), (21, 1), (21, 2), (21, 3), (21, 4), (21, 5), (21, 6), (21, 7), (21, 8), (21, 9), (21, 10), (21, 11), (21, 12), (21, 13), (21, 14), (21, 15), (21, 16), (21, 17), (21, 18), (21, 19), (22, 1), (22, 2), (22, 3), (22, 4), (22, 5), (22, 6), (22, 7), (22, 8), (22, 9), (22, 10), (22, 11), (22, 12), (22, 13), (22, 14), (22, 15), (22, 16), (22, 17), (22, 18), (22, 19), (23, 1), (23, 2), (23, 3), (23, 4), (23, 5), (23, 6), (23, 7), (23, 8), (23, 9), (23, 10), (23, 11), (23, 12), (23, 13), (23, 14), (23, 15), (23, 16), (23, 17), (23, 18), (23, 19), (24, 1), (24, 2), (24, 3), (24, 4), (24, 5), (24, 6), (24, 7), (24, 8), (24, 9), (24, 10), (24, 11), (24, 12), (24, 13), (24, 14), (24, 15), (24, 16), (24, 17), (24, 18), (24, 19), (25, 1), (25, 2), (25, 3), (25, 4), (25, 5), (25, 6), (25, 7), (25, 8), (25, 9), (25, 10), (25, 11), (25, 12), (25, 13), (25, 14), (25, 15), (25, 16), (25, 17), (25, 18), (25, 19), (26, 1), (26, 2), (26, 3), (26, 4), (26, 5), (26, 6), (26, 7), (26, 8), (26, 9), (26, 10), (26, 11), (26, 12), (26, 13), (26, 14), (26, 15), (26, 16), (26, 17), (26, 18), (26, 19), (27, 1), (27, 2), (27, 3), (27, 4), (27, 5), (27, 6), (27, 7), (27, 8), (27, 9), (27, 10), (27, 11), (27, 12), (27, 13), (27, 14), (27, 15), (27, 16), (27, 17), (27, 18), (27, 19), (28, 1), (28, 2), (28, 3), (28, 4), (28, 5), (28, 6), (28, 7), (28, 8), (28, 9), (28, 10), (28, 11), (28, 12), (28, 13), (28, 14), (28, 15), (28, 16), (28, 17), (28, 18), (28, 19), (29, 1), (29, 2), (29, 3), (29, 4), (29, 5), (29, 6), (29, 7), (29, 8), (29, 9), (29, 10), (29, 11), (29, 12), (29, 13), (29, 14), (29, 15), (29, 16), (29, 17), (29, 18), (29, 19), (30, 1), (30, 2), (30, 3), (30, 4), (30, 5), (30, 6), (30, 7), (30, 8), (30, 9), (30, 10), (30, 11), (30, 12), (30, 13), (30, 14), (30, 15), (30, 16), (30, 17), (30, 18), (30, 19), (31, 1), (31, 2), (31, 3), (31, 4), (31, 5), (31, 6), (31, 7), (31, 8), (31, 9), (31, 10), (31, 11), (31, 12), (31, 13), (31, 14), (31, 15), (31, 16), (31, 17), (31, 18), (31, 19), (32, 1), (32, 2), (32, 3), (32, 4), (32, 5), (32, 6), (32, 7), (32, 8), (32, 9), (32, 10), (32, 11), (32, 12), (32, 13),

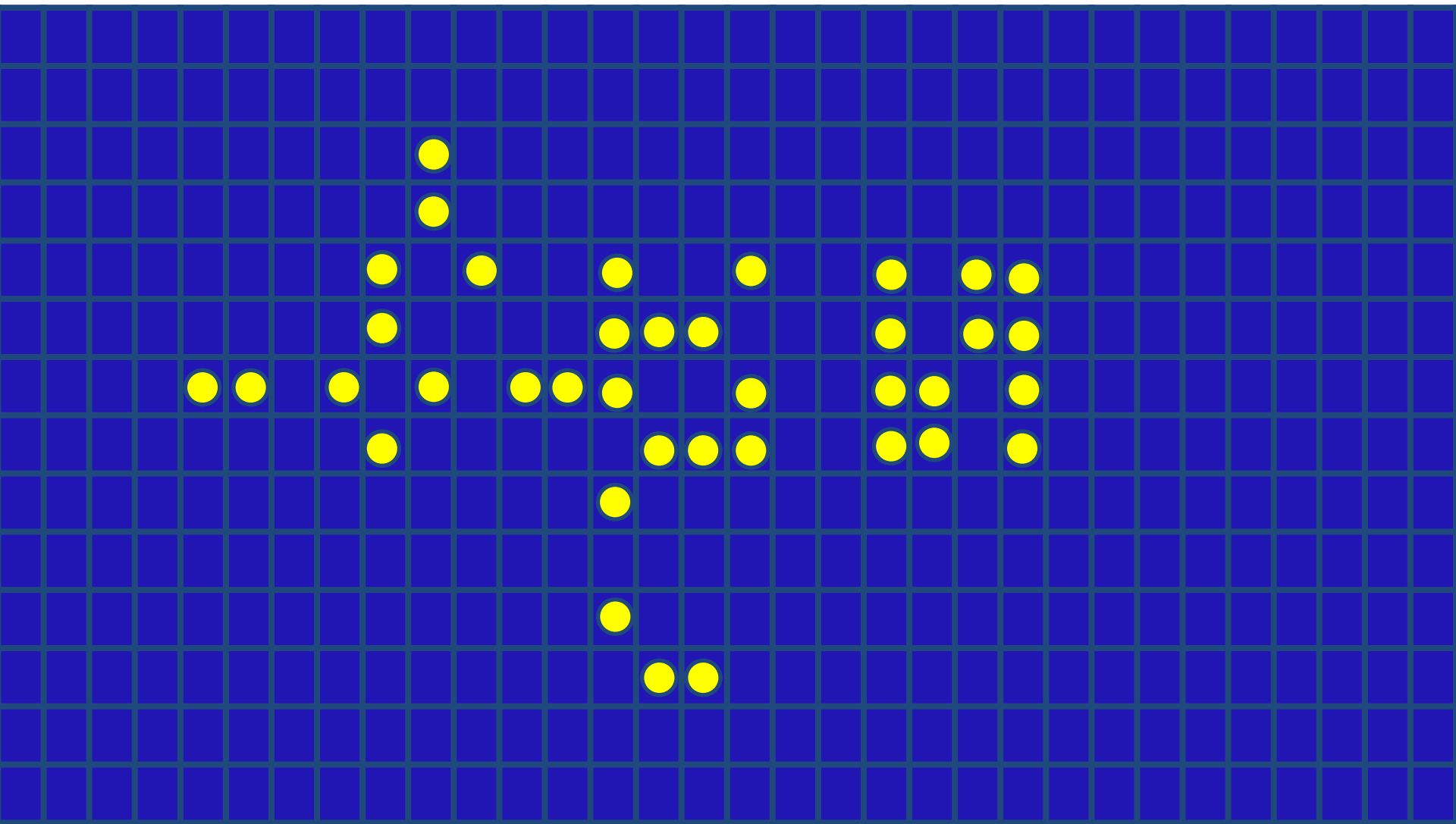
Peg Solitaire



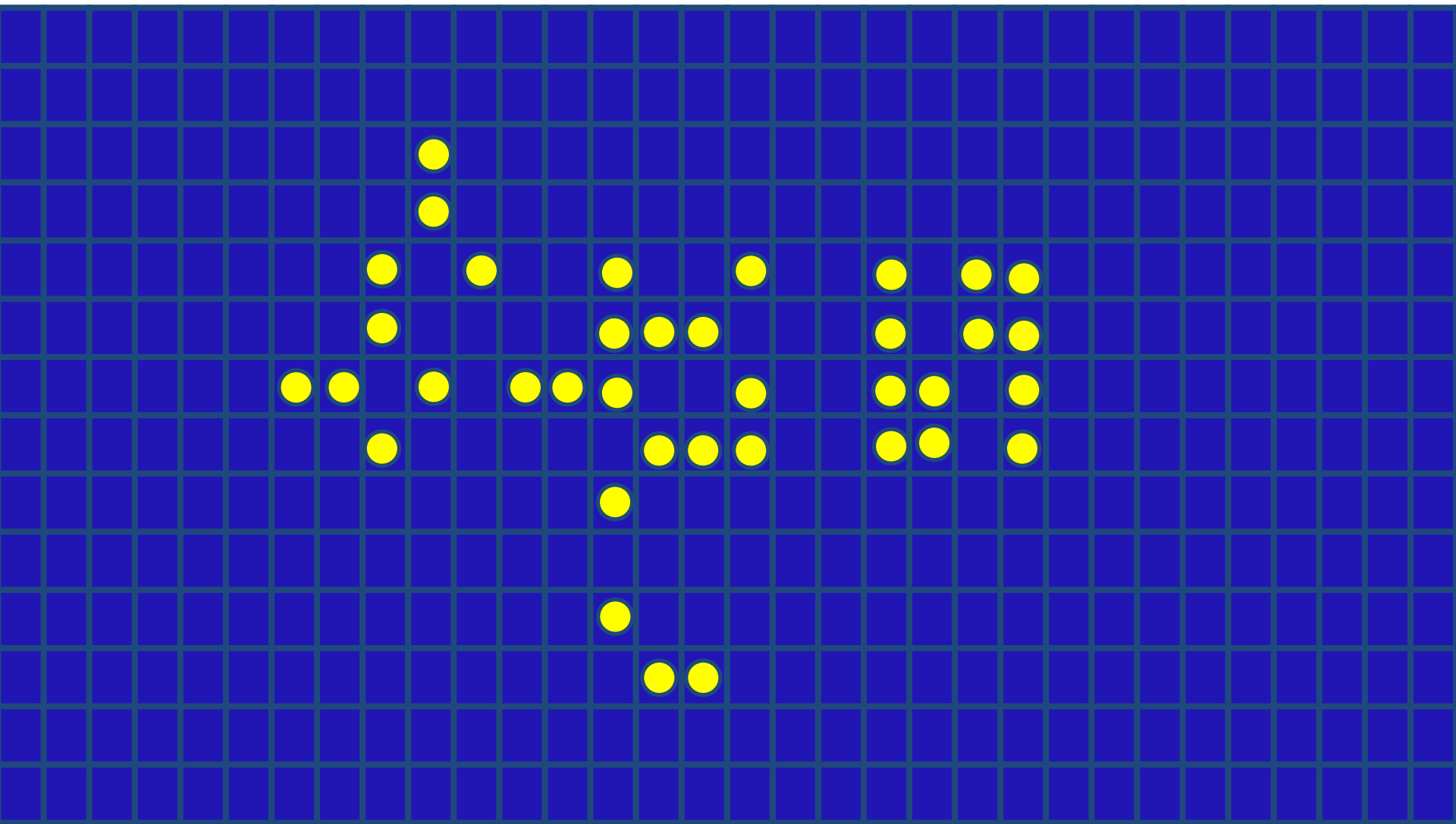
Peg Solitaire



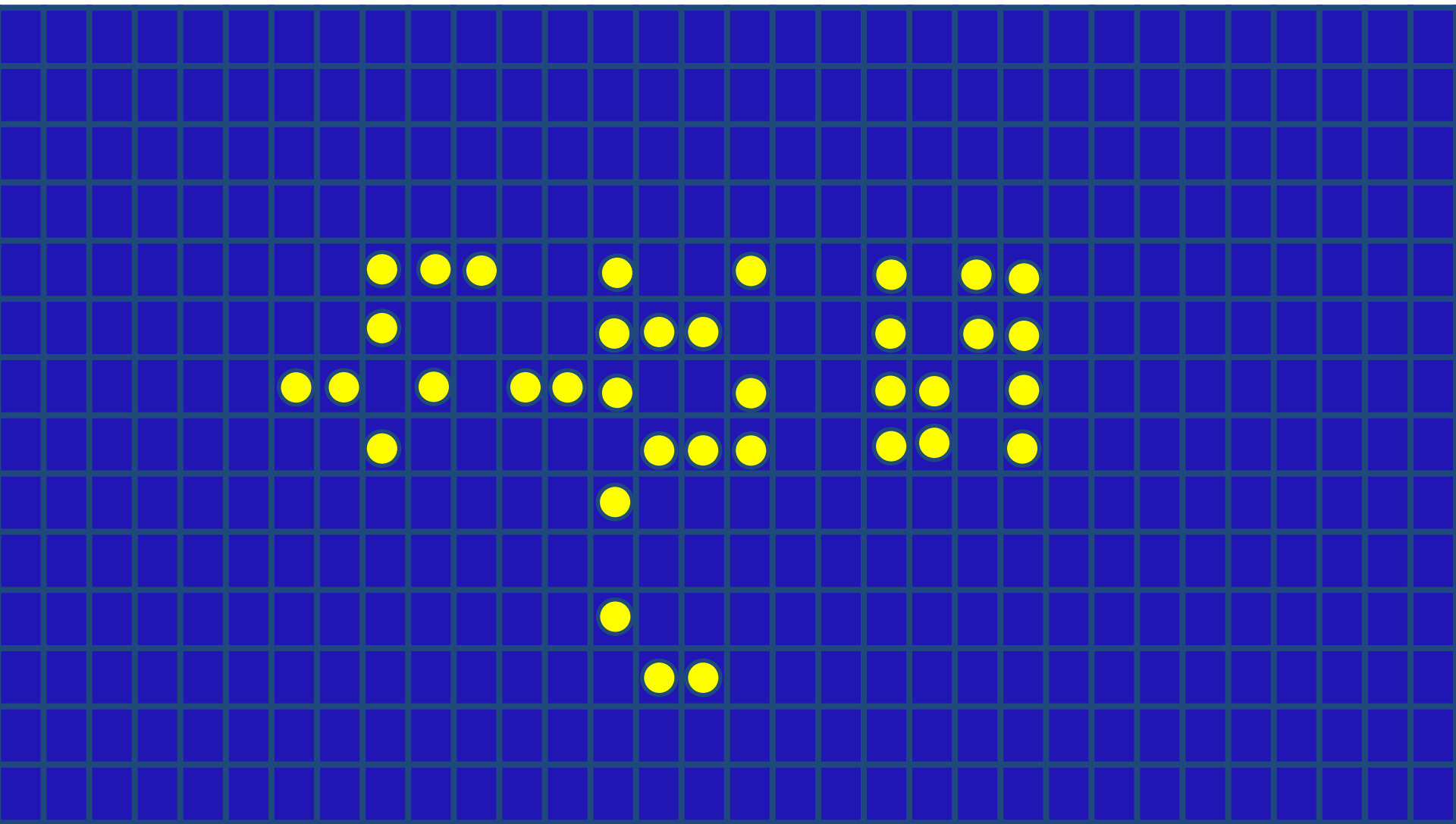
Peg Solitaire



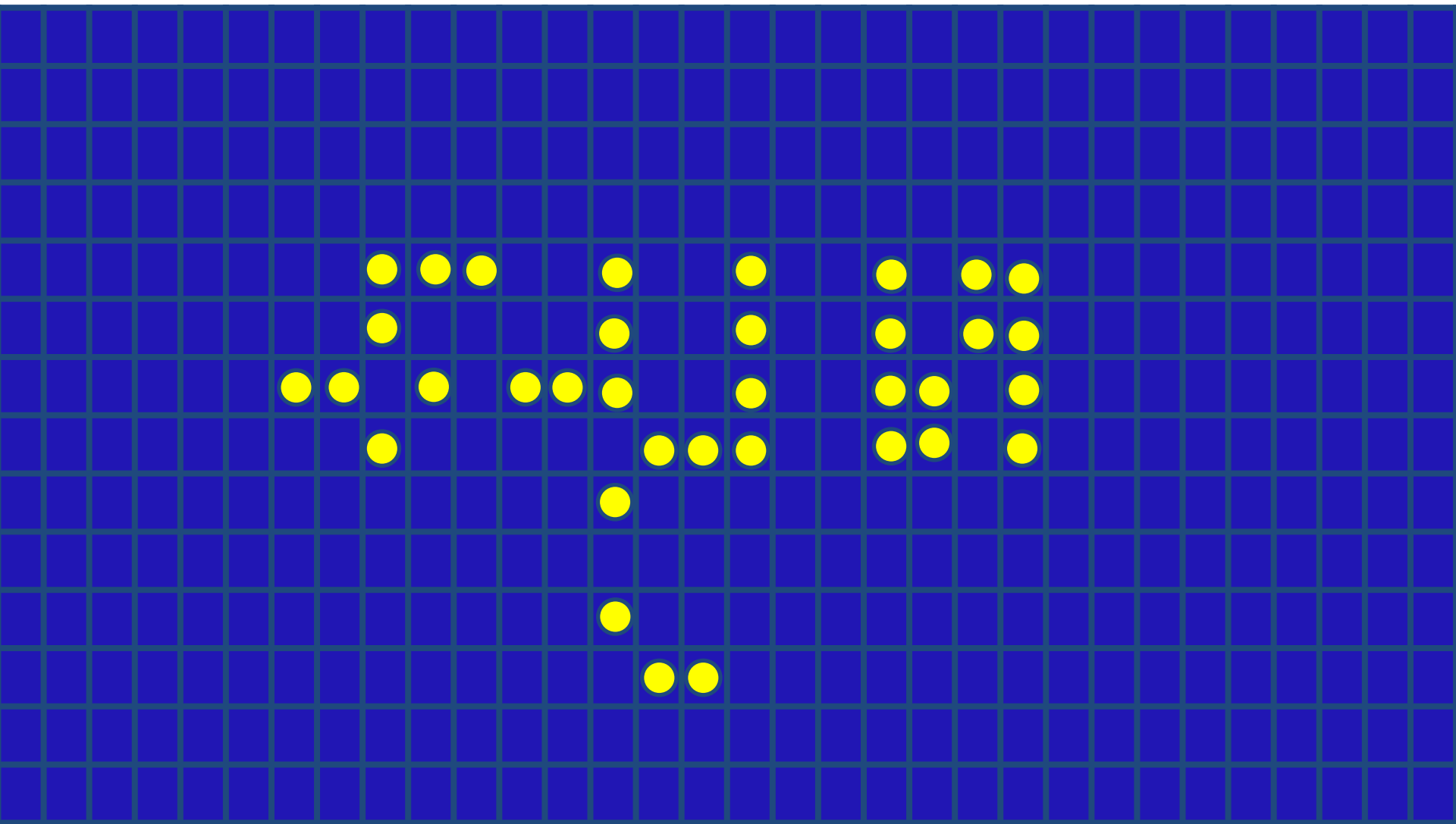
Peg Solitaire



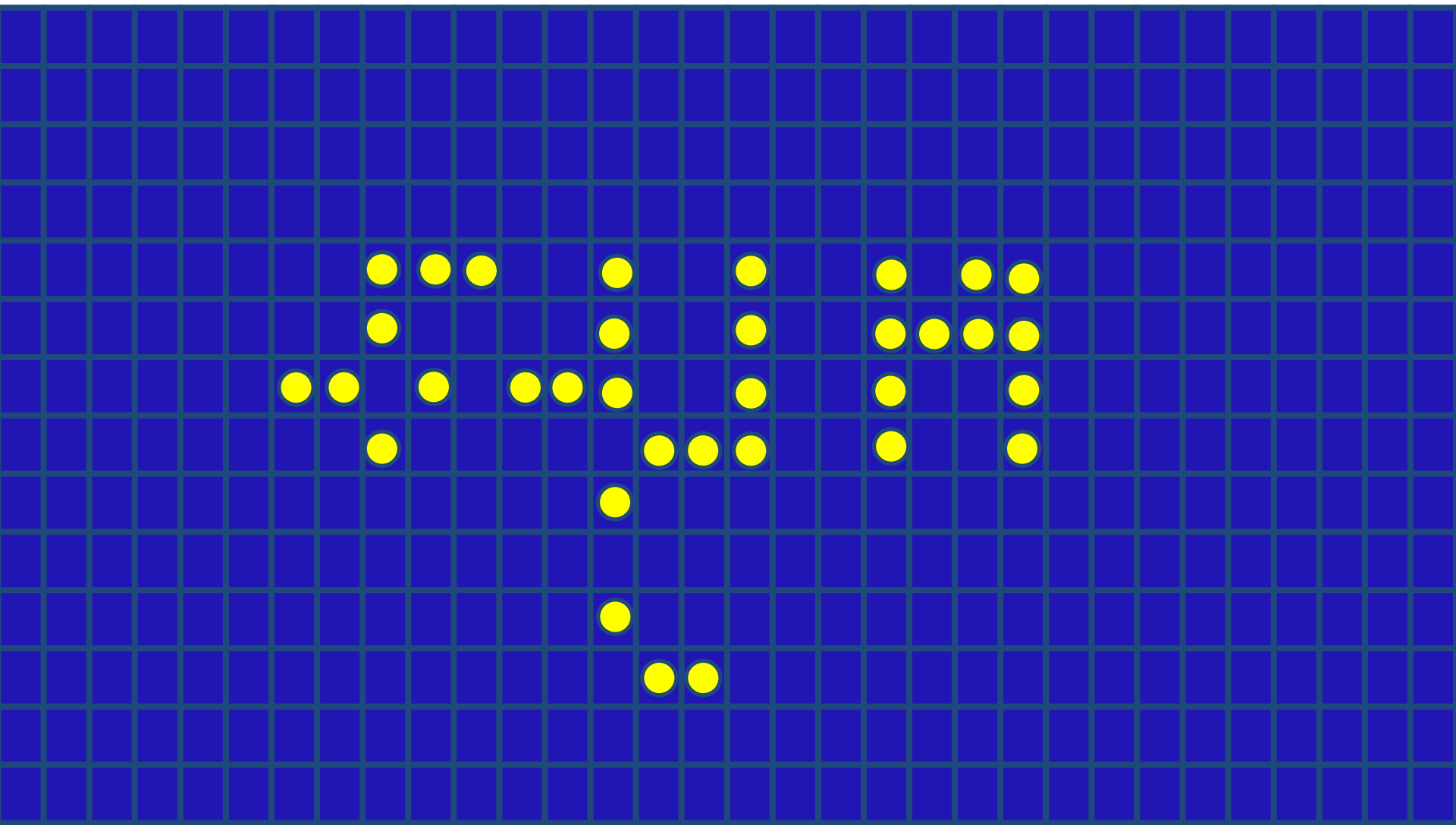
Peg Solitaire



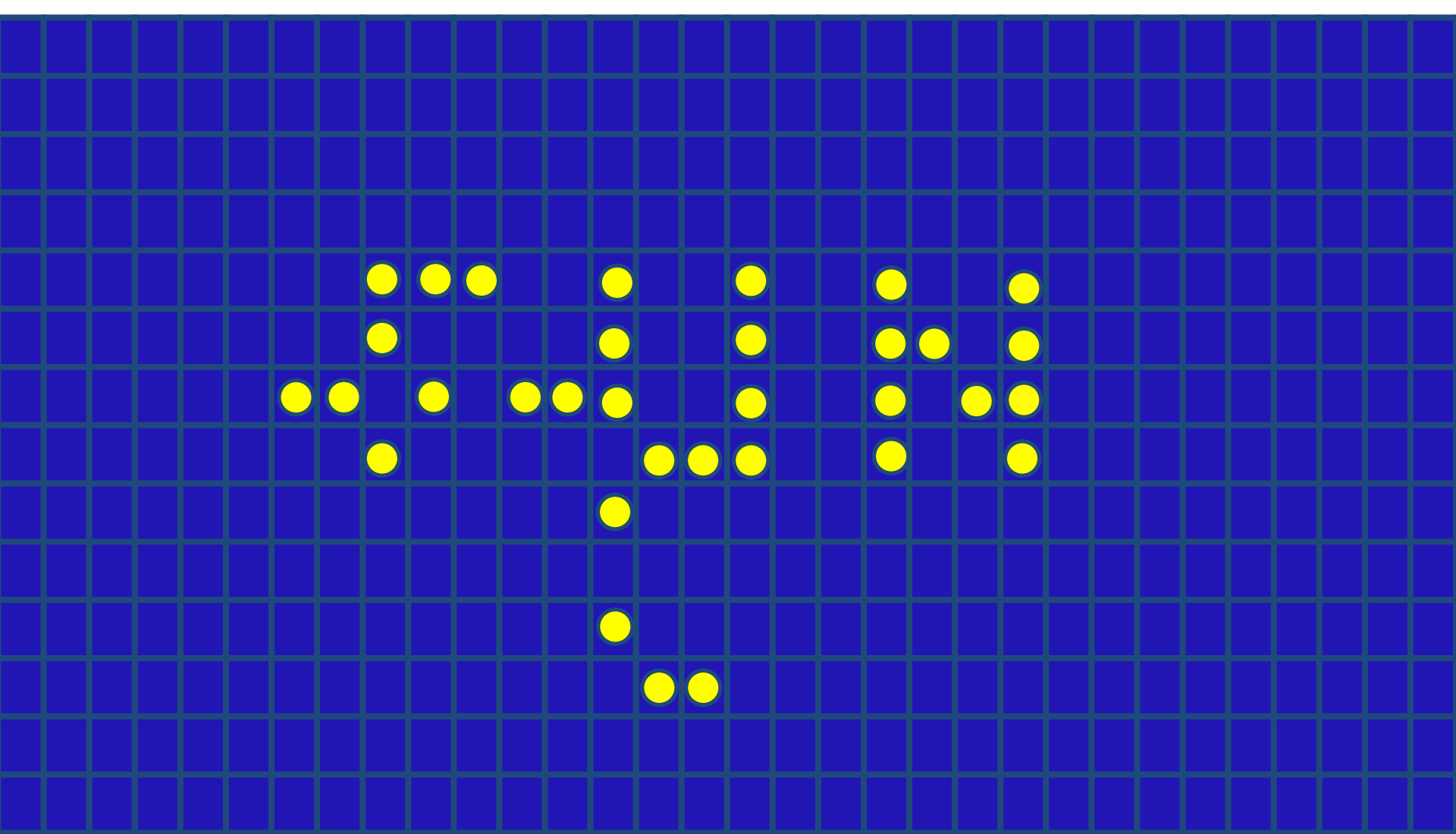
Peg Solitaire



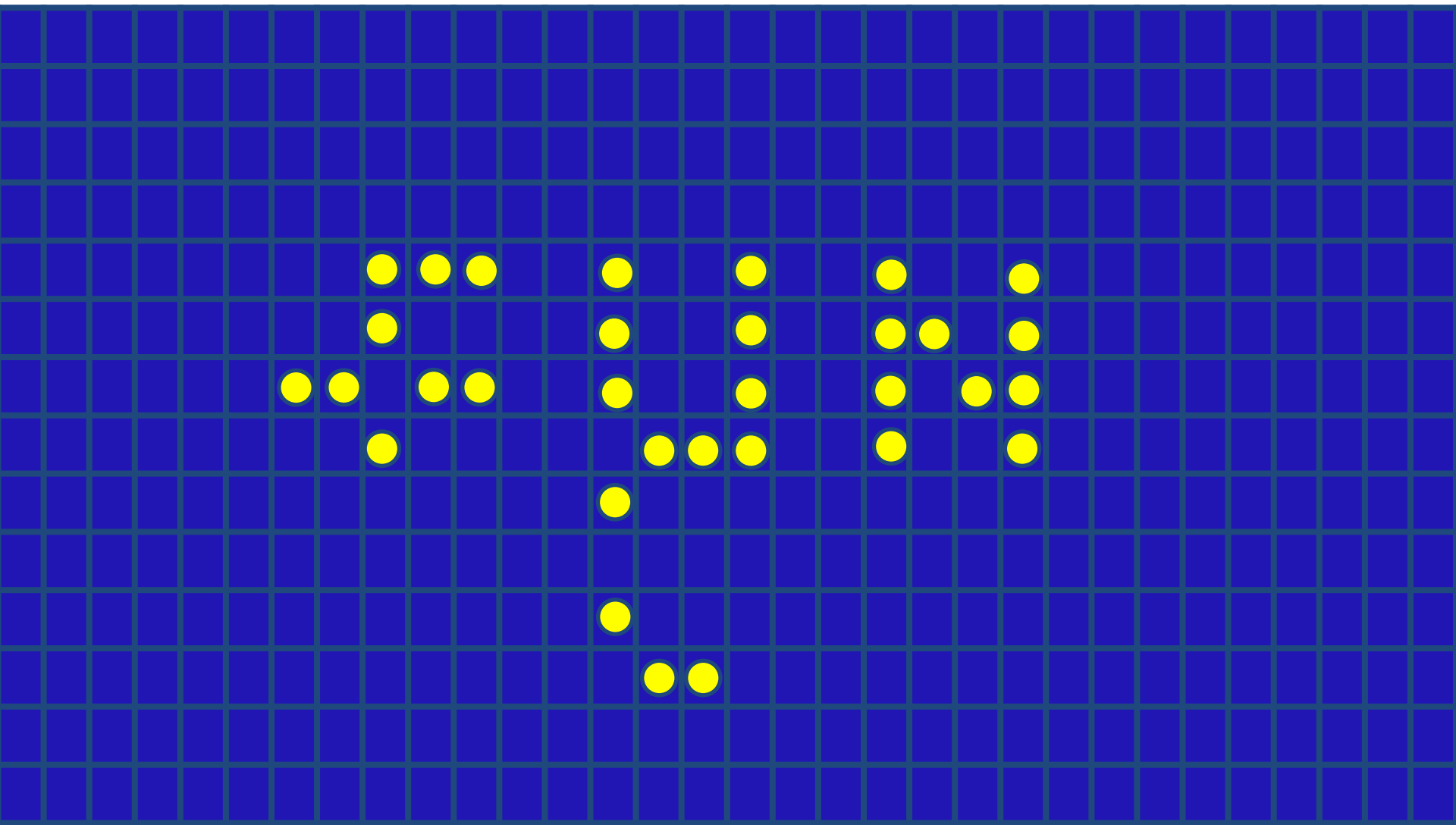
Peg Solitaire



Peg Solitaire

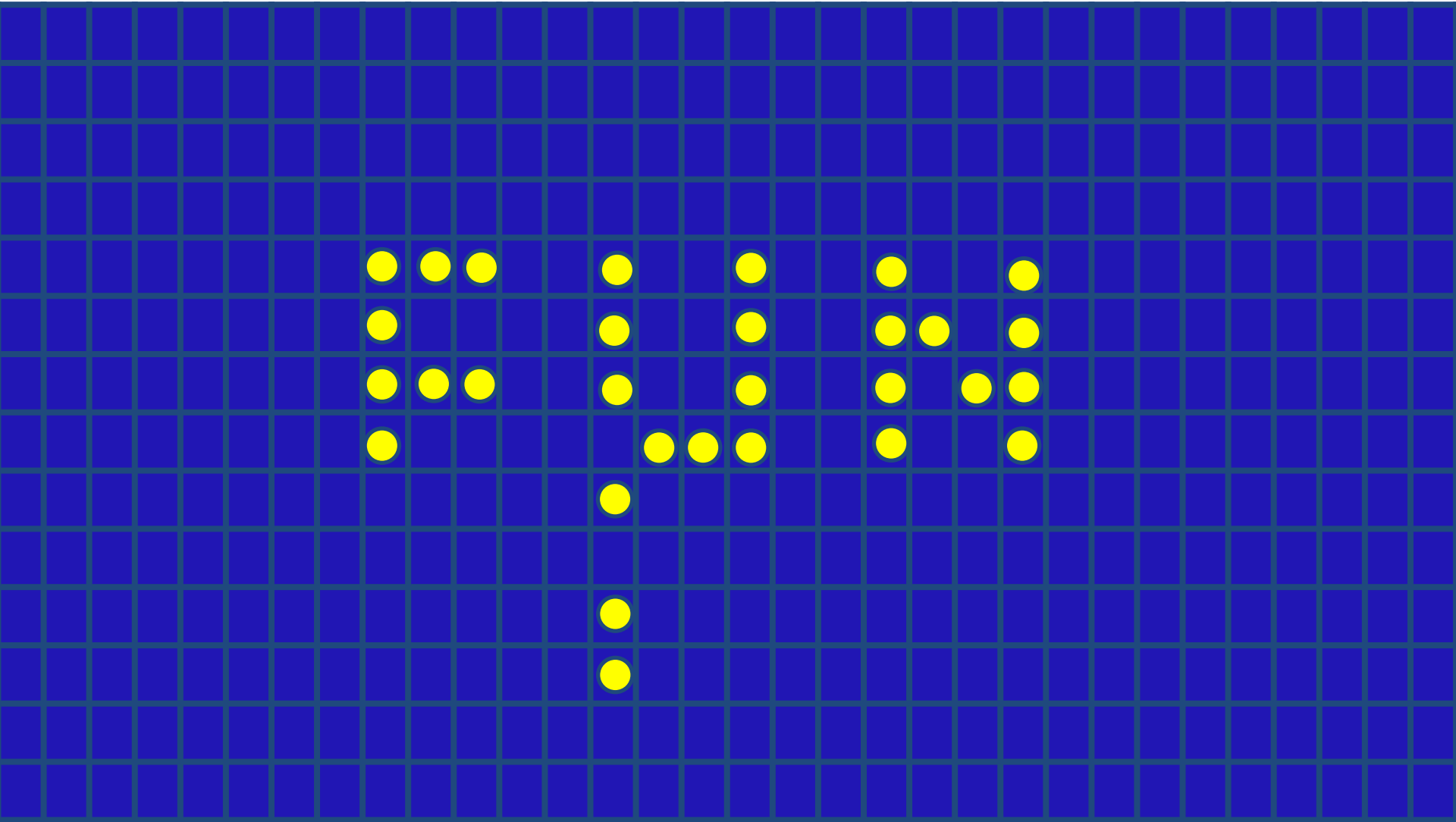


Peg Solitaire

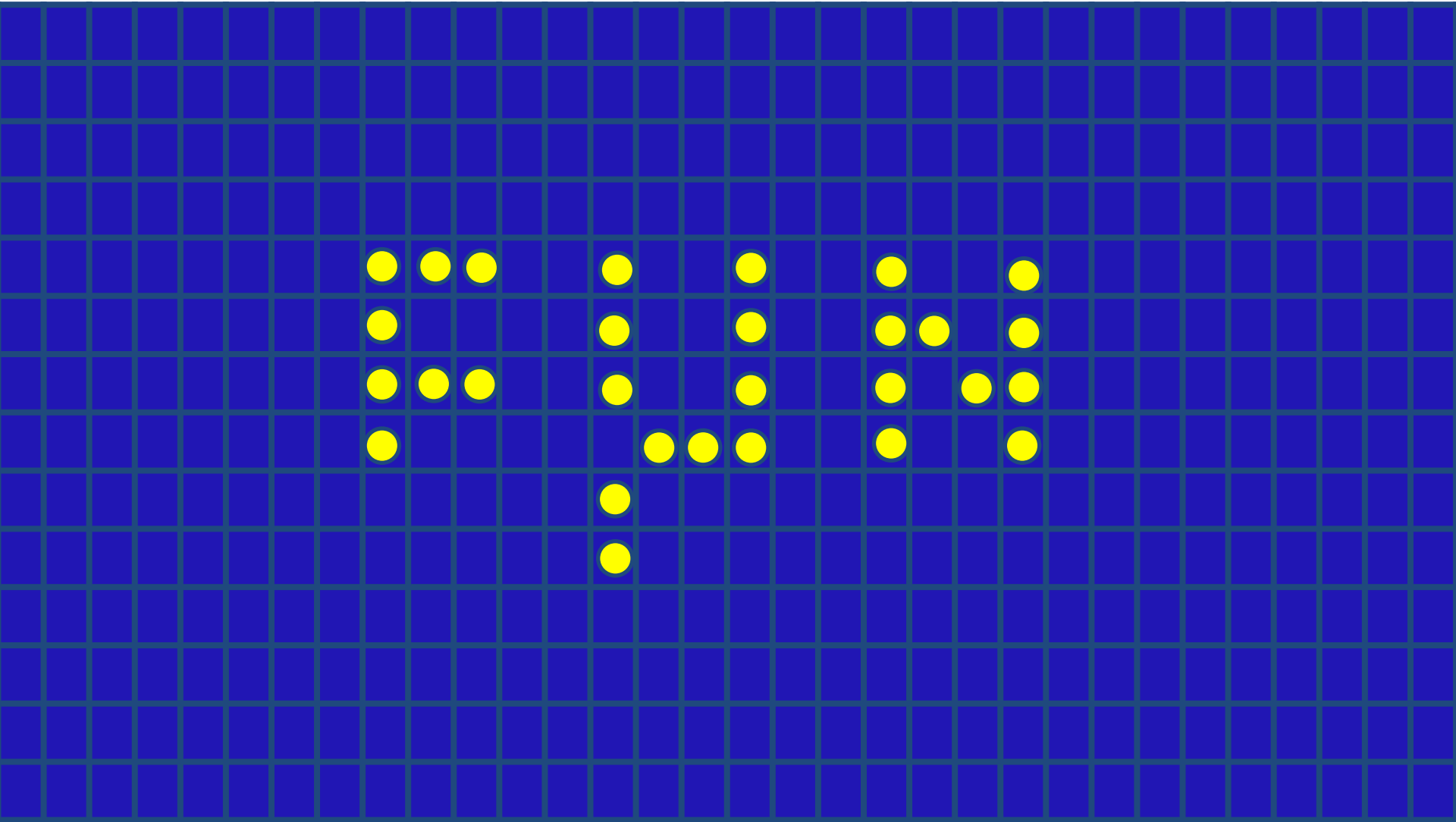


A 20x20 grid with a dark blue background and a lighter blue grid pattern. The letters 'MATH' are formed by yellow dots. The 'M' is composed of 13 dots, the 'A' of 10 dots, the 'T' of 10 dots, and the 'H' of 10 dots. The letters are arranged horizontally in the center of the grid.

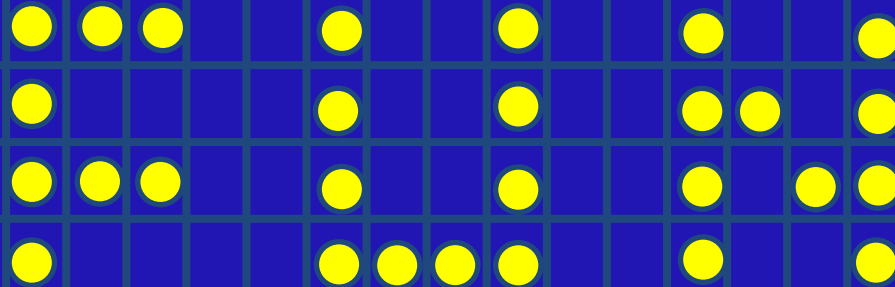
Peg Solitaire



Peg Solitaire



Peg Solitaire



goal: find a sequence of moves from an initial configuration to a target one

Peg Solitaire



Not so very long ago there became widespread an excellent kind of game, called Solitaire, where I play on my own, but as if with a friend as witness and referee to see that I play correctly. A board is filled with stones set in holes, which are to be removed in turn, but none (except the first, which may be chosen for removal at will) can be removed unless you are able to jump another stone across it into an adjacent empty place, when it is captured as in Draughts. He who removes all the stones right to the end according to this rule, wins; [...]. This game can more elegantly be played backwards [...] Thus we can either fill the board, or, what would be more clever, shape a predetermined figure from the stones; perhaps a triangle, a quadrilateral, an octagon, or some other, if this be possible; but such a task is by no means always possible: and this itself would be a valuable art, to foresee what can be achieved; and to have some way, particularly geometrical, of determining this.



Gottfried Wilhelm
von Leibniz
(1646-1716)

Not so very long ago there became widespread an excellent kind of game, called Solitaire, where I play on my own, but as if with a friend, and referee to see that I play correctly. It is filled with stones set in holes removed in turn, but none may be chosen for removal unless your stone is removed unless your stone is placed across it in its place, when it is carried. He who removes a stone to the end according to this rule can more elegantly play the game. Thus we can easily see, what would be more clearly determined figure from the stone, a triangle, a quadrilateral, an octagon, or some other, if this be possible; but such a task is by no means always possible: and this itself would be a valuable art, to foresee what can be achieved; and to have some way, particularly geometrical, of determining this.

That's fun!



Gottfried Wilhelm
von Leibniz
(1646-1716)

Peg Solitaire



NP-complete to
decide whether the
board can be cleared
[Uehara&Iwata,90]

Peg Solitaire

Generalized Hi-Q is NP-Complete

Ryuhei UEHARA[†] and Shigeki IWATA^{††}, *Members*

SUMMARY This paper deals with a popular puzzle known as Hi-Q. The puzzle is generalized: the board is extended to the size $n \times n$, an initial position of the puzzle is given, and a place is given on which only one token is finally placed. The complexity of the generalized Hi-Q is proved NP-complete.

1. Introduction

In general, combinatorial puzzles and games are hard to analyze, since we have to cope with enormous number of positions of the board. It is one of the main themes in artificial intelligence to solve these problems by heuristic methods. It is important at the same time to

initial position is given on the extended board of size $n \times n$, and a goal is also given on which only one token will finally be placed. We show that the problem to determine whether there is an answer for a given generalized Hi-Q is NP-complete. The NP-hardness can be obtained by reducing from a variation of the hamiltonian cycle problem.

2. Complexity Result

We extend the size of the board of Hi-Q to $n \times n$, and assume further that both a position and a goal of the puzzle are given. Now call the puzzle generalized Hi-Q.

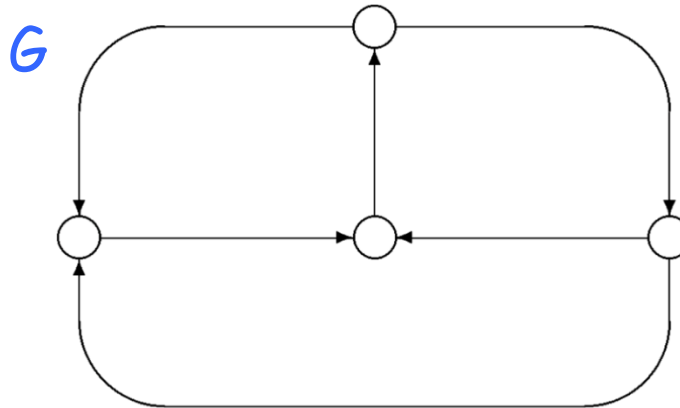
Reduction from:

Hamiltonian Cycle in planar directed graphs with degree 3

it can be drawn in a plane in such
a way that no edges cross each other

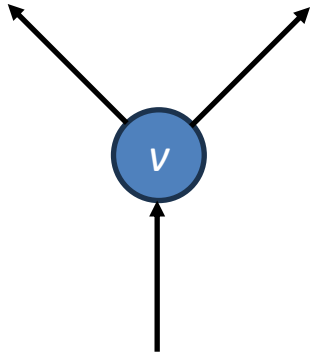
input: a directed planar graph G where each vertex has degree exactly 3

goal: does G has a directed Hamiltonian cycle?

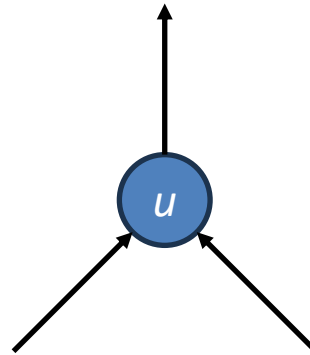


It is NP-complete

Only 2 types of vertices:

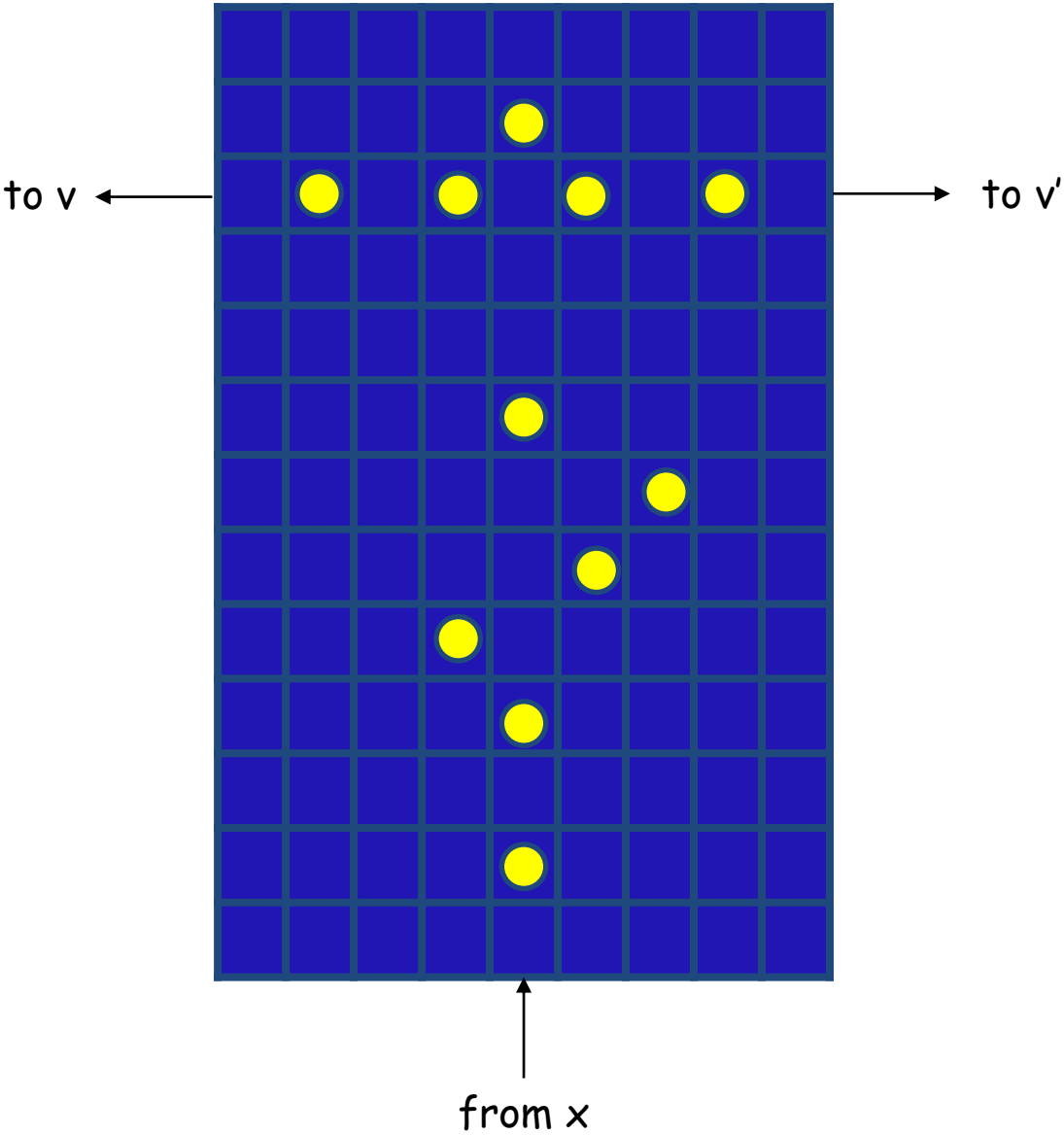
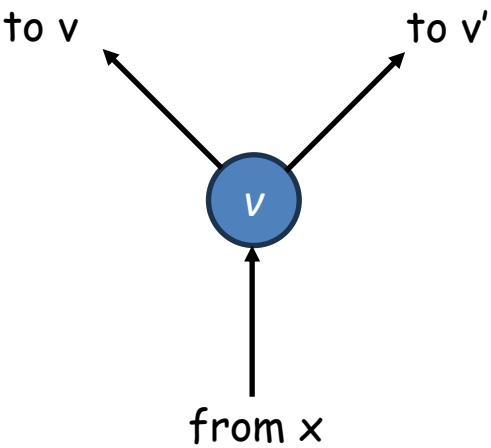


1-in 2-out
degree vertex

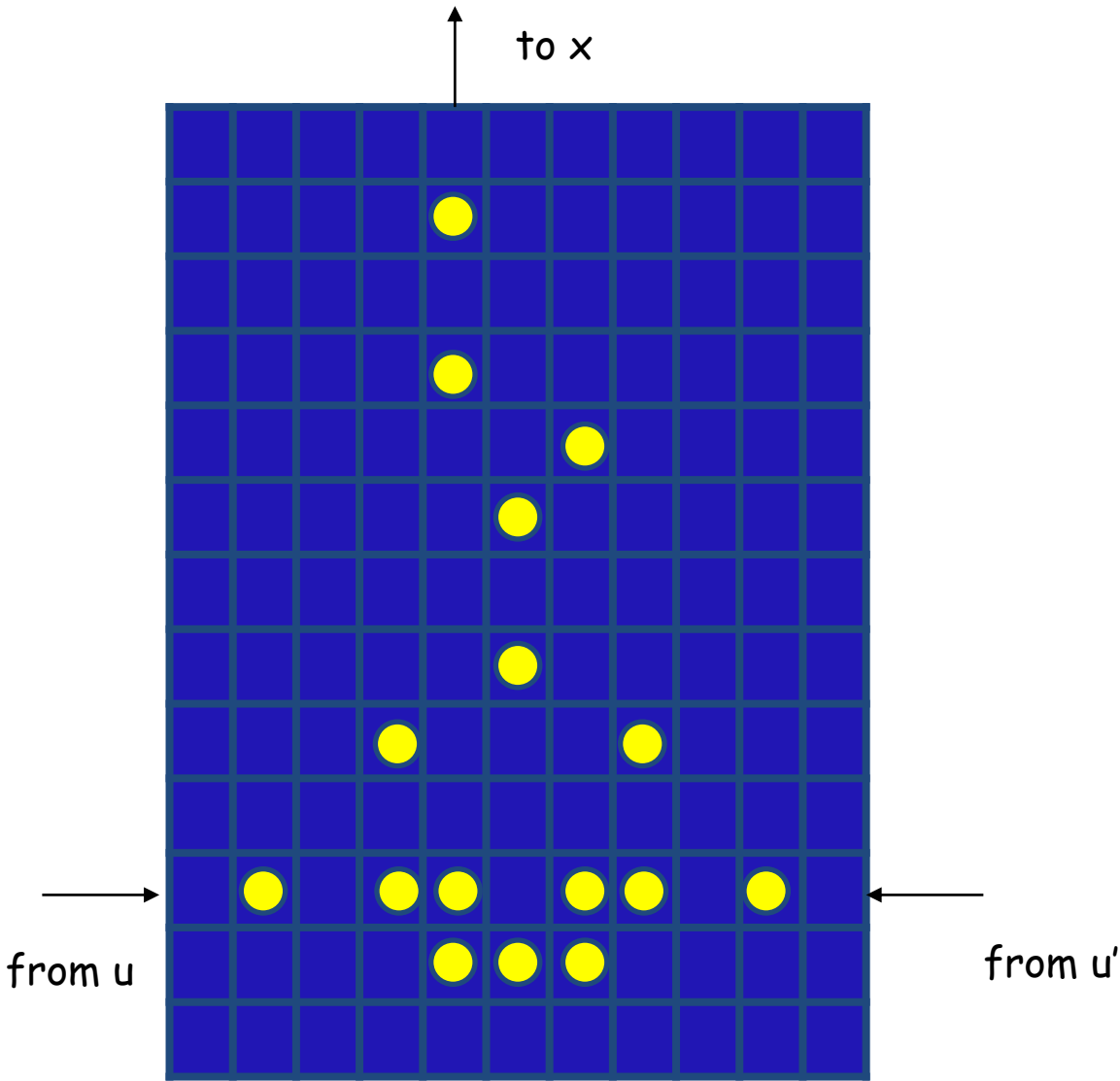
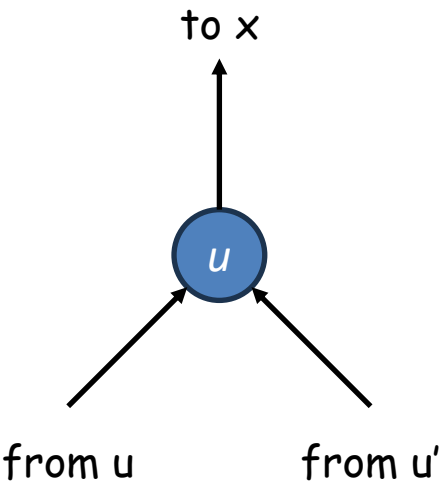


2-in 1-out
degree vertex

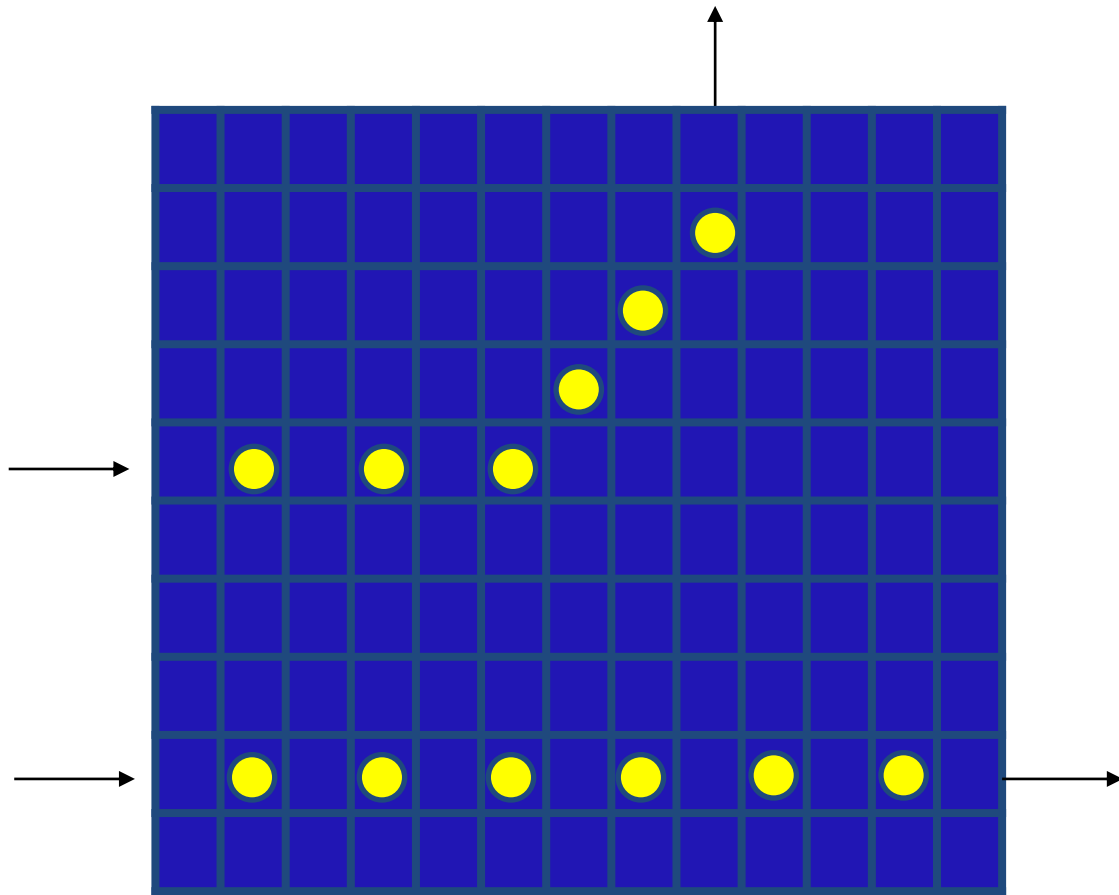
1-in 2-out degree vertex gadget



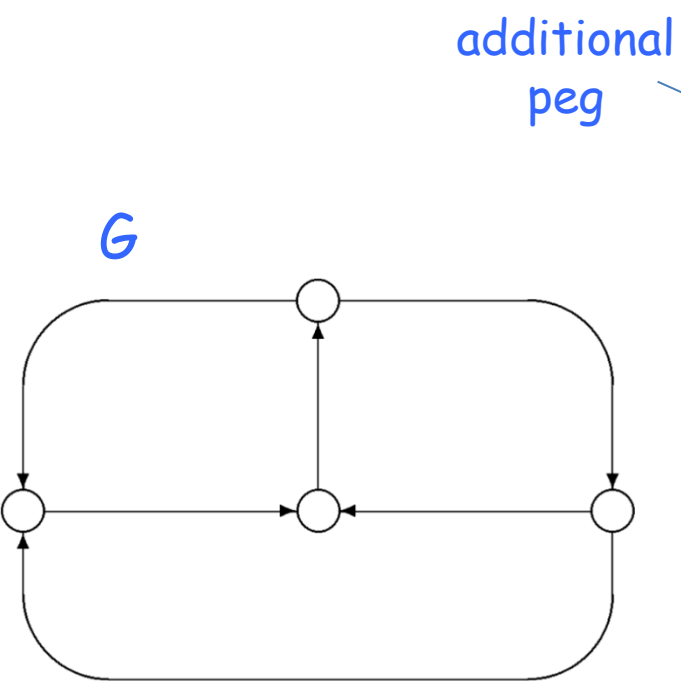
2-in 1-out degree vertex gadget



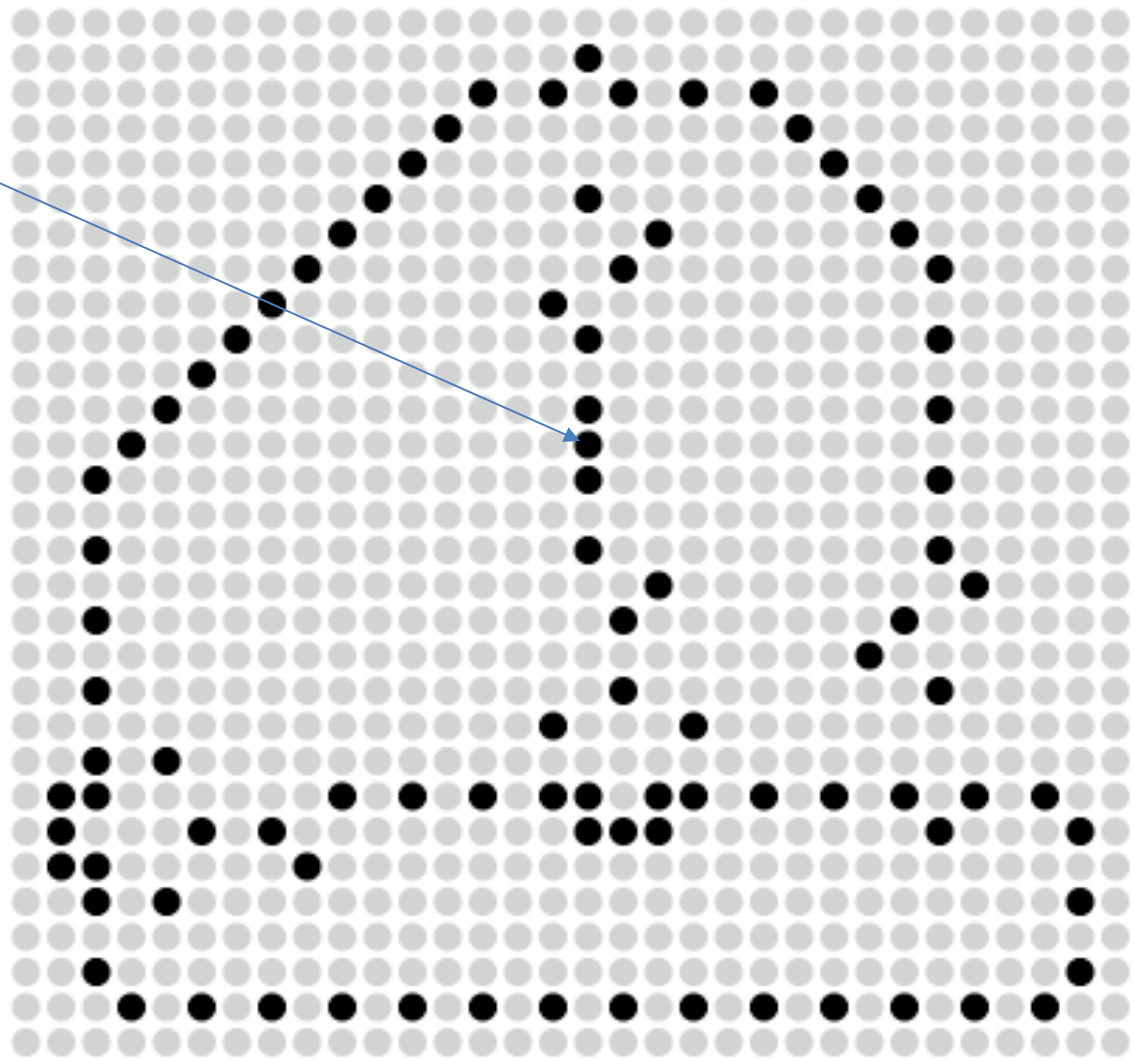
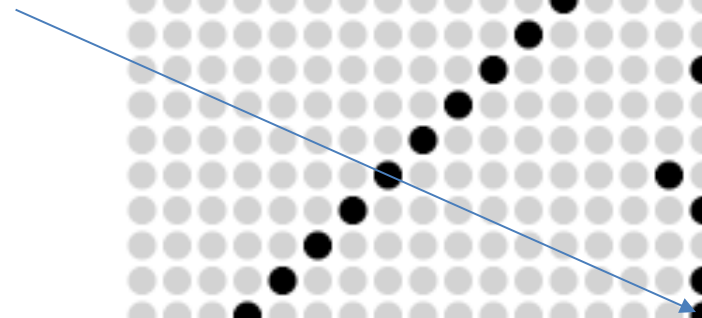
wire/edge gadgets



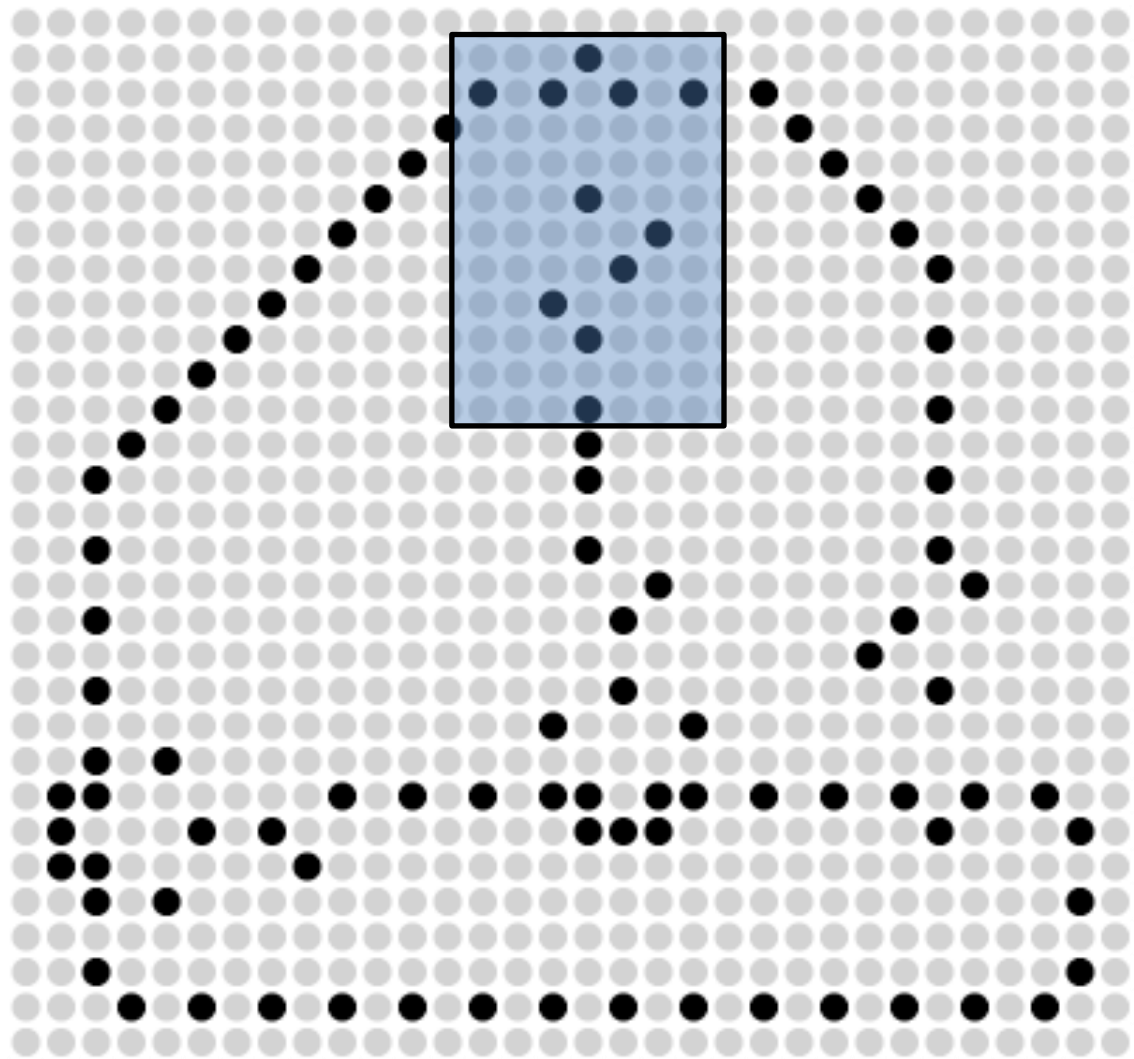
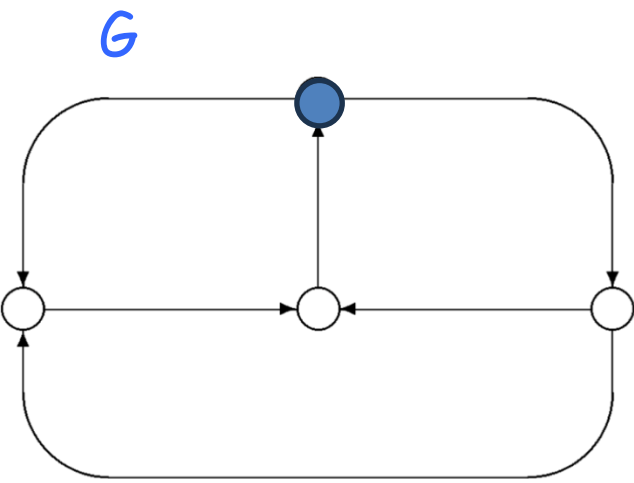
Connecting the gadgets



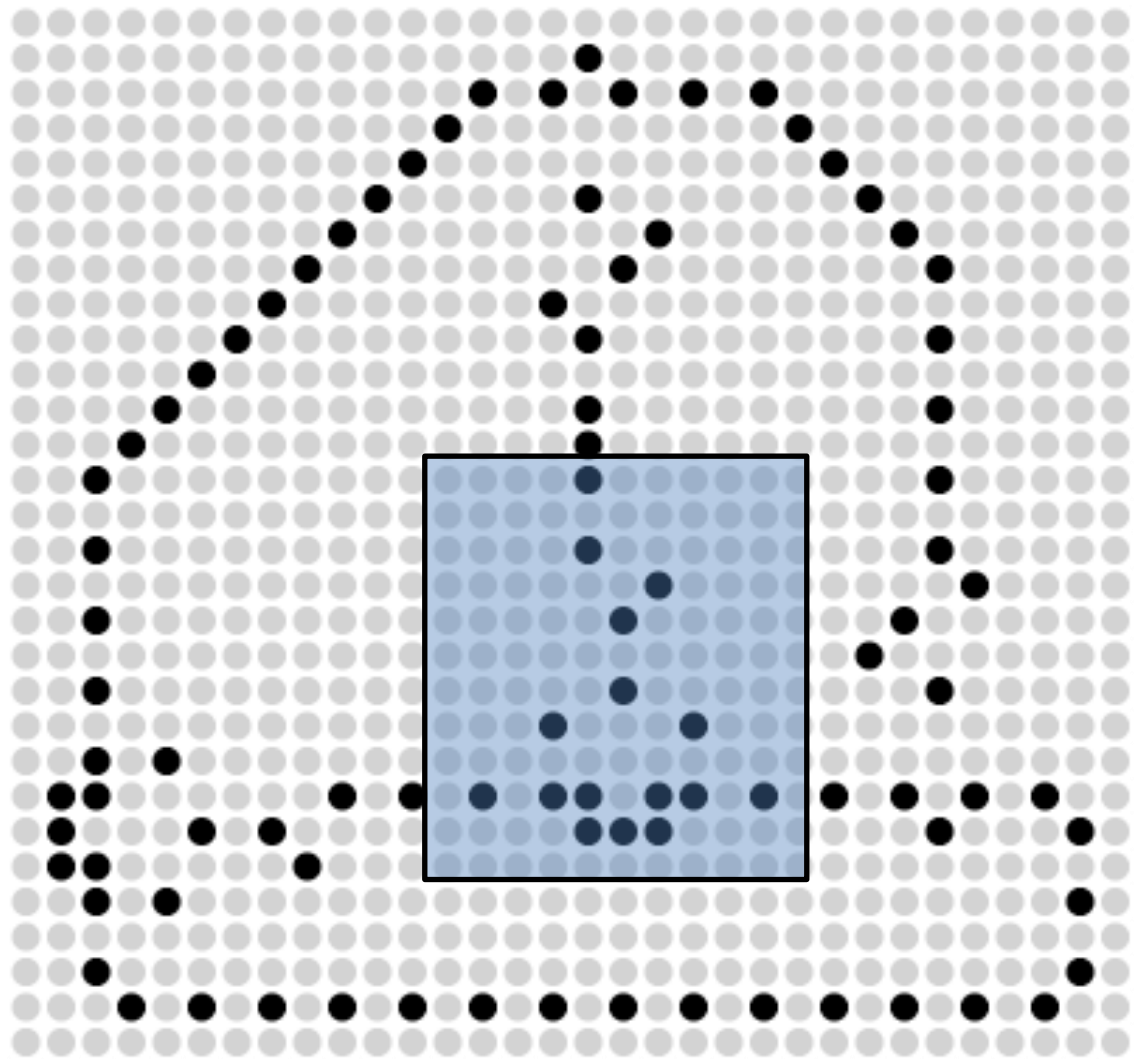
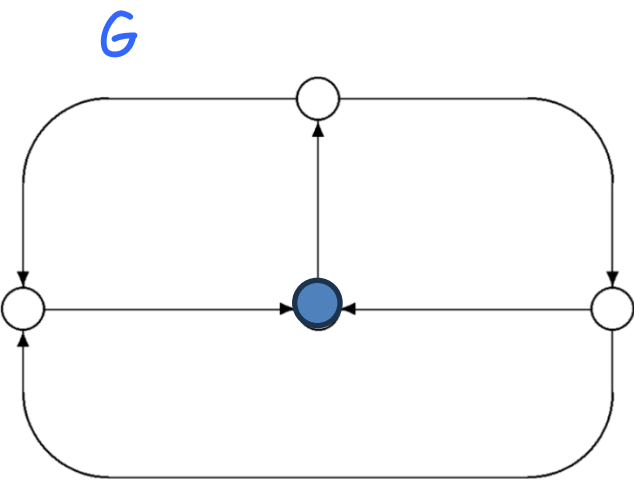
additional
peg



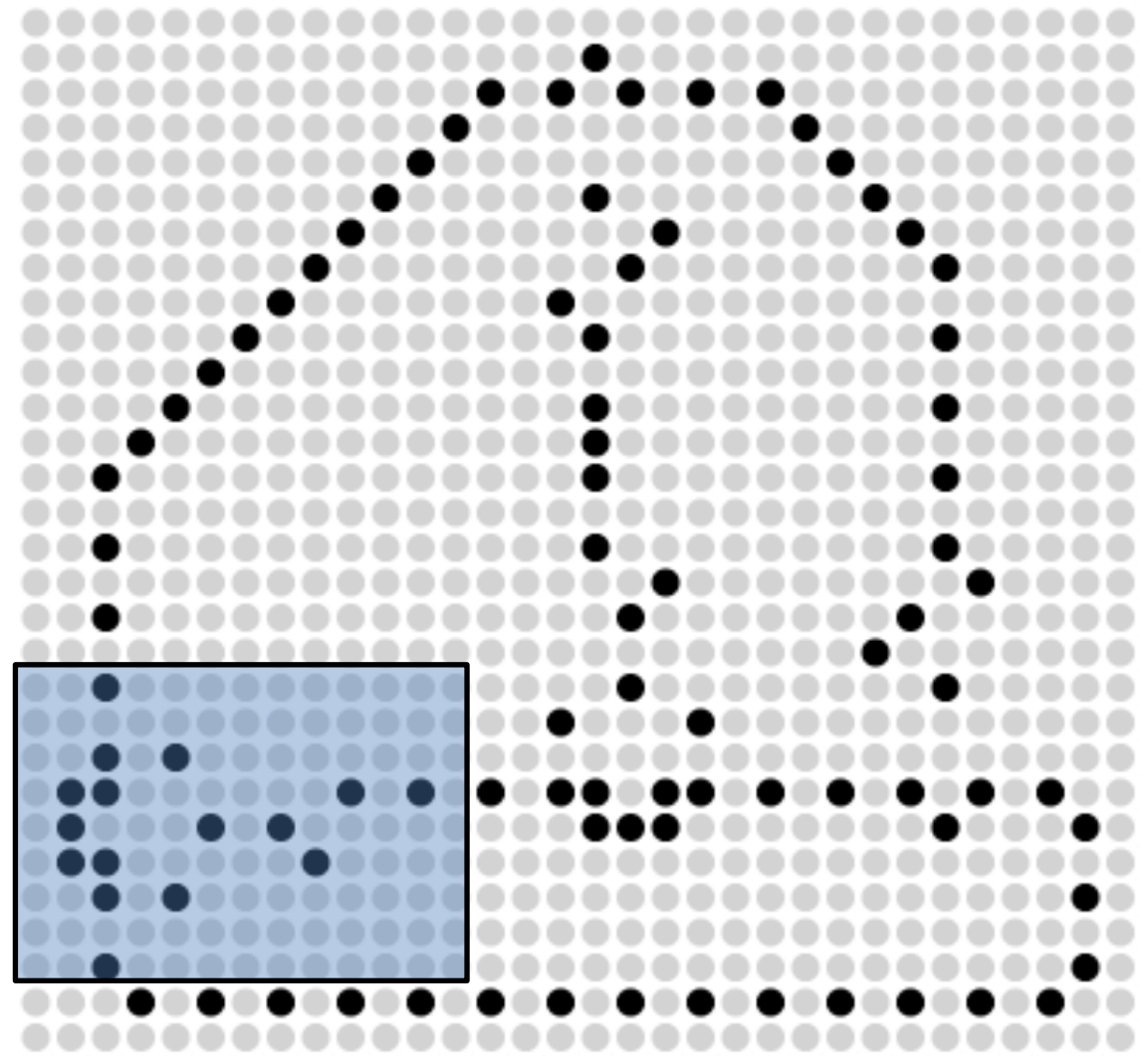
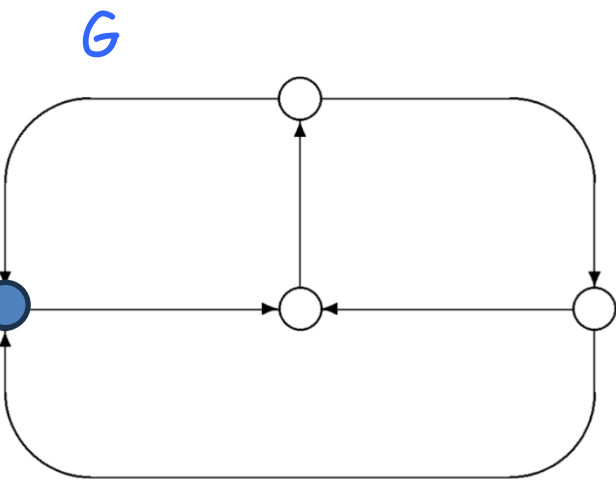
Connecting the gadgets



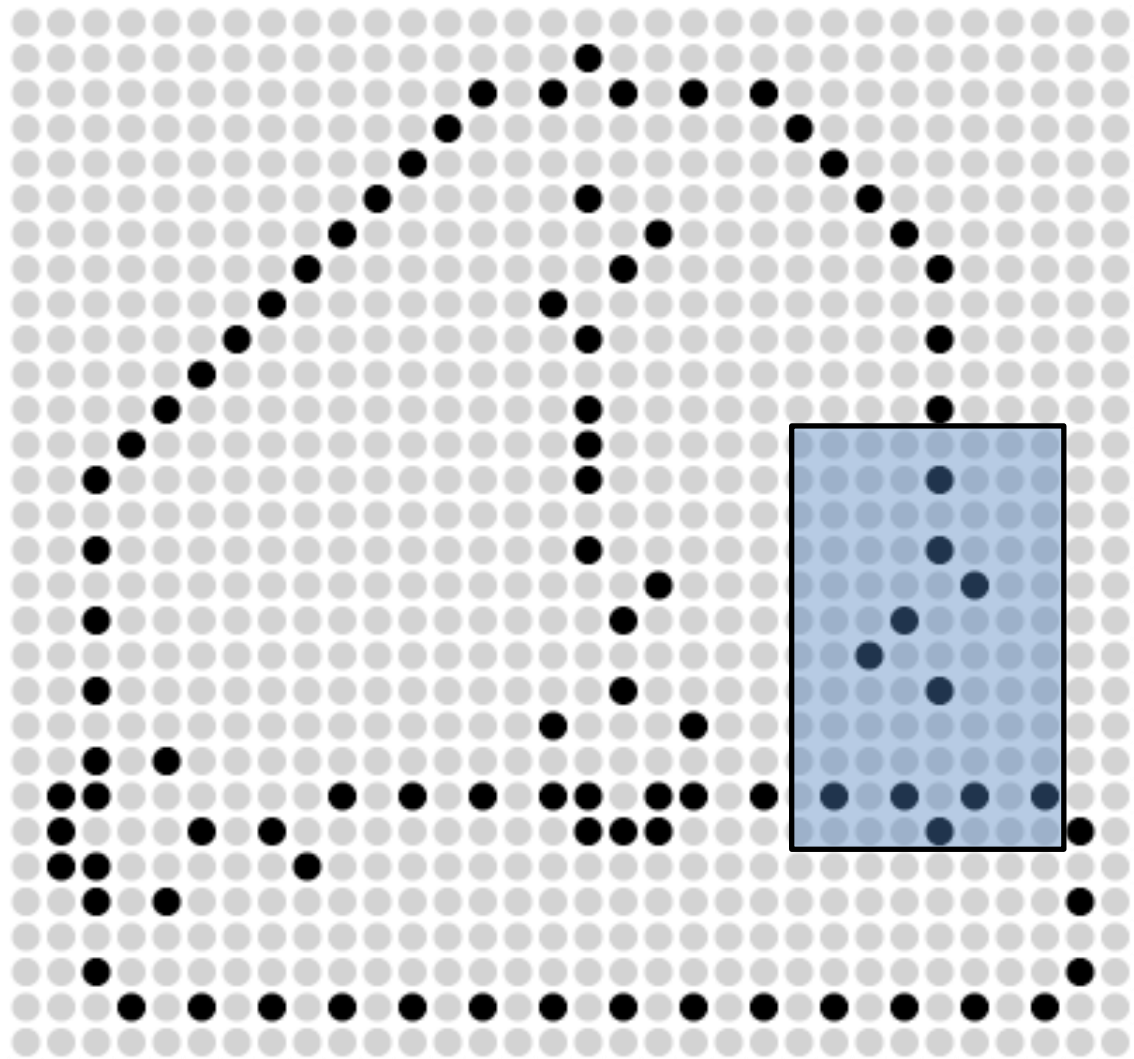
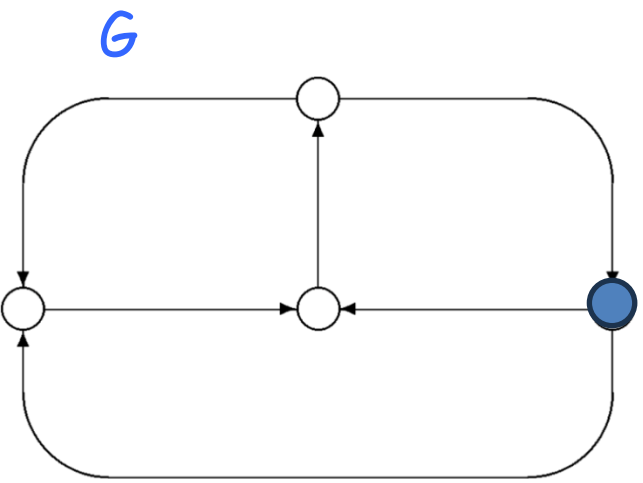
Connecting the gadgets



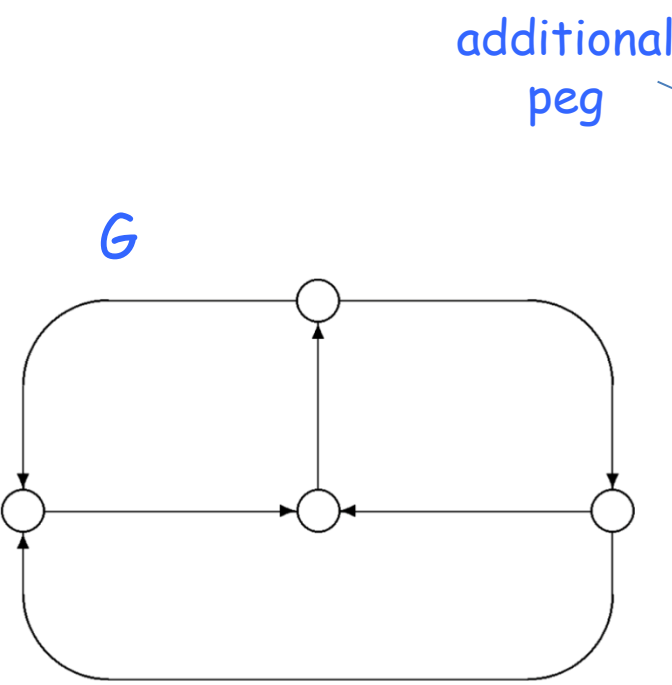
Connecting the gadgets



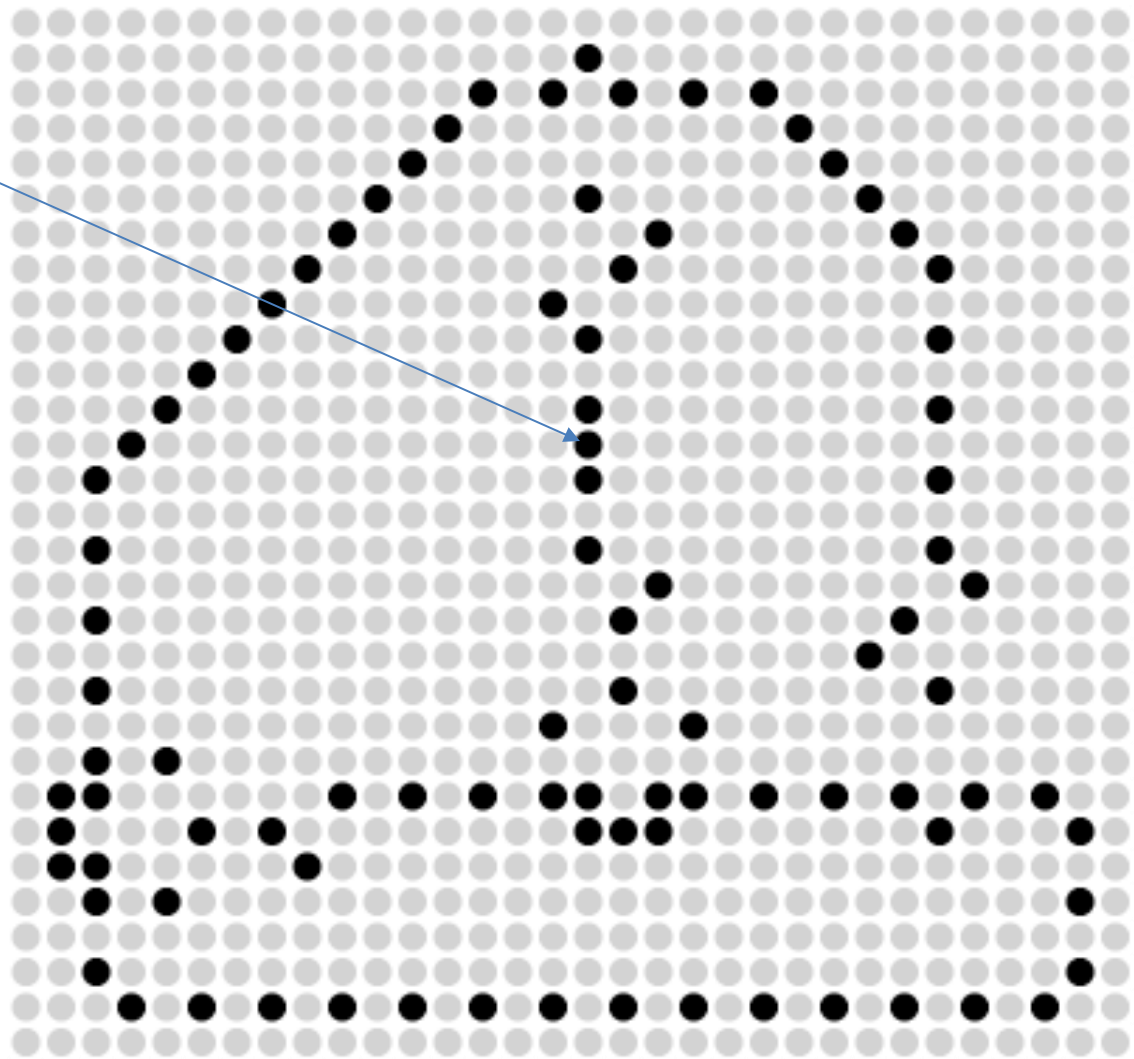
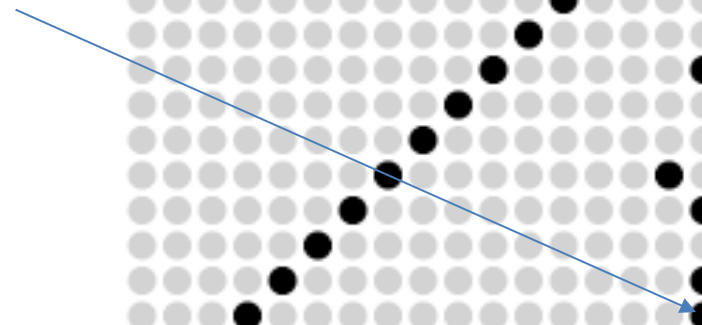
Connecting the gadgets



Connecting the gadgets



additional
peg



claim:

you can clear the board if and only if G has a Hamiltonian cycle

claim:

you can clear the board if and only if G has a Hamiltonian cycle

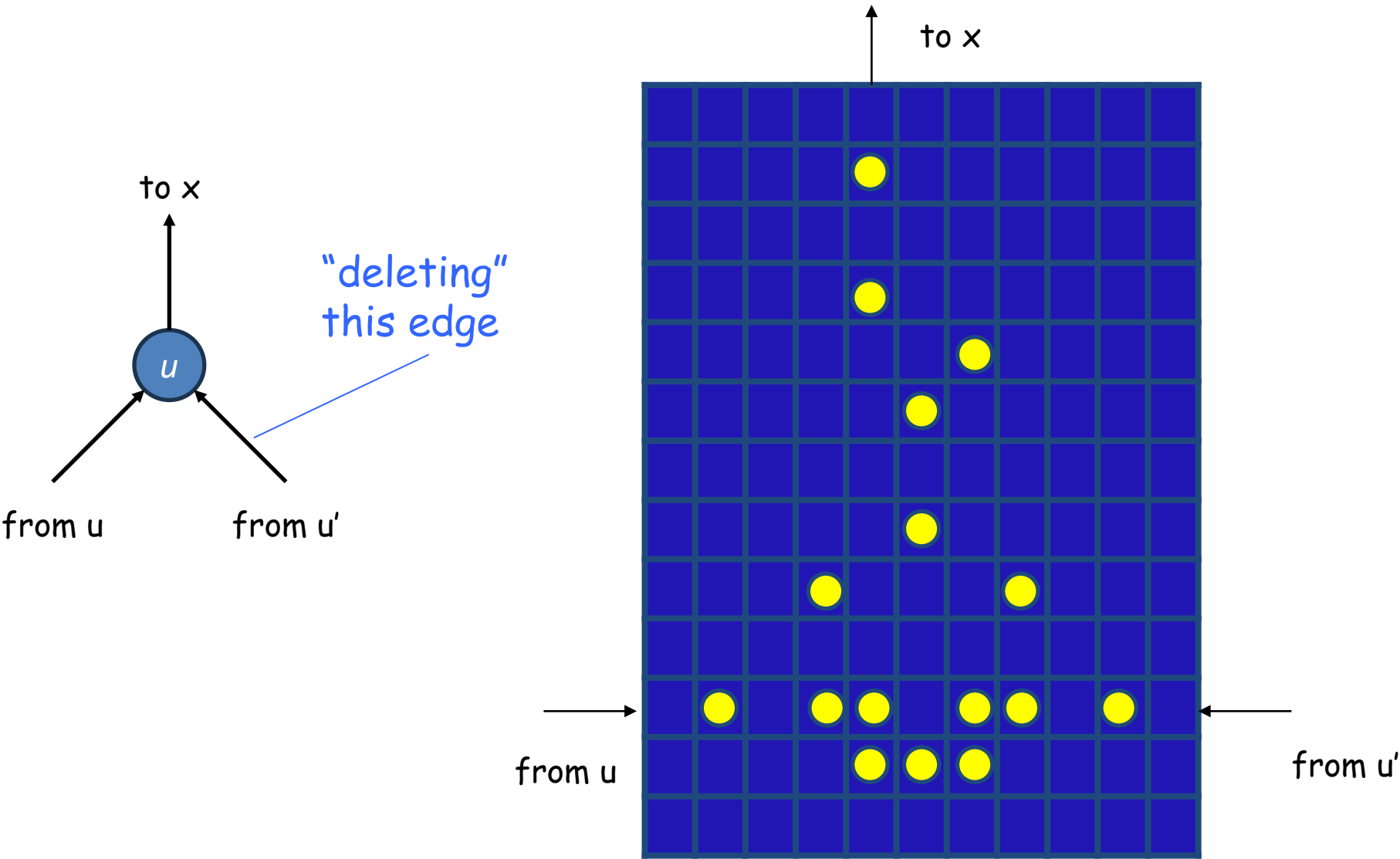
proof

($\Leftarrow$)

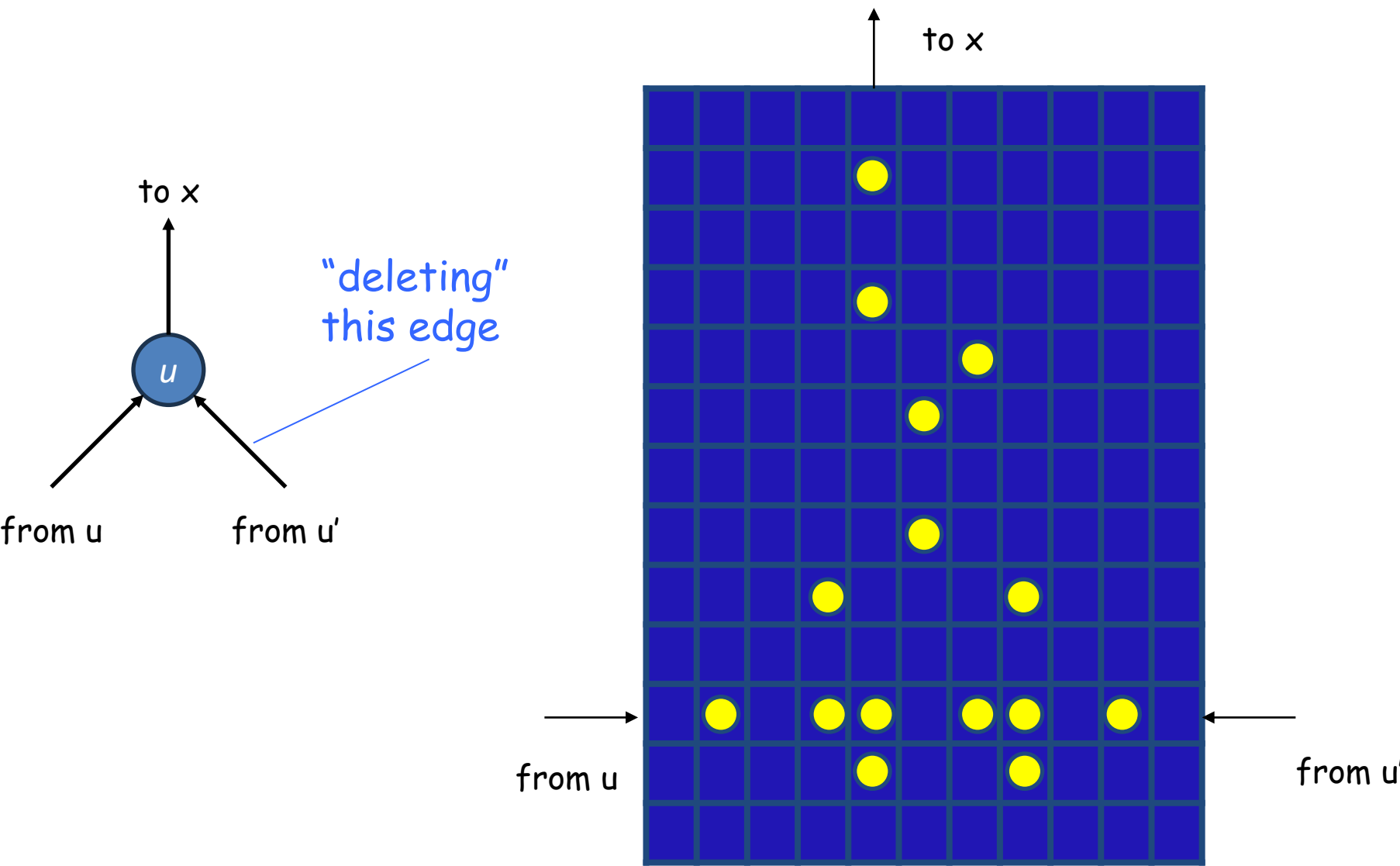
if there is a Hamiltonian cycle C :

- first delete/clear the edges that do not belong to C
- clear the remaining pegs by going through C

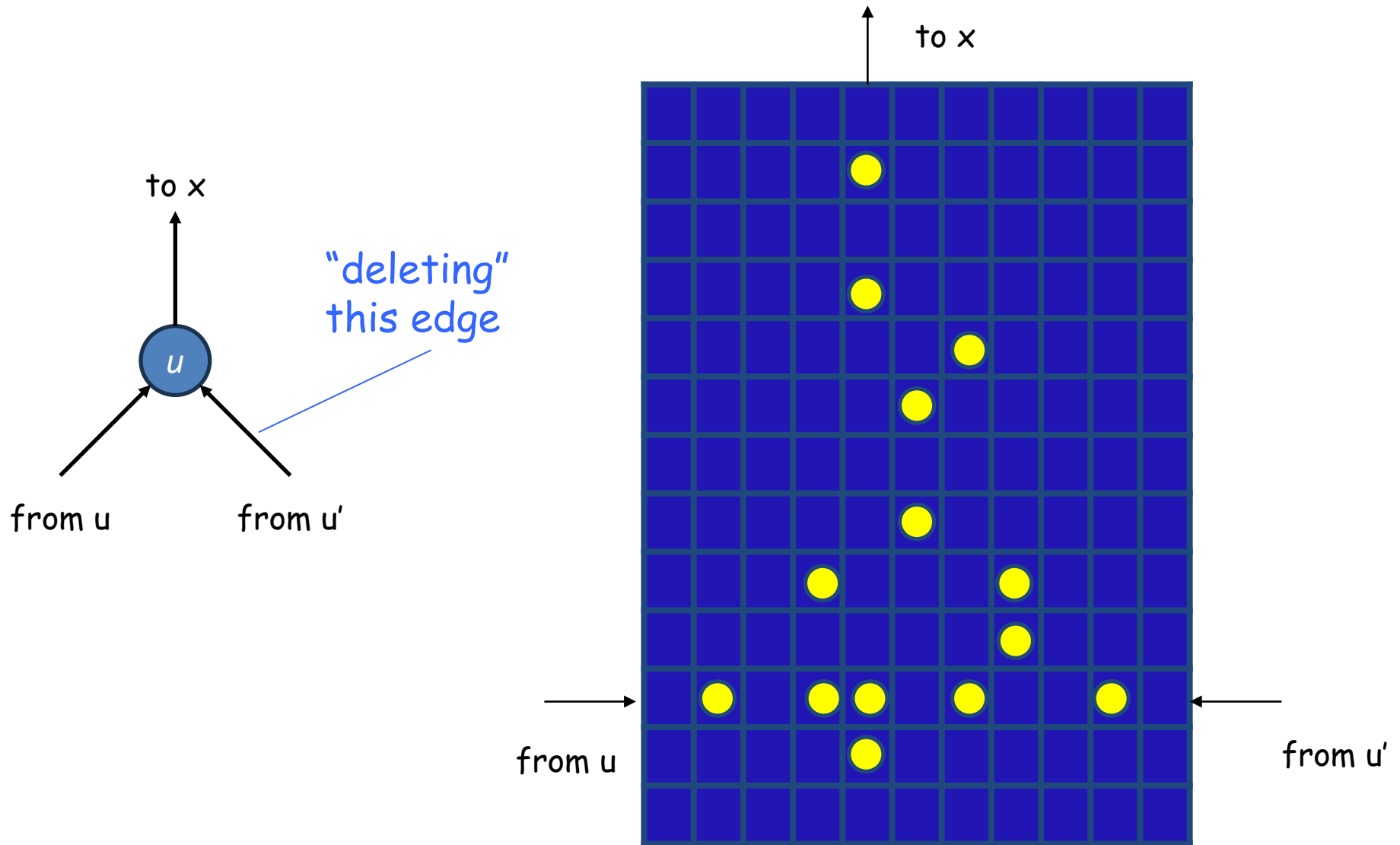
2-in 1-out degree vertex gadget: intended behavior



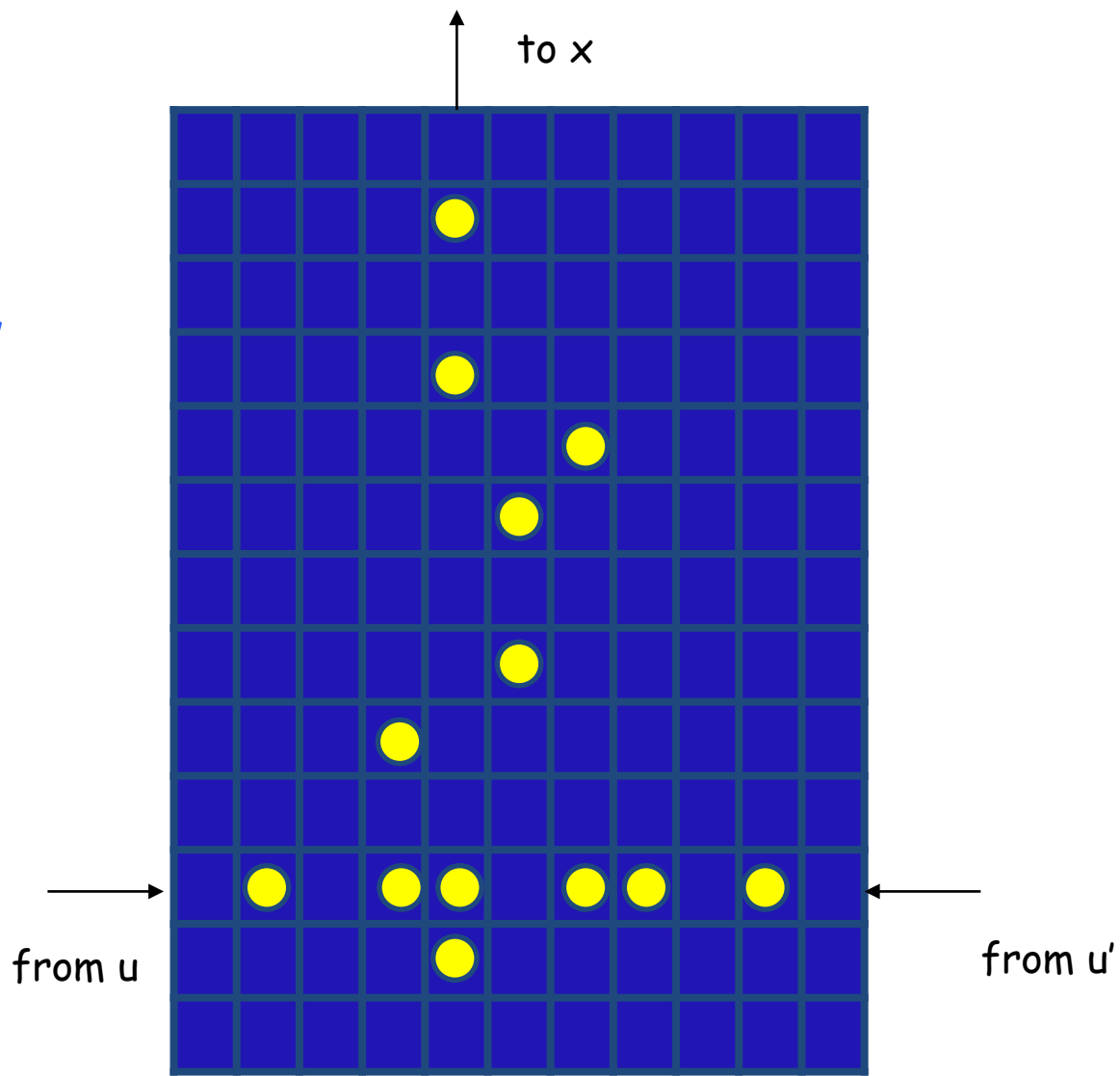
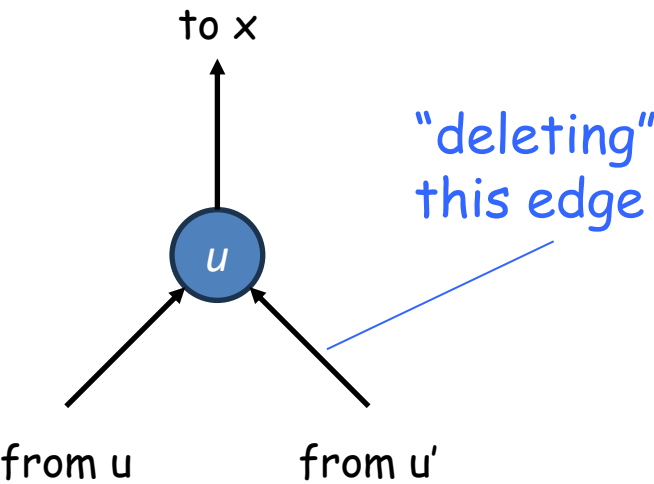
2-in 1-out degree vertex gadget: intended behavior



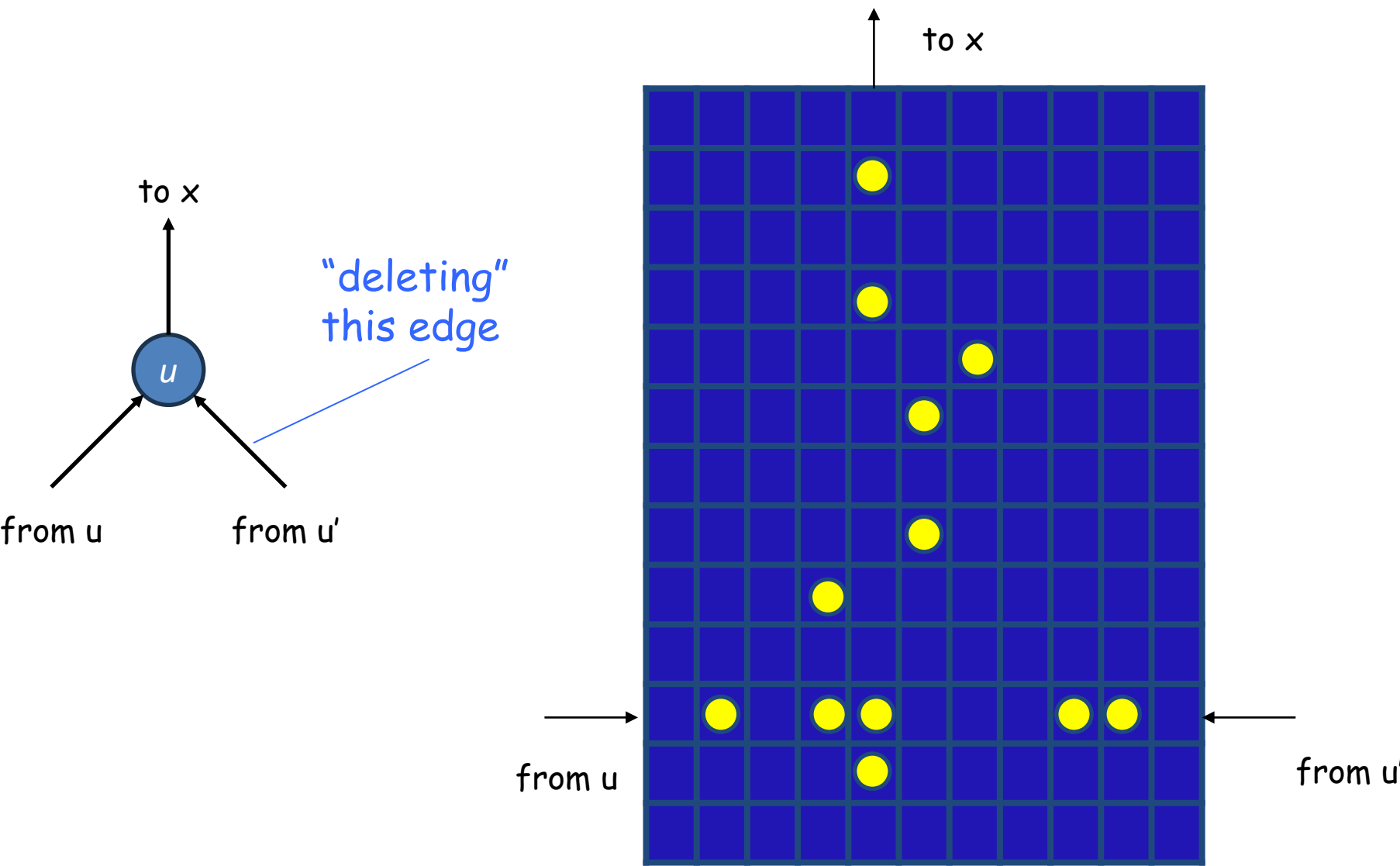
2-in 1-out degree vertex gadget: intended behavior



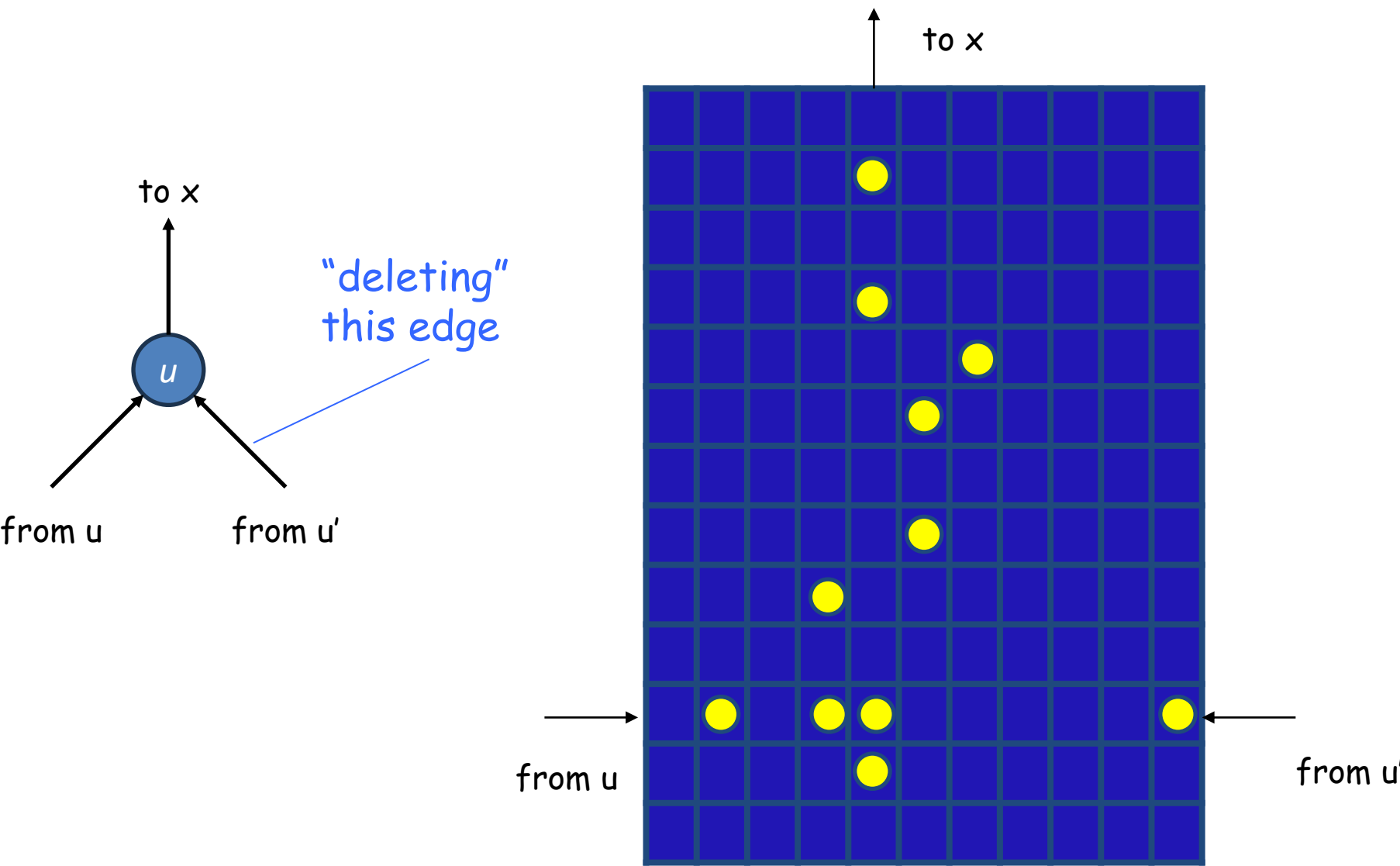
2-in 1-out degree vertex gadget: intended behavior



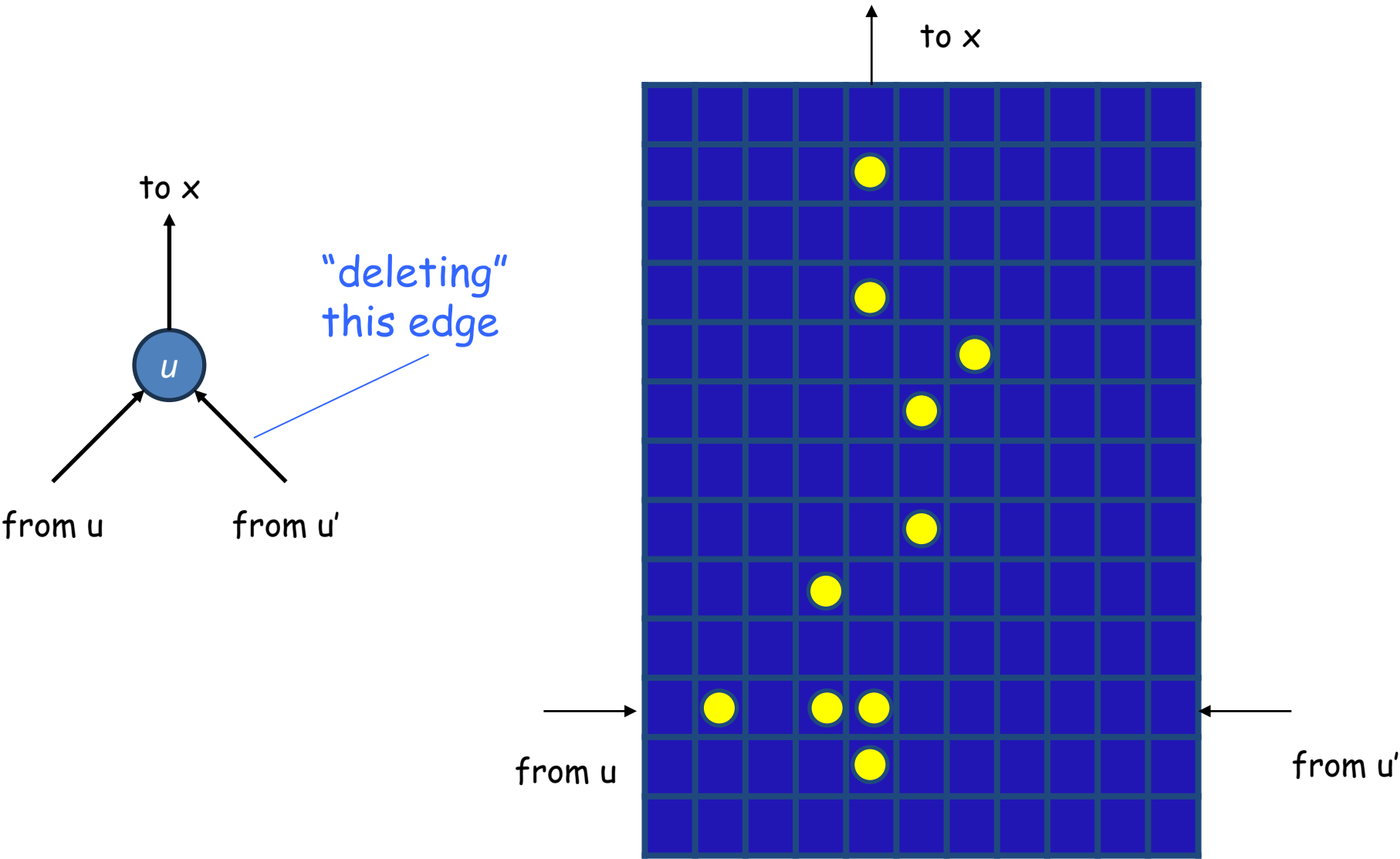
2-in 1-out degree vertex gadget: intended behavior



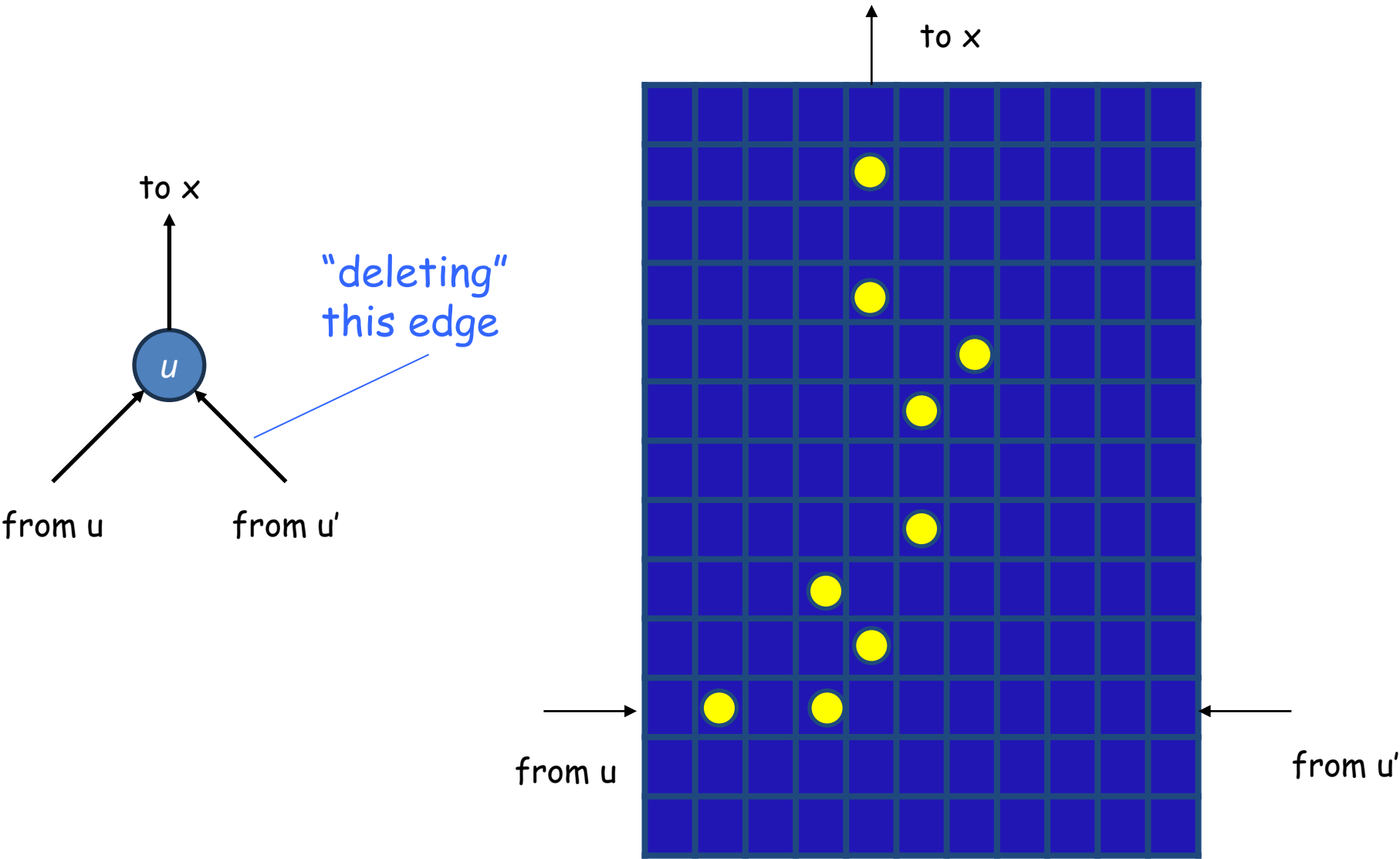
2-in 1-out degree vertex gadget: intended behavior



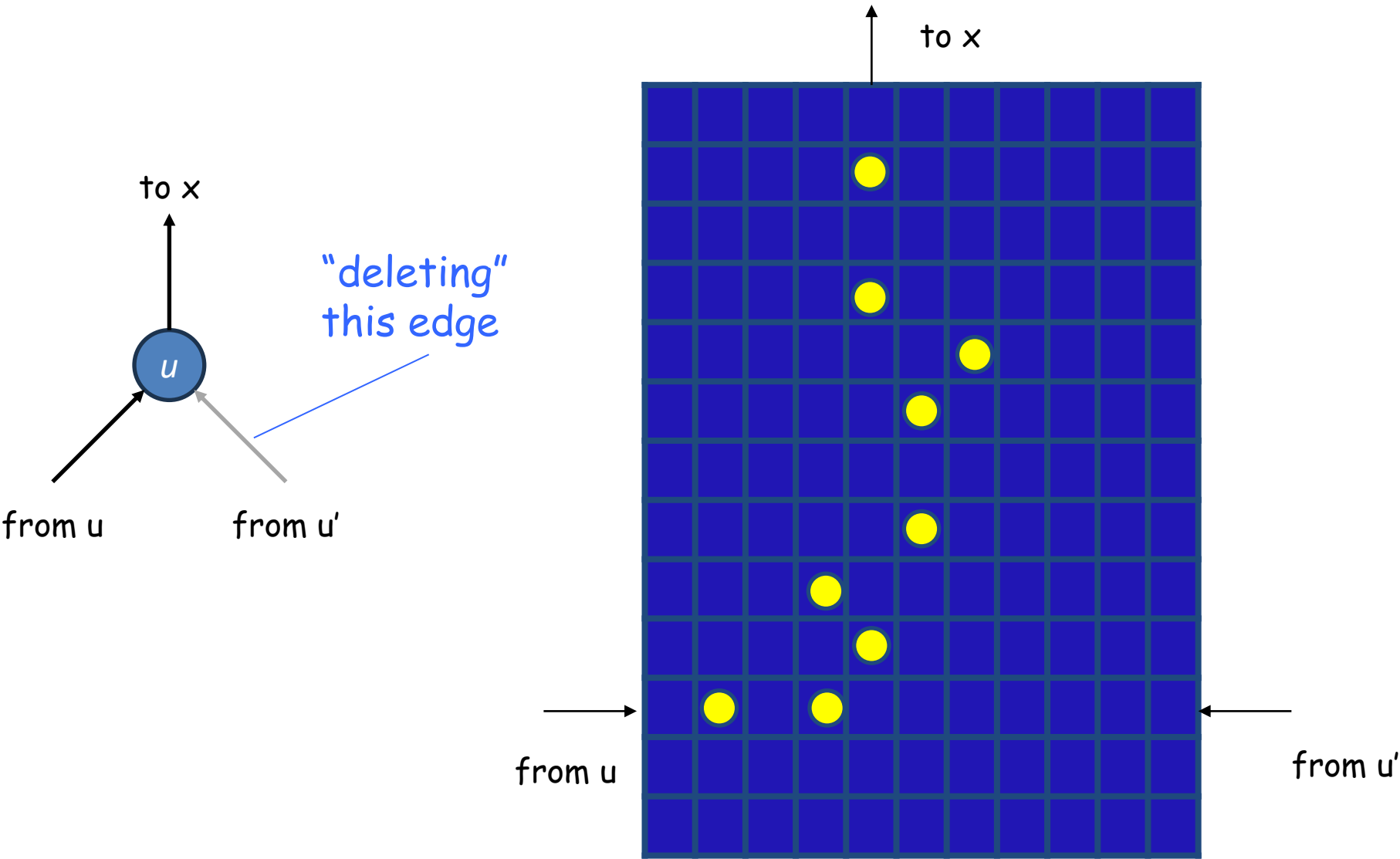
2-in 1-out degree vertex gadget: intended behavior



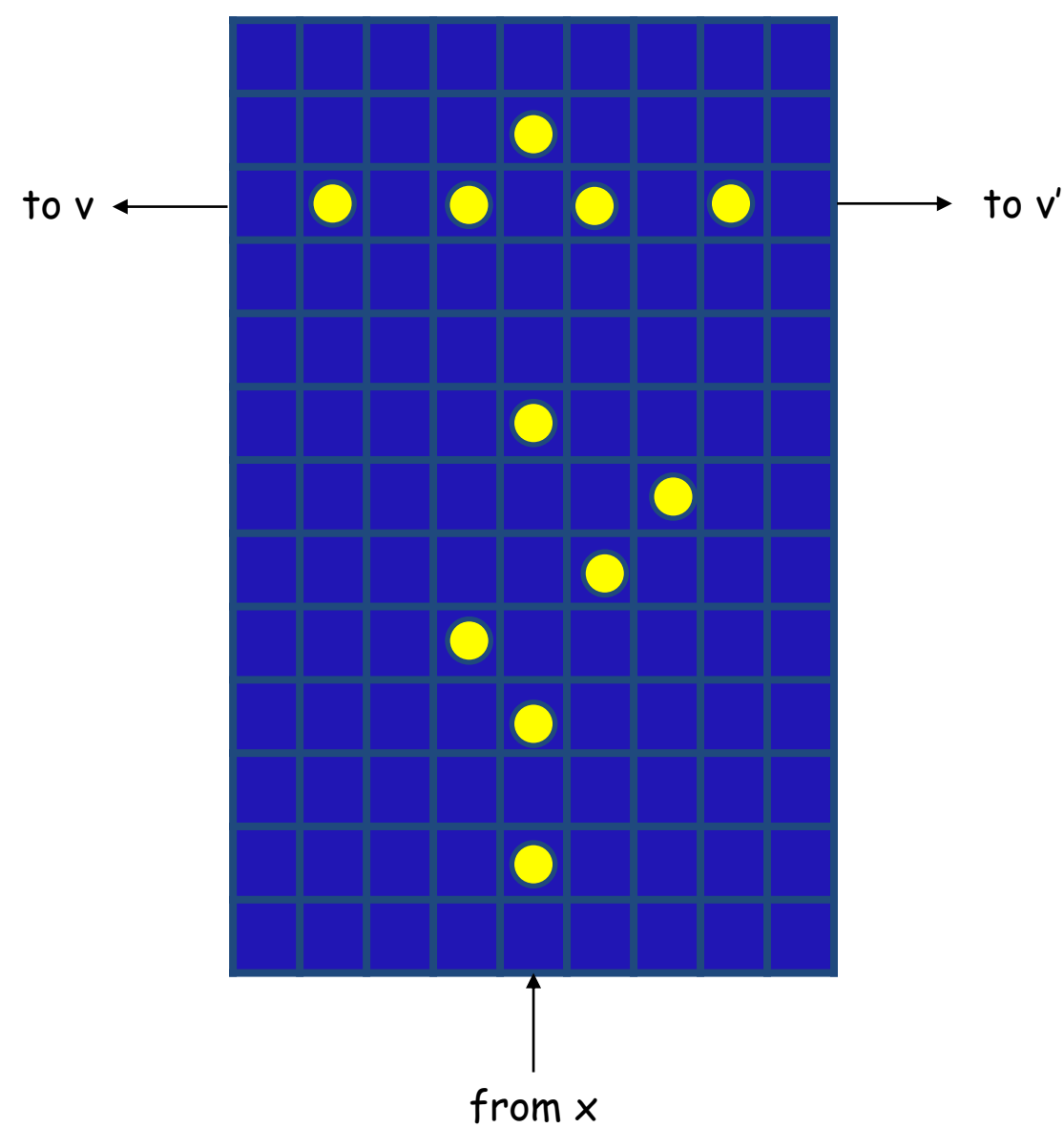
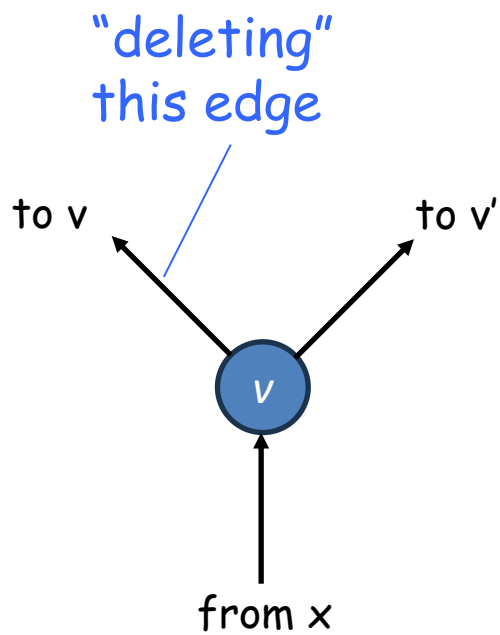
2-in 1-out degree vertex gadget: intended behavior



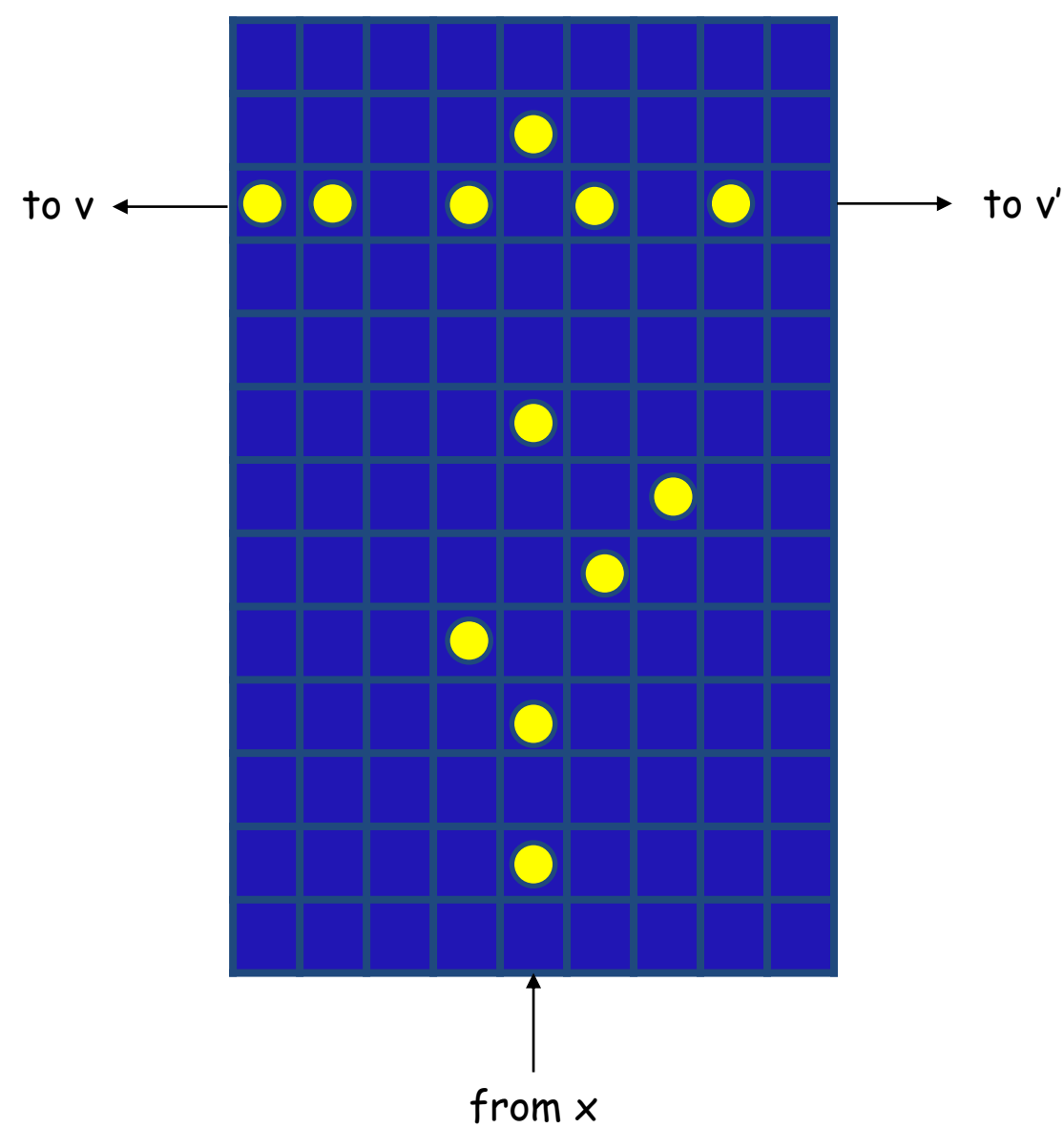
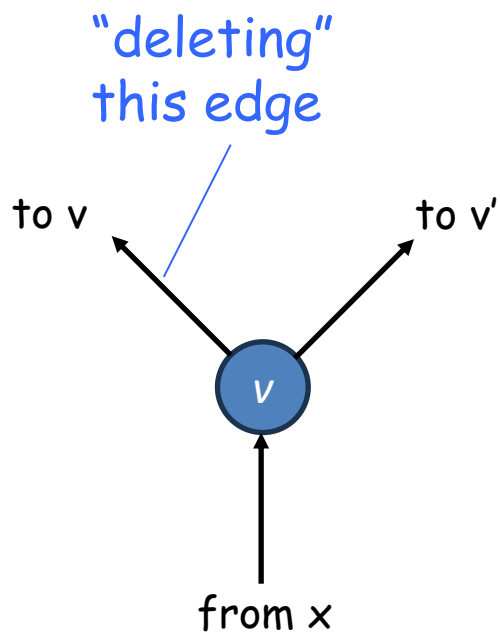
2-in 1-out degree vertex gadget: intended behavior



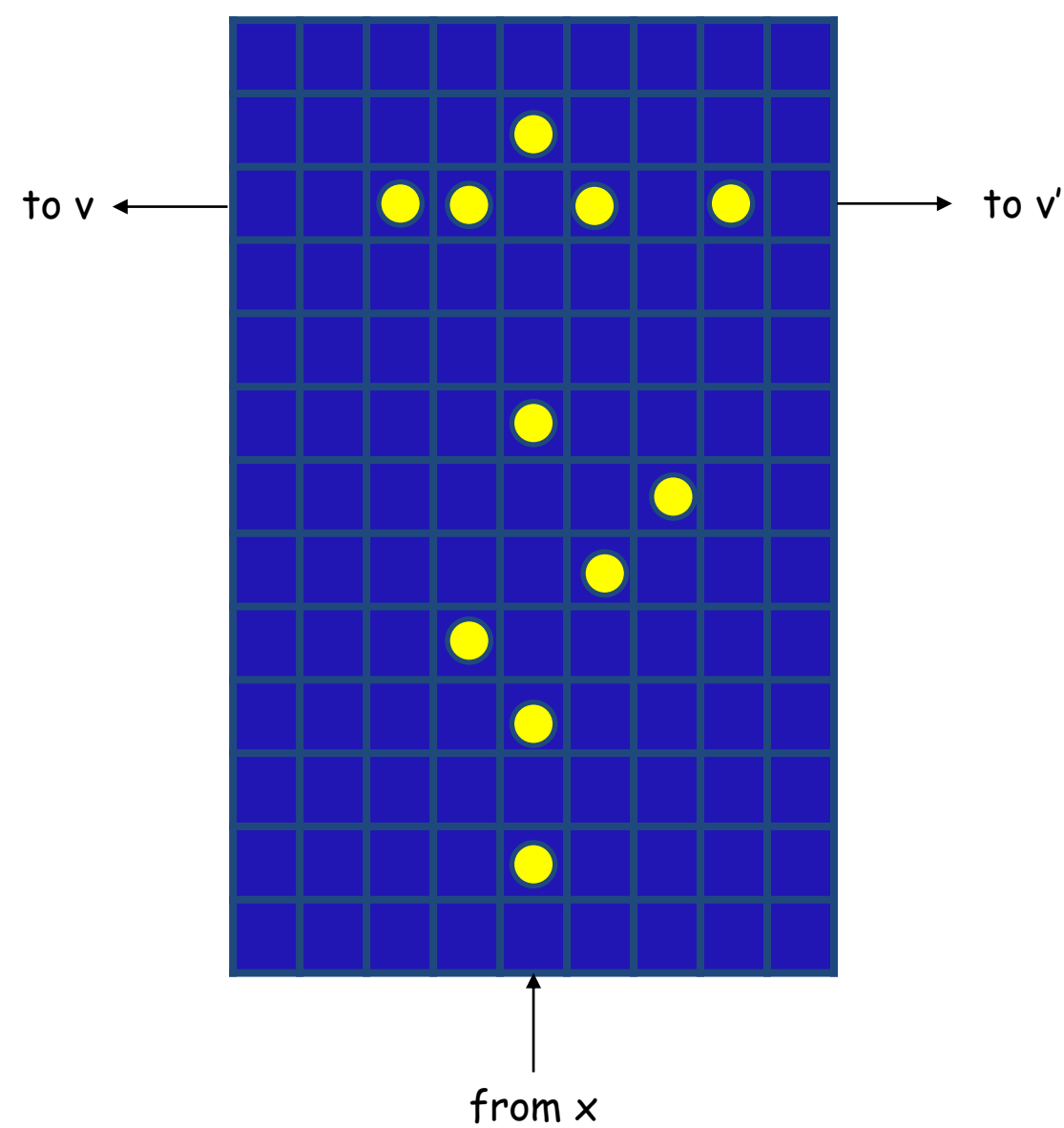
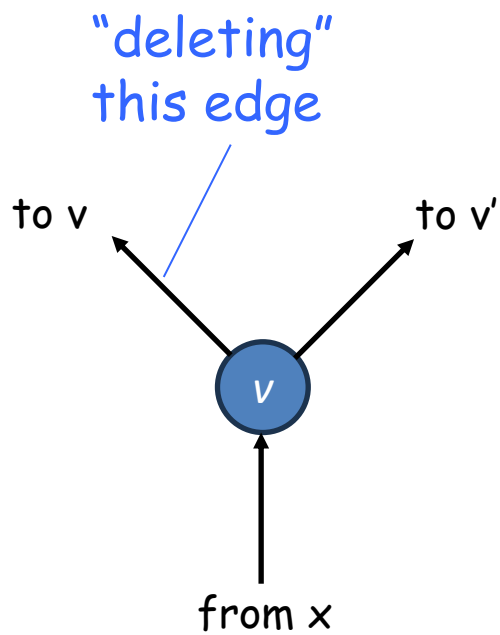
1-in 2-out degree vertex gadget: the intended behavior



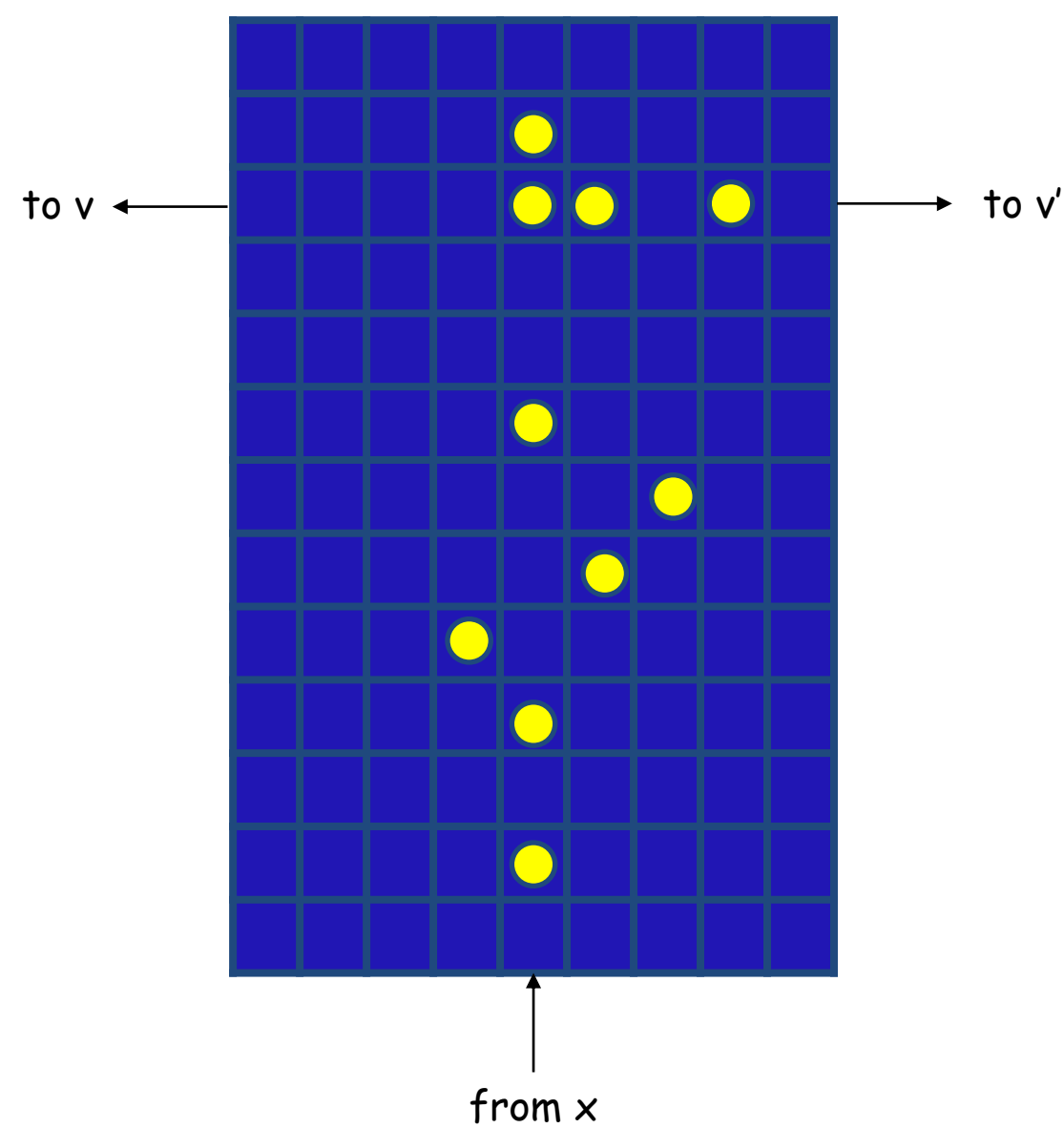
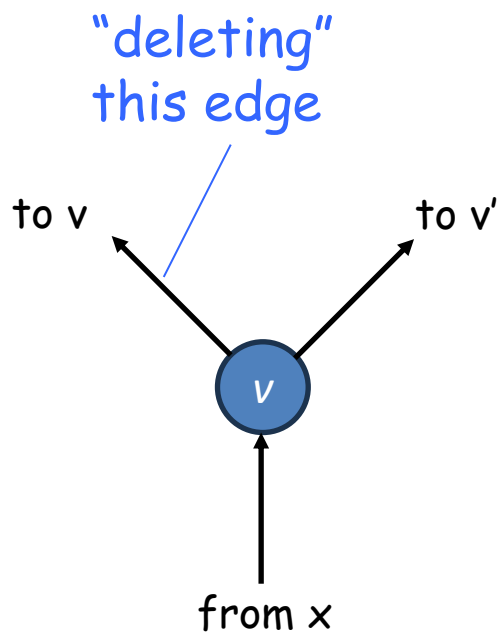
1-in 2-out degree vertex gadget: the intended behavior



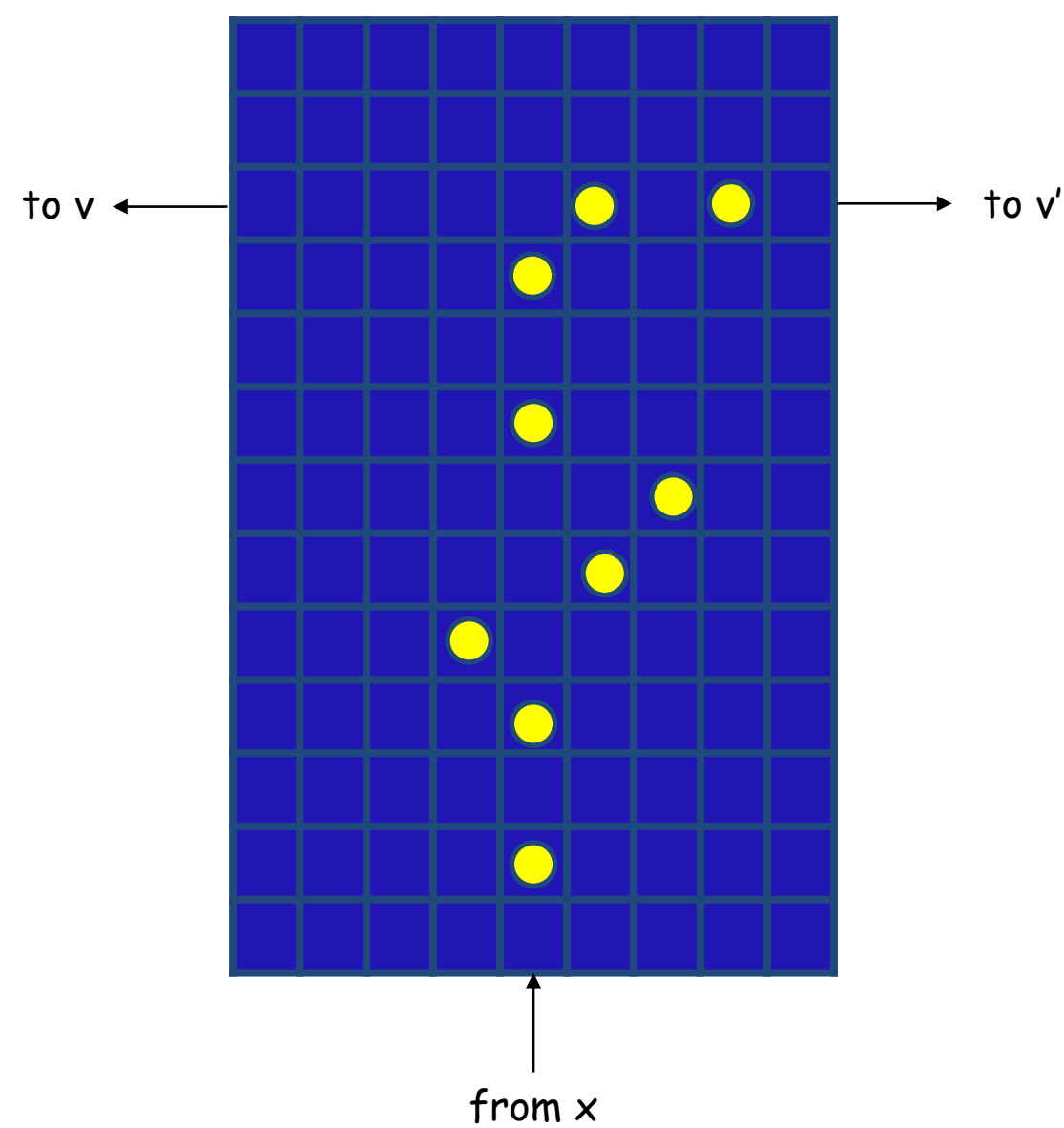
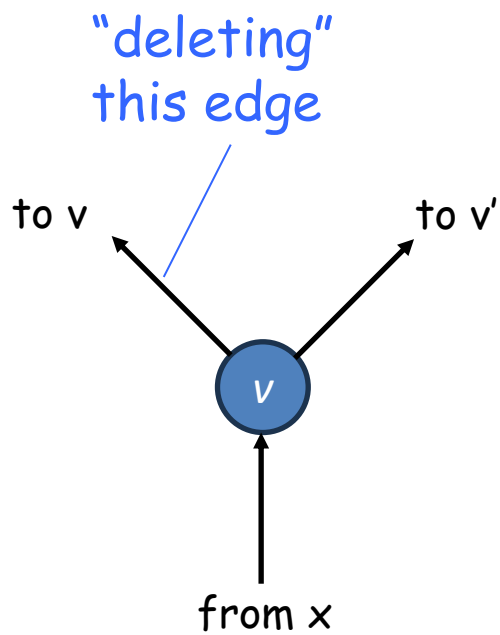
1-in 2-out degree vertex gadget: the intended behavior



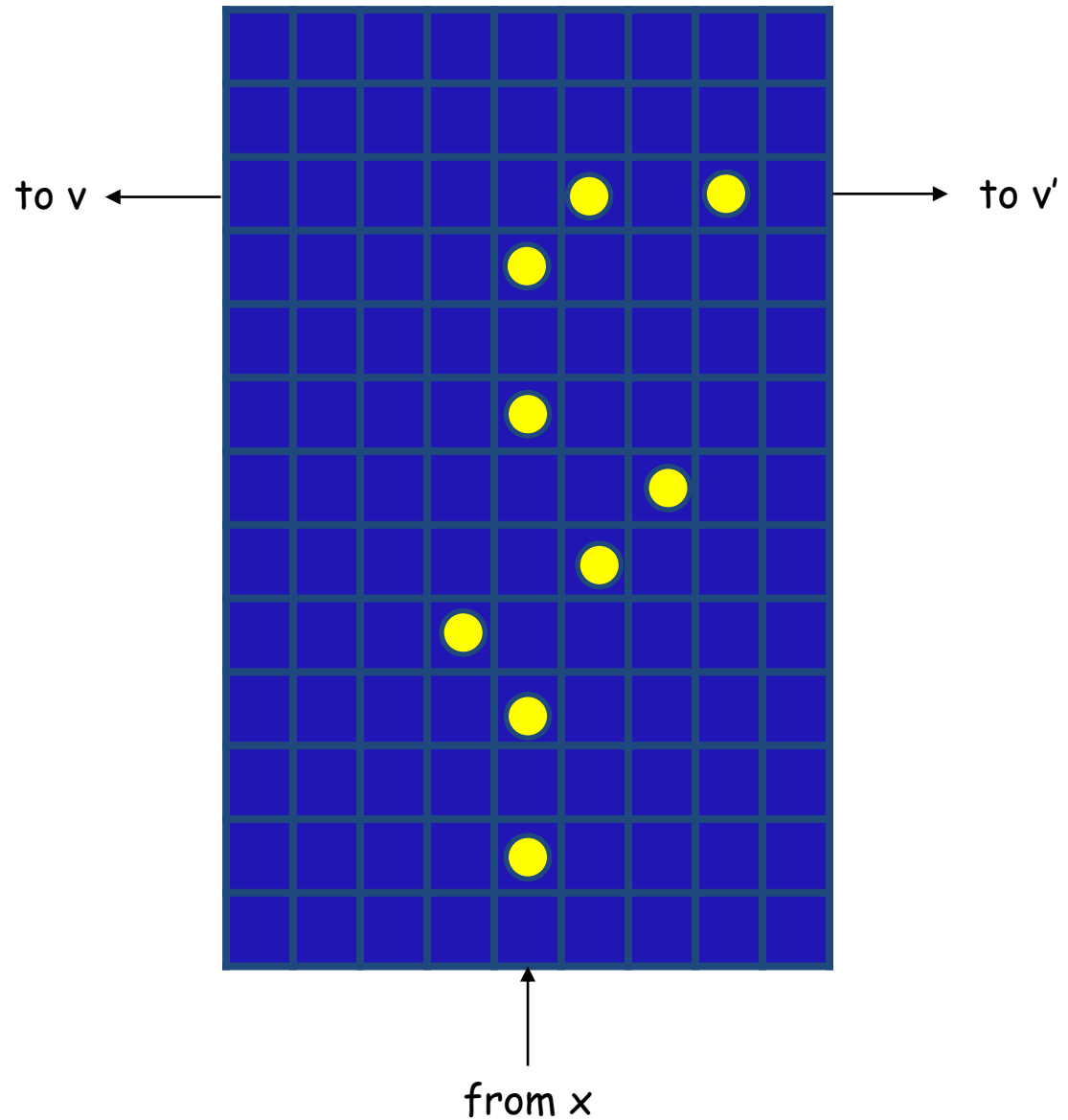
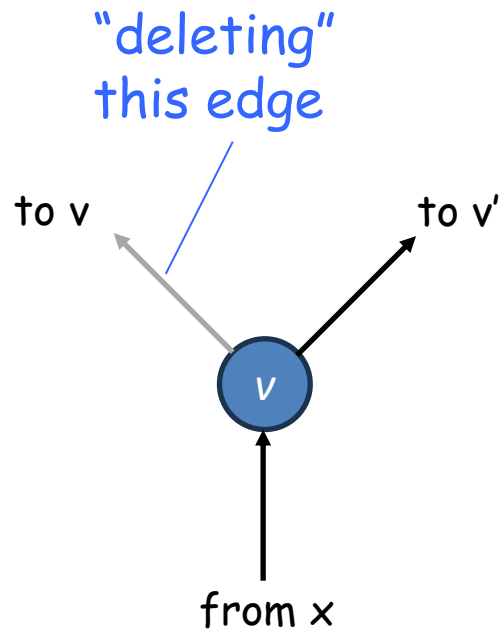
1-in 2-out degree vertex gadget: the intended behavior



1-in 2-out degree vertex gadget: the intended behavior



1-in 2-out degree vertex gadget: the intended behavior



claim:

you can clear the board if and only if G has a Hamiltonian cycle

proof

($\Leftarrow$)

if there is a Hamiltonian cycle C :

- first delete/clear the edges that do not belong to C
- clear the remaining pegs by going through C

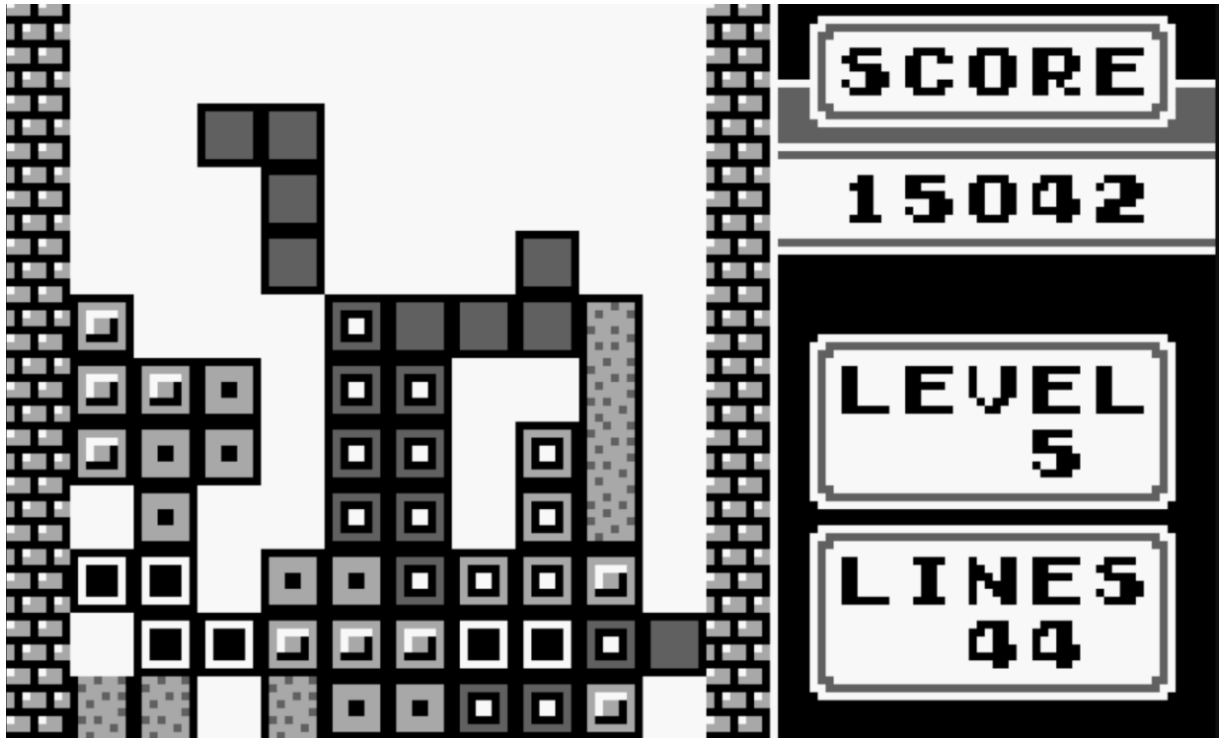
($\Rightarrow$)

intuitively: if you want to clear the board you must play the gadgets in the intended way (otherwise you loose)

Play the reduction here:

<https://www.isnphard.com/g/peg-solitaire/>

Tetris



Tetris



Tetris



Russian cosmonaut Aleksandr Serebrov became the first person to play a videogame in space when he packed a Game Boy and his personal copy of Tetris (Nintendo, 1989) for his trip to the MIR Space Station in 1993.

3-Partition problem

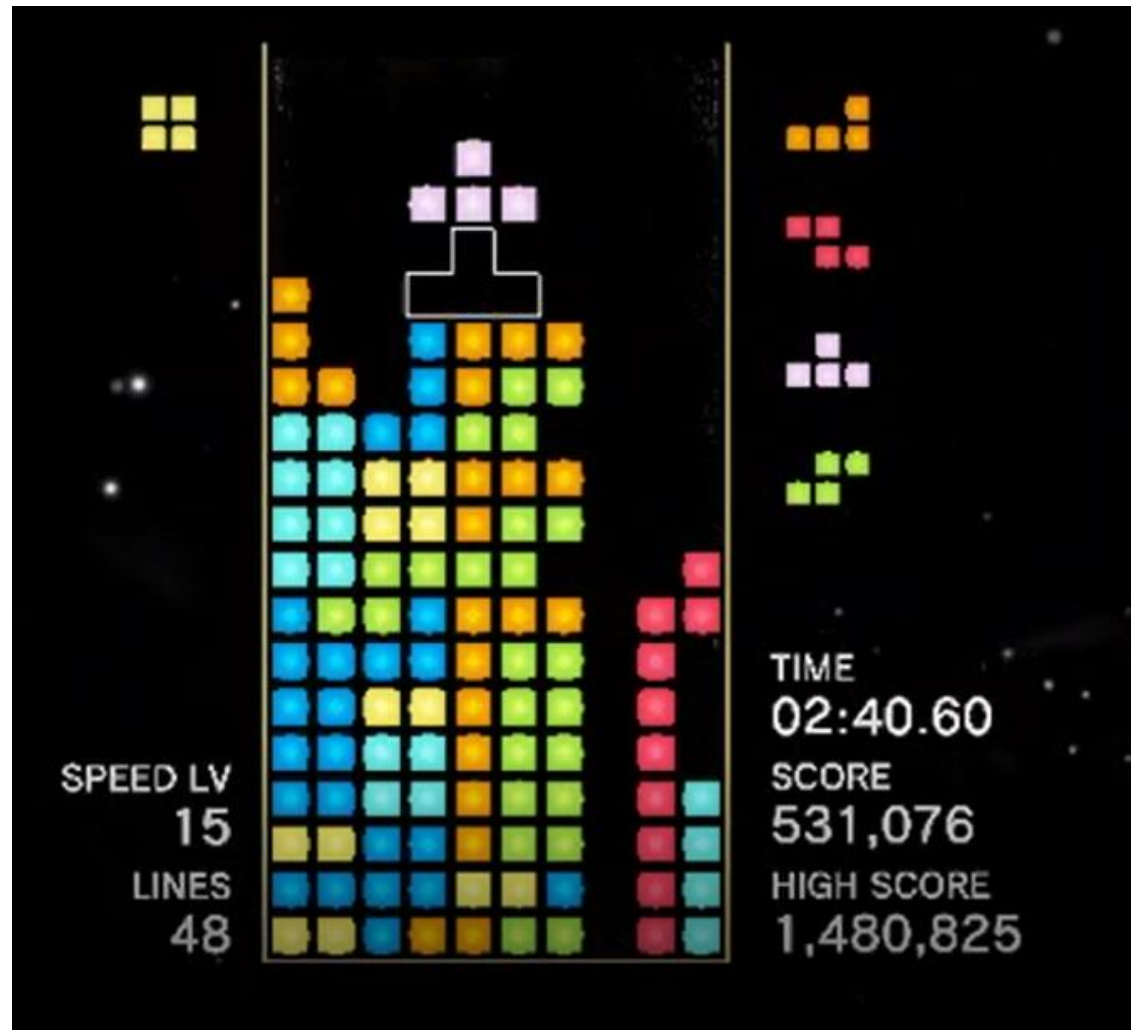
- **Input:** a collection A of n positive integers $a_1 \dots a_n$
- **question:** is it possible to **partition** A in $n/3$ collections $A_1 \dots A_{n/3}$ of equal sum, i.e.

$$\sum_{a \in A_1} a = \dots = \sum_{a \in A_{\frac{n}{3}}} a = \frac{\sum_{a \in A} a}{\frac{n}{3}} = t$$

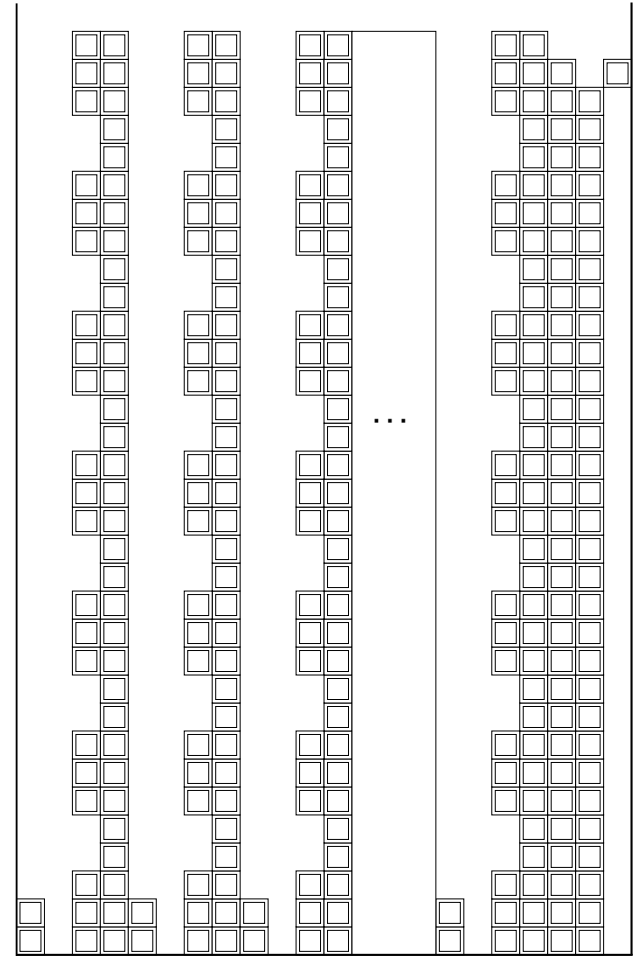
- **Fact 1:** 3-Partition is NP-complete, even if $t/4 < a_i < t/2$.
- **Obs.:** if we assume $t/4 < a_i < t/2$ we have $|A_i| = 3$, for each A_i
- **Fact 2:** 3-Partition is **strong** NP-hard, i.e. it is NP-complete even if every a_i is polynomially bounded in n (n : the number of numbers).

input: an initial configuration of the board, and the entire (offline) sequence of the pieces

goal: can you clear all the board?

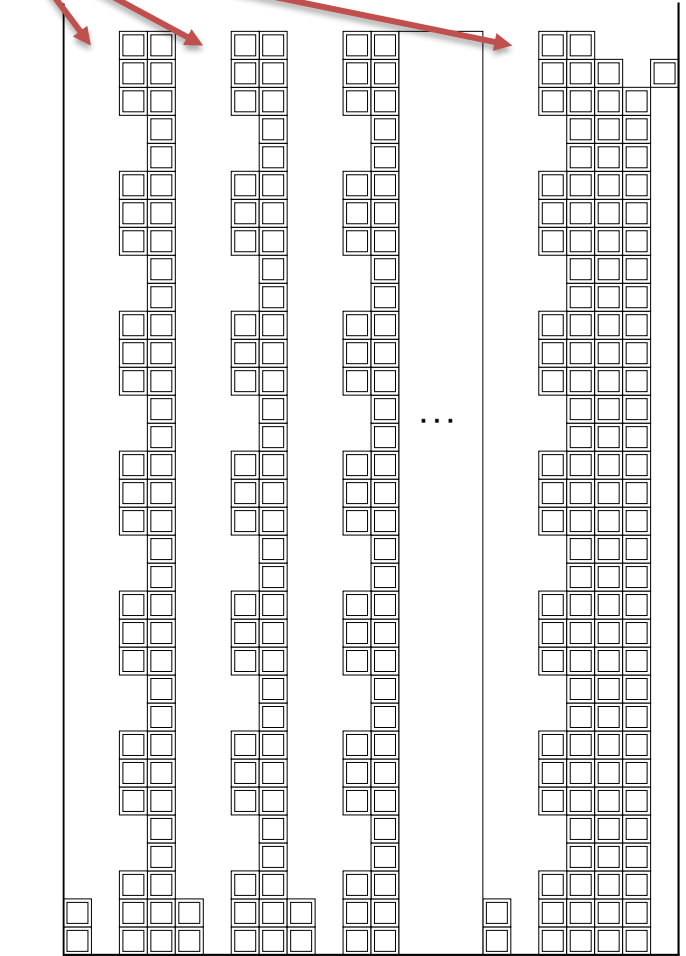


3-Partition to Tetris

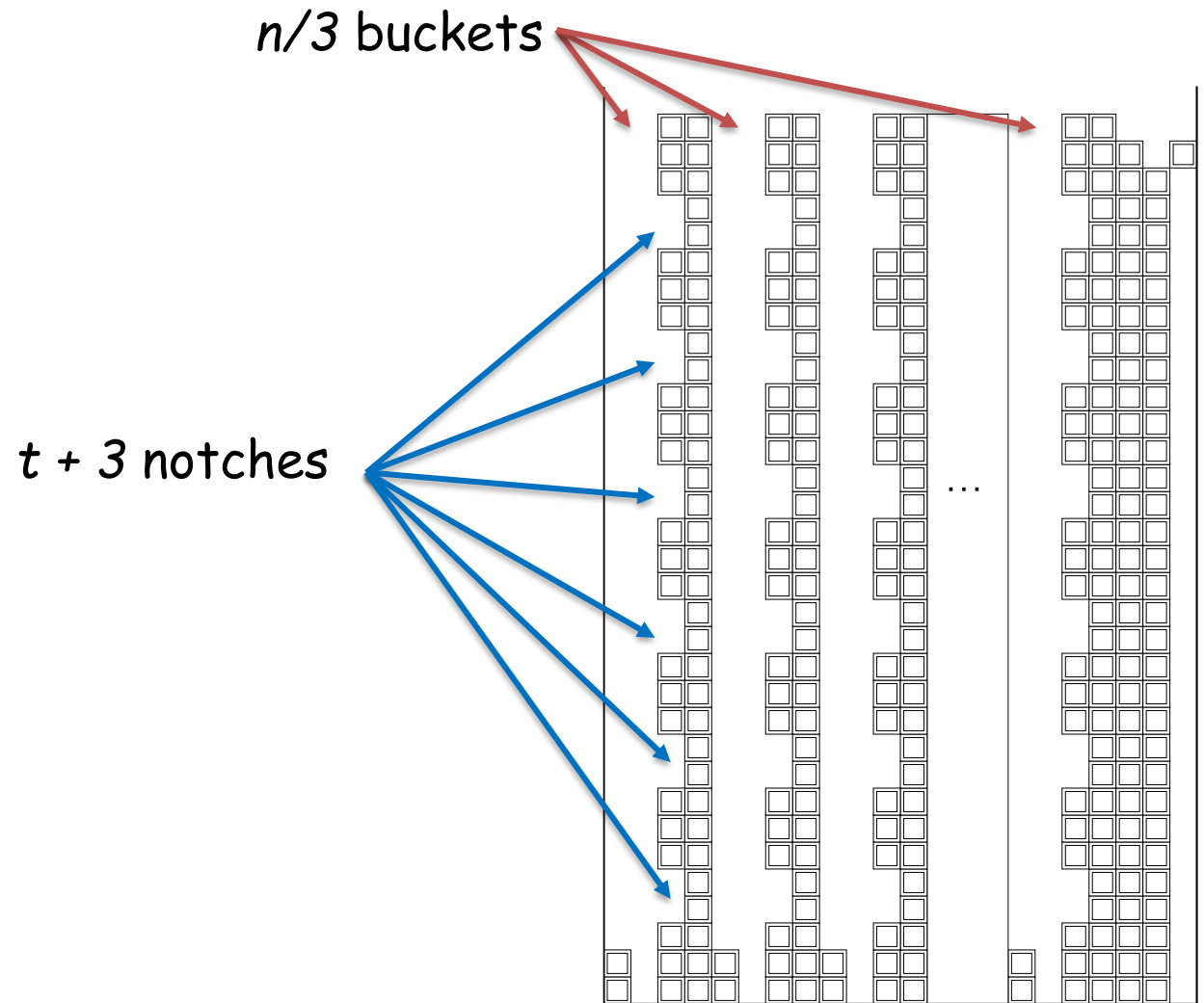


3-Partition to Tetris

$n/3$ buckets



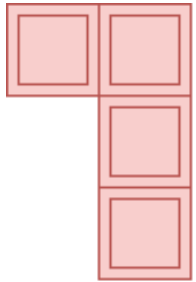
3-Partition to Tetris



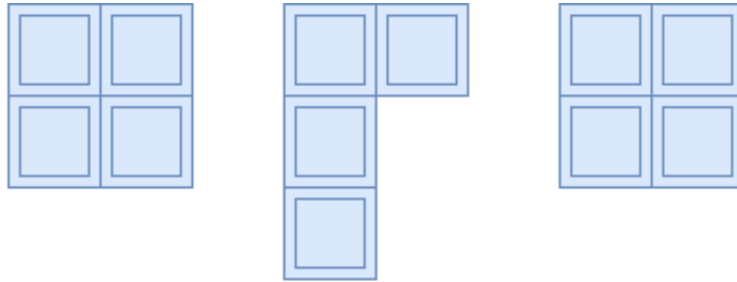
3-Partition to Tetris

a_i gadget

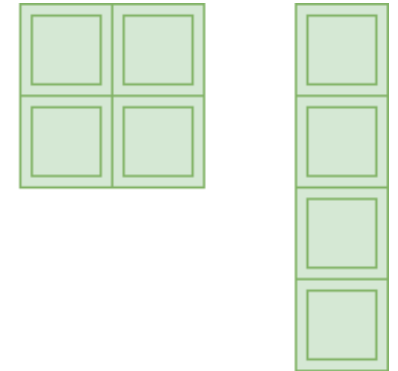
initiator



filler (a_i times)

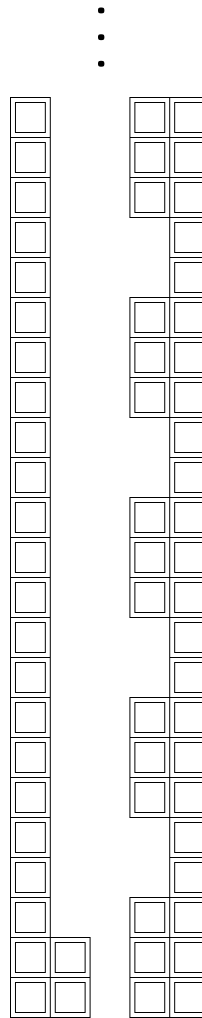


terminator



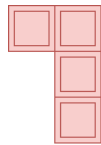
3-Partition to Tetris

$$a_i = 3$$

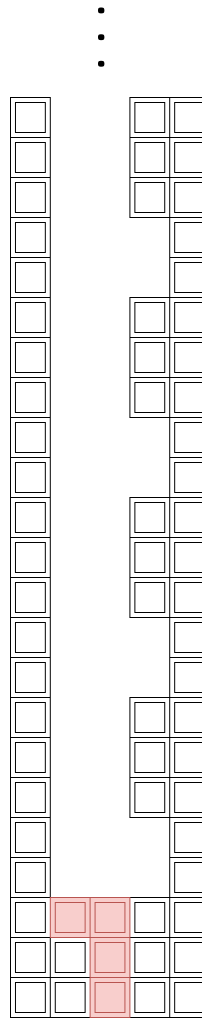


3-Partition to Tetris

$$a_i = 3$$

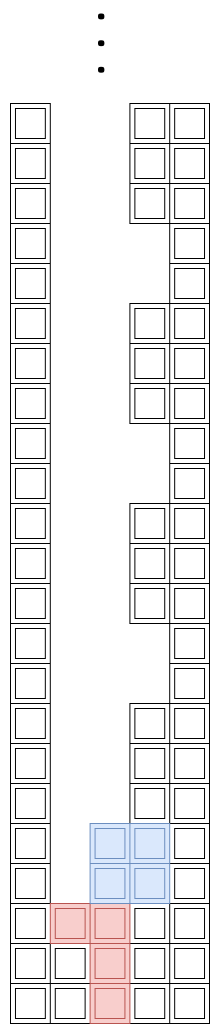


initiator



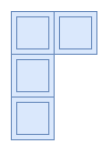
3-Partition to Tetris

$a_i = 3$

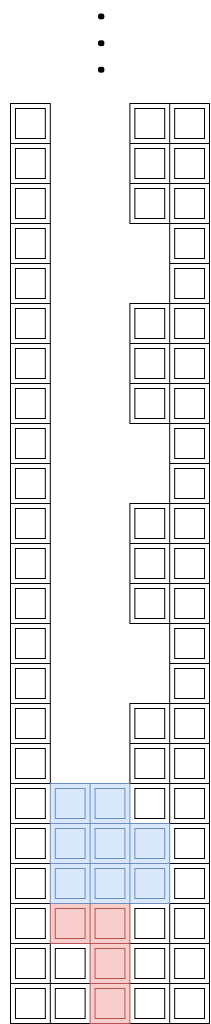


3-Partition to Tetris

$a_i = 3$

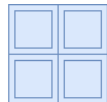


filler 1

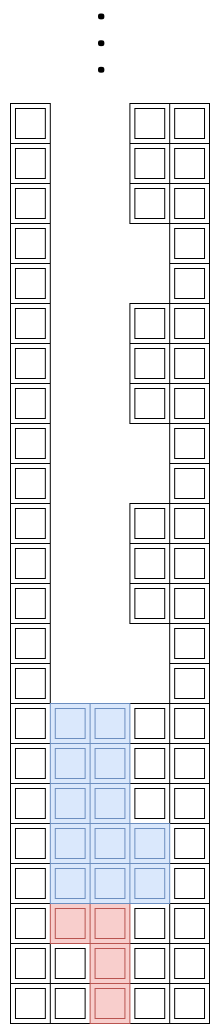


3-Partition to Tetris

$a_i = 3$

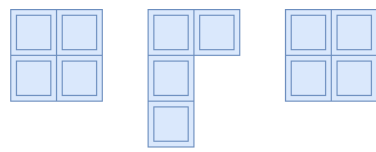


filler 1

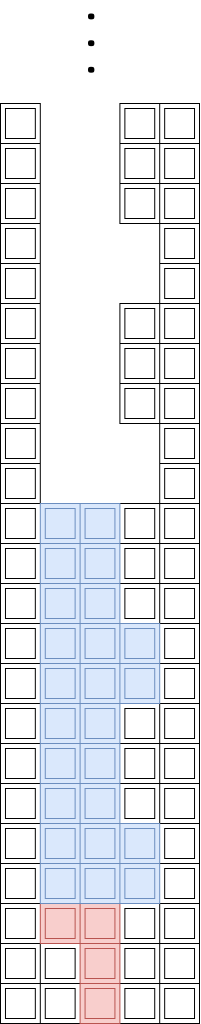


3-Partition to Tetris

$a_i = 3$

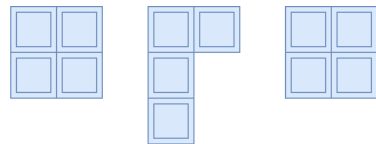


filler 2

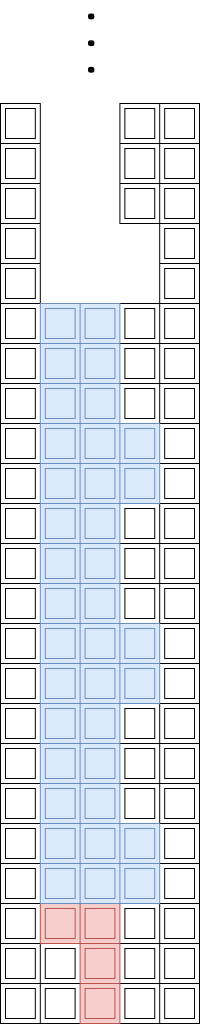


3-Partition to Tetris

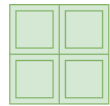
$a_i = 3$



filler 3

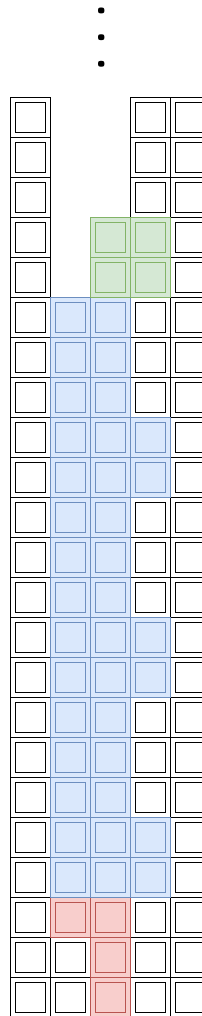


3-Partition to Tetris



terminator

$$a_i = 3$$



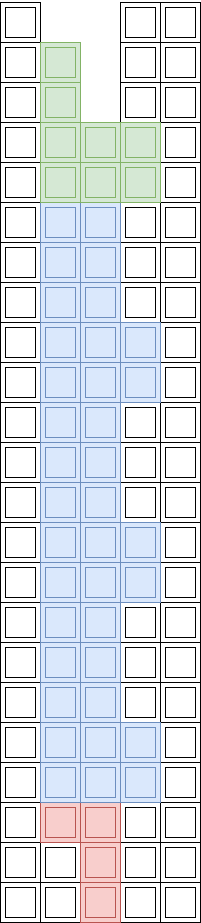
3-Partition to Tetris

$a_i = 3$



terminator

⋮

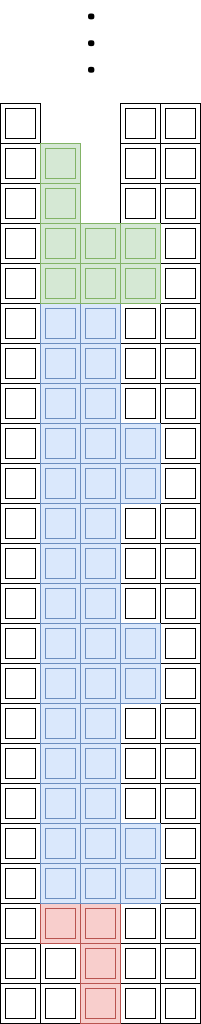


3-Partition to Tetris

$a_i = 3$



terminator



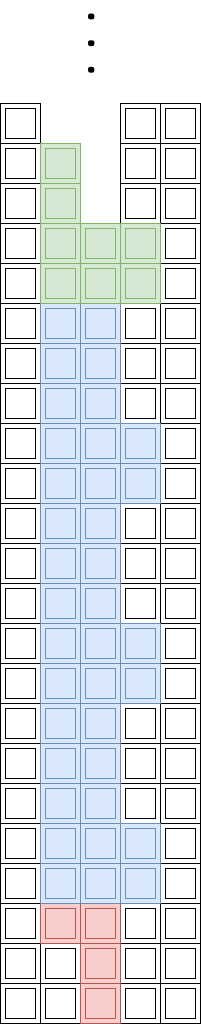
exactly $a_i + 1$
notches are used

3-Partition to Tetris

$a_i = 3$



terminator

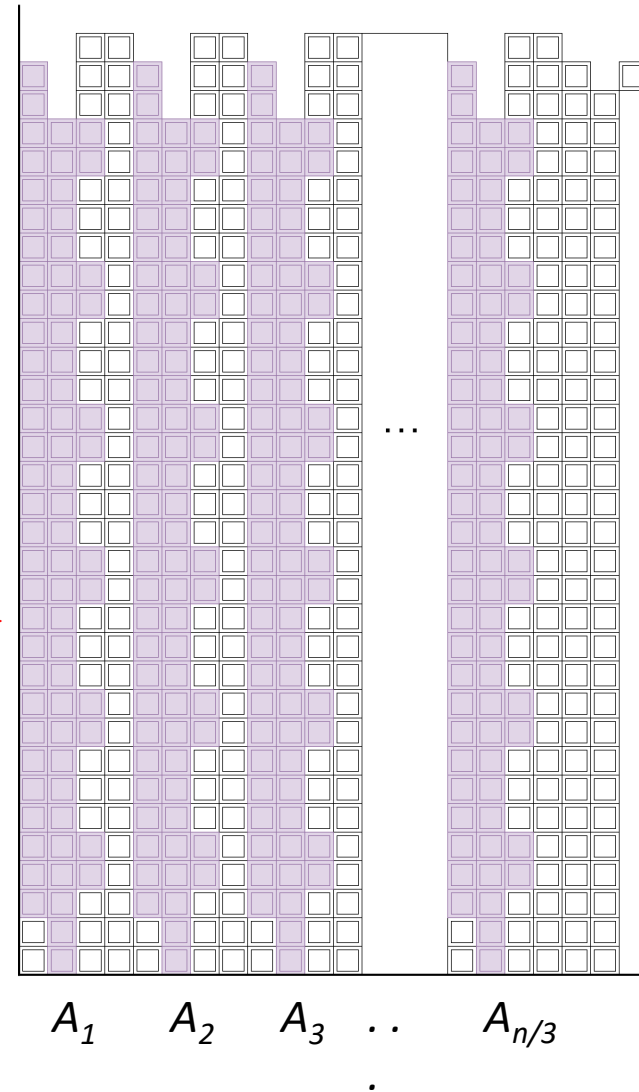


exactly $a_i + 1$
notches are used

Since each $|A_k| = 3$ and $sum(A_k) = t$ then we need $t + 3$ notches.

3-Partition to Tetris

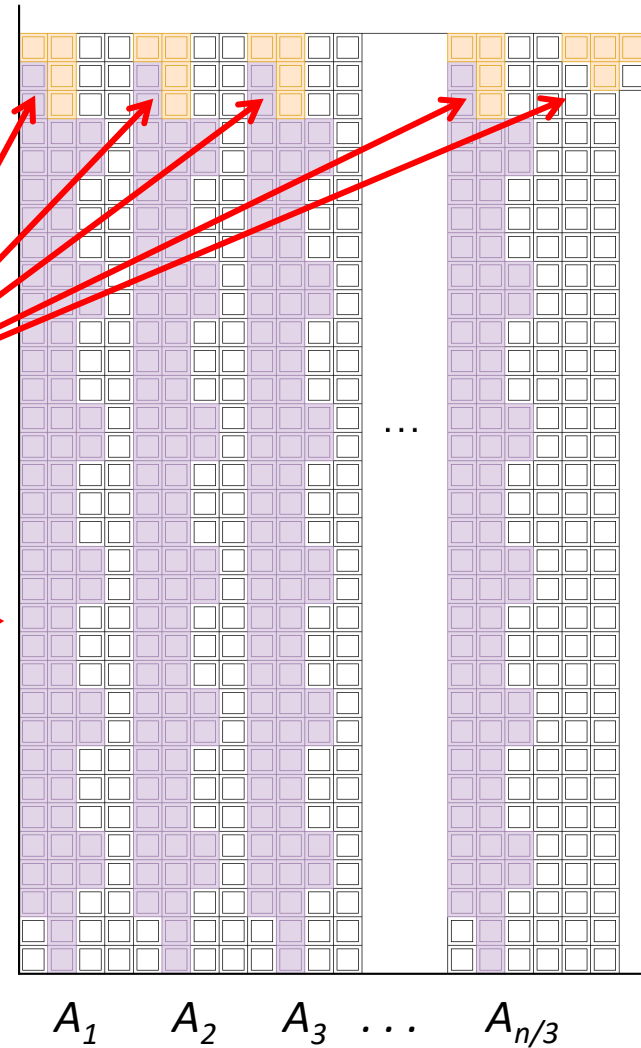
A solution $A_1 \dots A_{n/3}$ exists iff
we can feel all the $n/3$ buckets
in this way



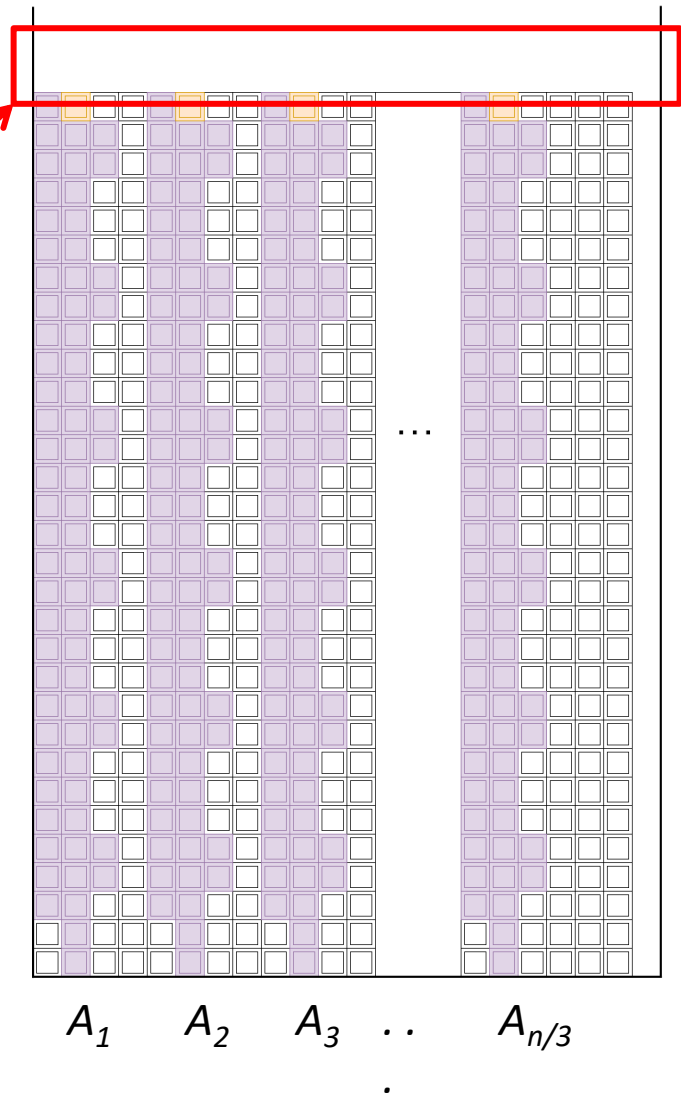
3-Partition to Tetris

A solution $A_1 \dots A_{n/3}$ exists iff
we can feel all the $n/3$ buckets
in this way

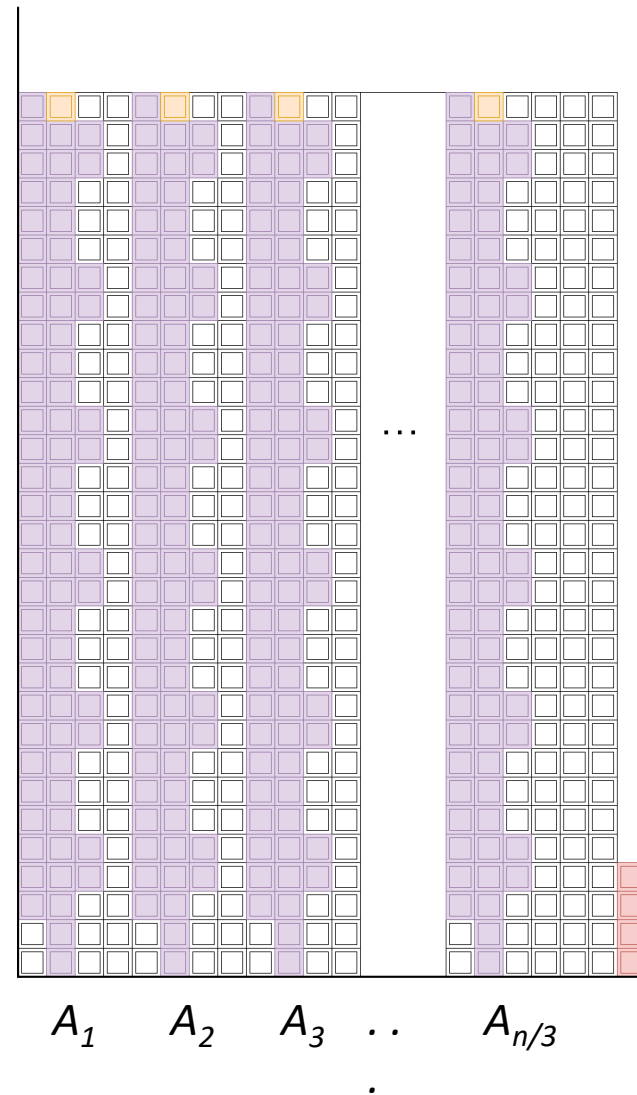
final pieces



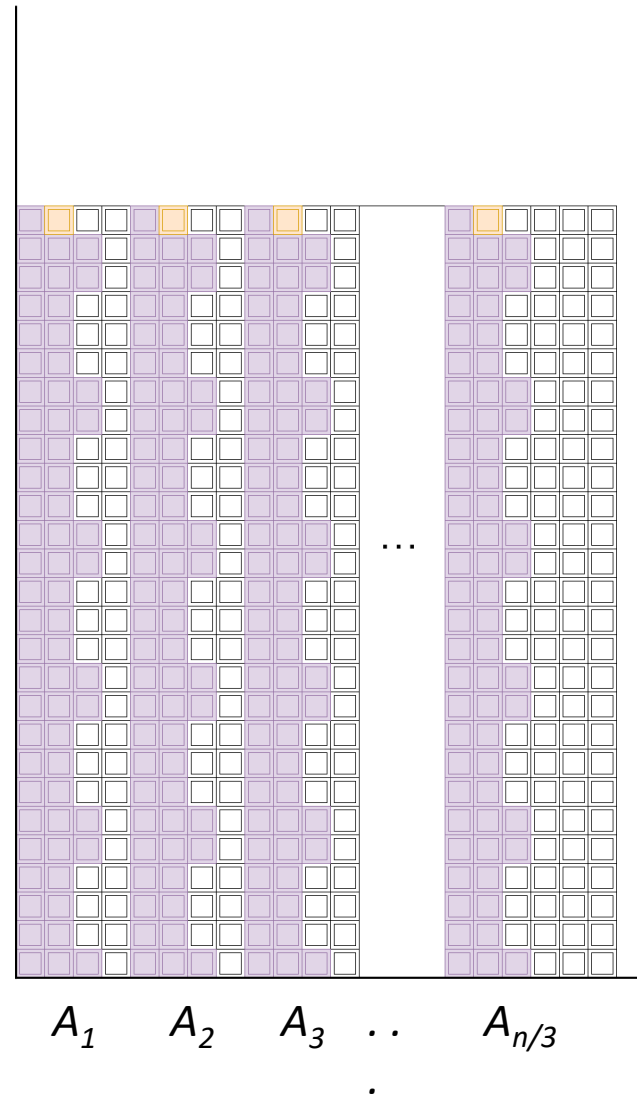
First two line disappear



3-Partition to Tetris

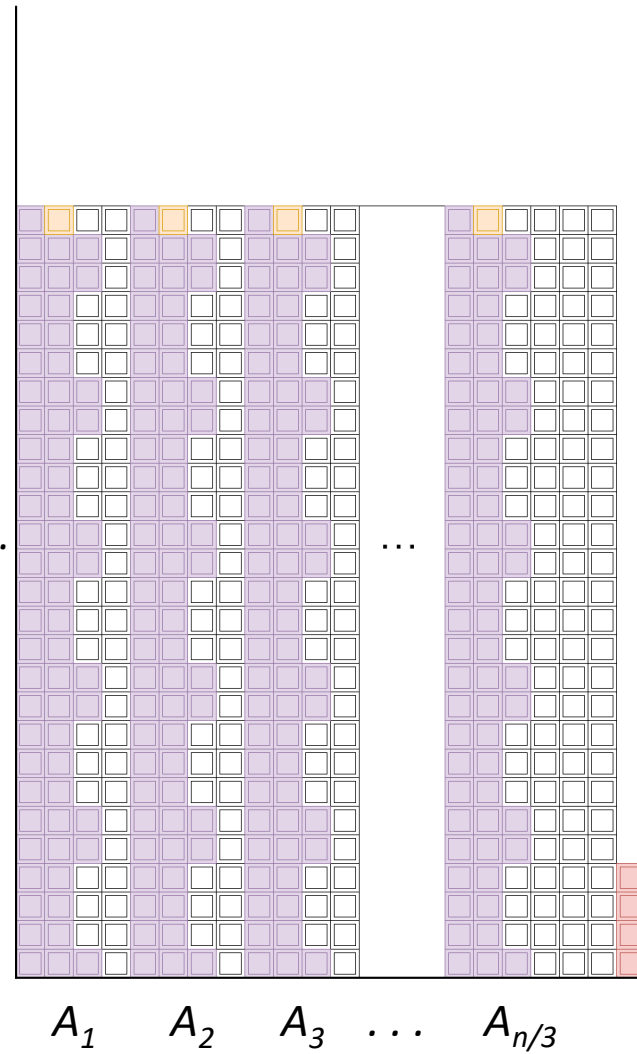


3-Partition to Tetris



3-Partition to Tetris

Continue until the entire board is clean...

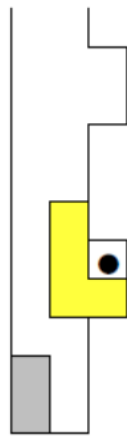


3-Partition to Tetris

If you try to position blocks in a different way into a bucket



(a)



(b) *



(c)



(d) *

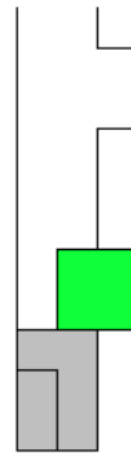
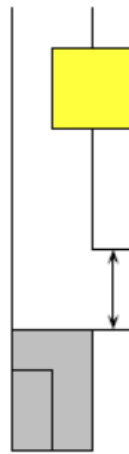
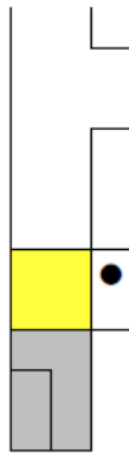


(e) *

Forced moves - initiator

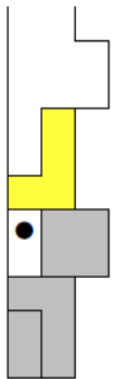
3-Partition to Tetris

Forced moves - filler (first piece)

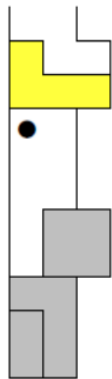


3-Partition to Tetris

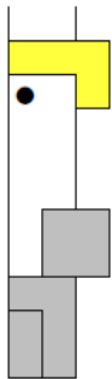
If you try to position blocks in a different way into a bucket



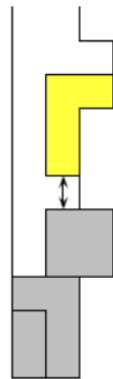
(d)



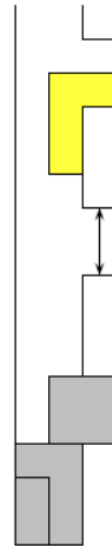
(e) *



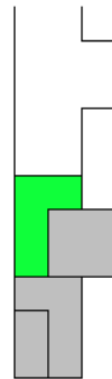
(f) *



(g)



(h) *

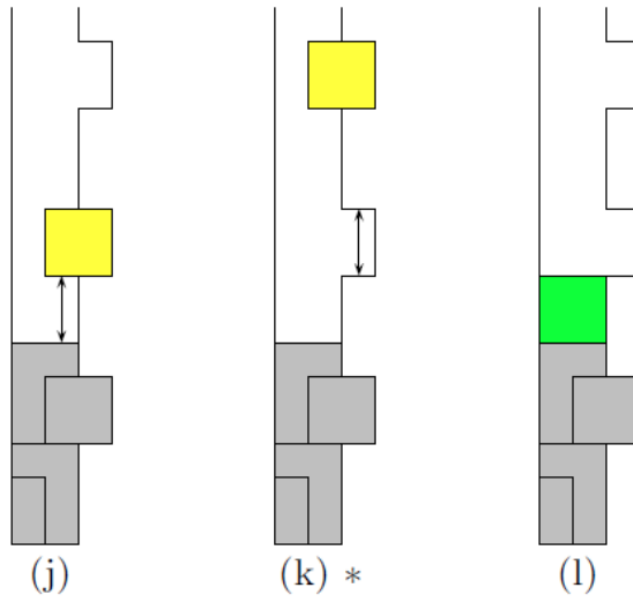


(i)

Forced moves - filler (second piece)

3-Partition to Tetris

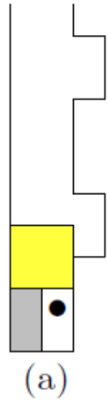
If you try to position blocks in a different way into a bucket



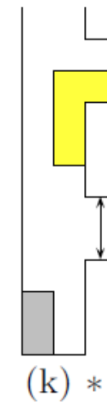
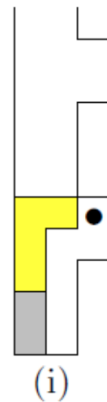
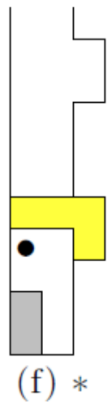
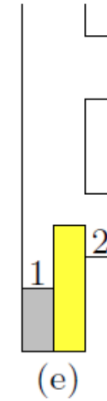
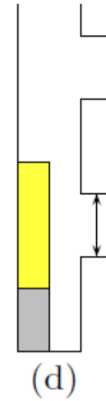
Forced moves - filler (third piece)

3-Partition to Tetris

If you try to break a element gadget



“unprimed”
buckets



6.892 Algorithmic Lower Bounds: Fun with Hardness Proofs (Spring 2019)

Prof. [Erik Demaine](#) TAs: Jeffrey Bosboom, Jayson Lynch

[\[Home\]](#) [\[Lectures\]](#) [\[Problem Sets\]](#) [\[Project\]](#) [\[Coauthor\]](#) [\[Accessibility\]](#)

Overview

Need to figure out when to give up the search for efficient algorithms?

Want to know why Tetris and Mario are computationally intractable?

Love seeing the connections between problems and how they can be transformed into each other?

Like solving puzzles that can turn into publishable papers?

This class takes a practical approach to proving problems can't be solved efficiently (in polynomial time and assuming standard complexity-theoretic assumptions like $P \neq NP$). We focus on reductions and techniques for proving problems are computationally hard for a variety of complexity classes. Along the way, we'll create many interesting gadgets, learn many hardness proof styles, explore the connection between games and computation, survey several important problems and complexity classes, and crush hopes and dreams (for fast optimal solutions).

The ability to show a problem is computationally hard is a valuable tool for any algorithms designer to have. Lower bounds can tell us when we need to turn to weaker goals or stronger models of computation, or to change the problem we're trying to solve. Trying to find lower bounds can help us see what makes a problem difficult or what patterns we might be able to exploit in an algorithm. The hardness perspective can help us understand what makes problems easy, or difficult to solve.

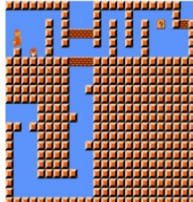
Inverted Lectures

This year, we're experimenting with inverted lectures: most material is covered in [video lectures](#) recorded in 2014 (already watched by over 14,000 people), which you can conveniently play at faster speed than real time. In-class time will be focused on **in-class problem solving**, with some **new material** presented by the professor and/or guest lecturers. Particularly unusual is that the problems we'll solve in groups will include a choose-your-own-mix of **problem-set style** problems with known solutions, **coding** problems for those who love programming, and **open research problems** that no one knows the answer to, with the goal of publishing papers about whatever we discover. (The past offering of this class led to over a dozen published papers.) You can work on whatever type of problem most interests you. To facilitate collaboration, we'll be using a new [open-source software platform](#) called [Coauthor](#), along with [Github](#) for (optional) coding.

Topics

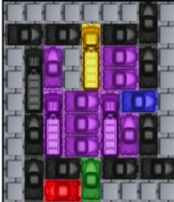
This is an advanced class on algorithmic reduction. We will focus on techniques for proving problems are complete with respect to various complexity classes, not on the complexity theory itself. Here is a tentative list of topics:

ALGORITHMIC LOWER BOUNDS: FUN WITH HARDNESS PROOFS



Super Mario Bros.

Crossover gadget for NP-hardness



Rush Hour

AND gadget for PSPACE-hardness



Minesweeper

OR gadget for NP-hardness

Hardness Made Easy*

Learn when to give up the search for efficient algorithms; see **connections** between computational problems; **solve puzzles** to prove theorems, solve **open problems**, and write papers.

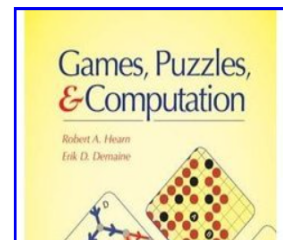
Topics: NP, PSPACE, EXPTIME, EXPSPACE, JSUM, approximation, fixed parameter, games & puzzles, key problems, gadgets, and proof styles.



6.892 taught by Professor Erik Demaine
II, AAGS, and Theoretical CS Concentration
Wednesday 7:00-9:30pm in room 32-082
<http://courses.csail.mit.edu/6.892/spring19/>
sign up for our mailing list to join the class

Spring 2019

*Business not guaranteed. Side effects such as open problems and a heightened sense of complexity may occur. Ask your advisor if 6.892 is right for you!



MIT course (video lectures available):
<https://courses.csail.mit.edu/6.892/spring19/>

Compendium

Here is the list of games and puzzles that are currently in our index.

Is your favorite game missing? Are you aware of a new complexity result for one of the listed games? You are welcome add or edit the listed games by following the instructions on [this page](#).

15-puzzle ($n^2 - 1$.puzzle)

$n^2 - 1$ numbered tiles can be slid in a $n \times n$ board with the goal of arranging them in increasing order.

Amazons

Two players move amazons on a square board. After moving, an amazon shoots an arrow that blocks movement. The last player to move wins.

Bejeweled

A player swaps adjacent items in a $n \times m$ grid in order to form as many matches of three as possible.

Boulder Dash

A single-player game in which the character digs through a rectangular grid to find diamonds within a time limit, while avoiding various dangers.

Candy Crush

A variant of Bejeweled.

Clickomania (SameGame)

A single player game in which the player removes groups of tiles of the same color in a rectangular board.

Deflektor

compendium on hardness for games and puzzles:
<https://www.isnphard.com/i/>

